MAI 5100: Fundamentals of AI

Week 2: Problem Solving by Searching

Dr. Christopher Clarke · University of Guyana · 31 October 2026

Announcements

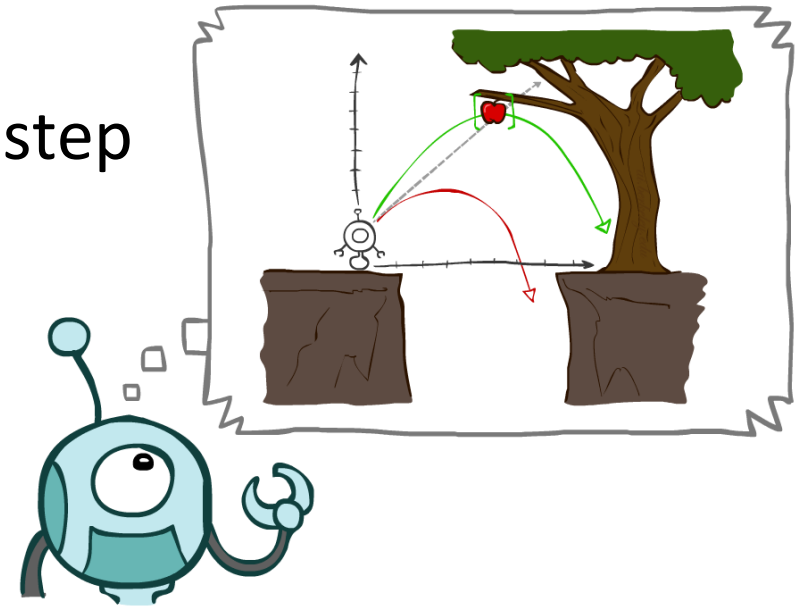
- **Homework 0 (Search)** is released today on GitHub: hw/hw0
  - Due **Saturday, 21 November, 11:59 PM** Guyana time
  - Pac-Man Q1–Q3 (DFS, BFS, UCS) use today's material; A* and heuristics follow on 14 November
- **No class next Saturday (7 November)**
- Next lecture (14 November): informed search, heuristics, and A*
- Reading: Russell & Norvig, **Chapter 3.1–3.4**

Recap: Week 1

- AI as **designing rational agents**: act to maximize expected utility
- An agent **perceives** its environment through sensors and **acts** through actuators
- Properties of the environment (observable? deterministic? static? discrete?) determine which techniques fit
- Today's setting: fully observable, deterministic, static, discrete, single agent
  - In this setting, an agent can find a whole plan by **searching** before it acts

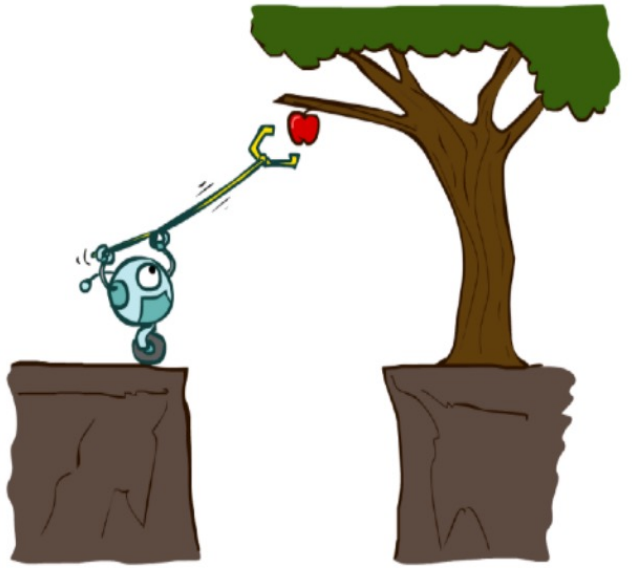
Today

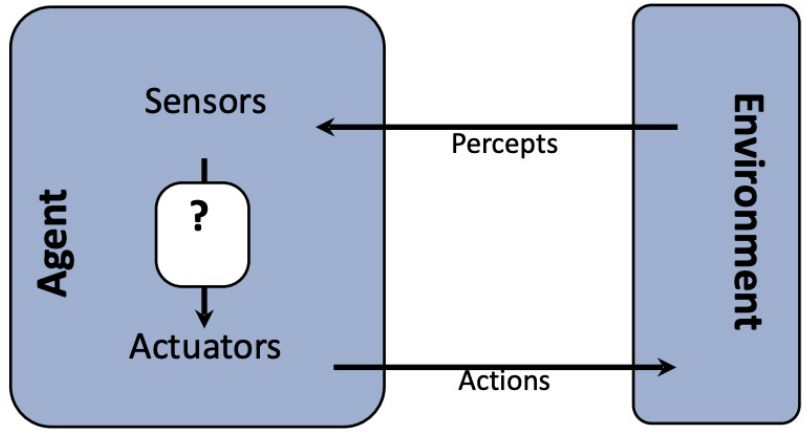
- Intelligent agents: rationality, PEAS, environments, agent types
- Agents that plan ahead
- Search problems (worked example: water jugs)
- Uninformed search methods, each with a step-by-step worked example
  - Depth-first search
  - Breadth-first search
  - Iterative deepening
  - Uniform-cost search
- Practice problem and Homework 0



Intelligent Agents

- An **agent** is anything that **perceives** its environment through **sensors** and **acts** on it through **actuators**
  - Physical: robots, self-driving cars, drones
  - Software: chatbots, recommender systems, game AI, trading bots
- The **agent function** maps the percept history to an action
- The **agent program** implements that function on a real machine
- Designing an agent means understanding the **task**, the **environment**, and the **actions** available





Good Behavior: The Concept of Rationality

- A **rational agent** chooses the action that is expected to **maximize its performance measure**, given:
  - what it has perceived so far (the percept sequence)
  - what it knows about the environment
  - the actions it can perform
- Rational ≠ omniscient: it maximizes **expected** performance, not actual outcome
- Rational ≠ perfect: agents should **gather information** and **learn** when they can
- The **performance measure** defines success. Choose it carefully: reward what you want achieved, not how you think the agent should behave

Specifying the Task Environment: PEAS

Before designing an agent, describe its task environment.

Example: an automated taxi

P	**Performance measure** How do we judge success?	Safe, fast, legal, comfortable trip; maximize profit
E	**Environment** Where does the agent operate?	Roads, traffic, pedestrians, weather, passengers
A	**Actuators** How does it act?	Steering, accelerator, brake, signals, horn, display
S	**Sensors** How does it perceive?	Cameras, lidar, radar, GPS, speedometer, odometer

PEAS Examples

Agent	Performance	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, lower costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments	Touchscreen or voice entry of symptoms and findings
Crop-monitoring drone	Early pest detection, area covered, battery use	Farms, weather, crops, livestock	Rotors, camera gimbal, alerts to farmers	Cameras, multispectral sensor, GPS, altimeter
Pac-Man agent	Score: eat food, avoid ghosts, finish fast	Maze, food pellets, ghosts	Move N, S, E, W, or Stop	Game state: positions of walls, food, ghosts
Customer-support chatbot	Issues resolved, customer satisfaction, cost	Customers, product data, ticket system	Text replies, ticket updates, hand-off to a human	Customer messages, account records

Quick exercise: write the PEAS description for a GPS navigation app used in Georgetown traffic.

Properties of Task Environments

Property	Question to ask	Example
Fully vs. partially observable	Can the sensors see the complete relevant state?	Chess vs. poker
Deterministic vs. stochastic	Is the next state completely determined by the current state and action?	Chess vs. weather forecasting
Episodic vs. sequential	Does the current decision affect future decisions?	Spam filter vs. chess
Static vs. dynamic	Can the world change while the agent is deliberating?	Crossword vs. stock market
Discrete vs. continuous	Are states, time, and actions countable?	Chess vs. robot navigation
Single vs. multi-agent	Are other agents acting in the environment?	Crossword vs. soccer

The hardest case: partially observable, stochastic, sequential, dynamic, continuous, multi-agent (e.g. driving a taxi).

Classify These Environments

Task	Observable	Deterministic	Episodic	Static	Discrete	Agents
Chess						
Poker						
Stock market						
Robot navigation						
Email spam filter						
Soccer						
8-puzzle						

Work with a neighbour for 5 minutes, then we fill it in together.

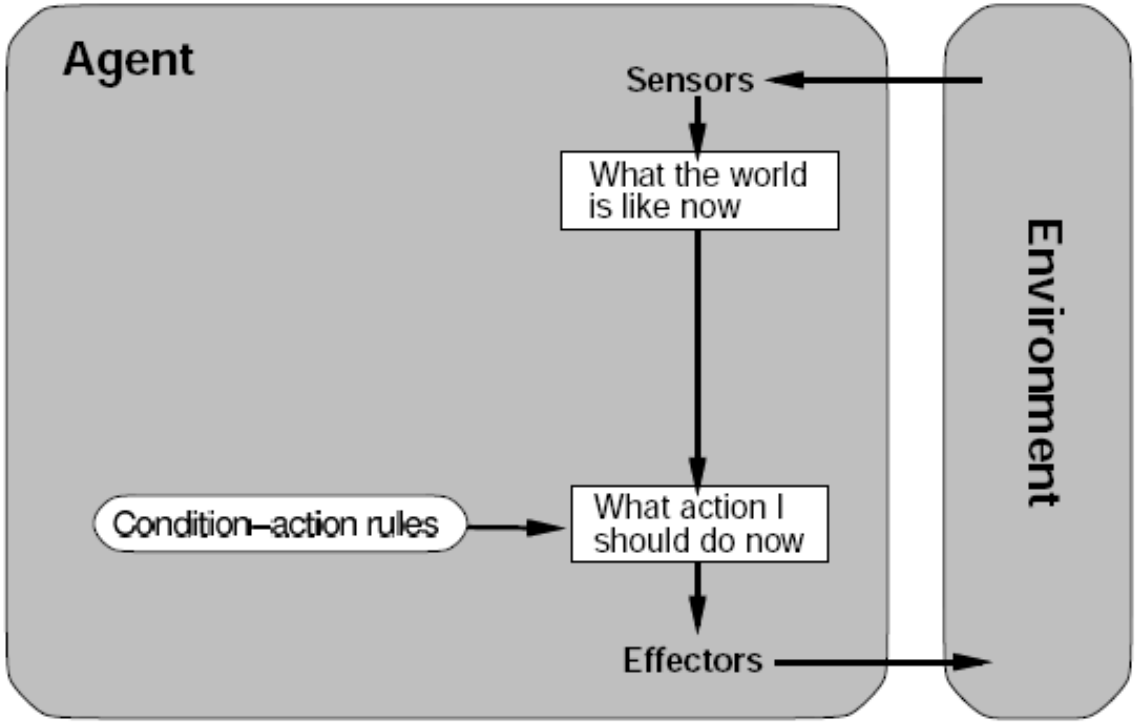
Classify These Environments: Answers

Task	Observable	Deterministic	Episodic	Static	Discrete	Agents
Chess	Fully	Deterministic	Sequential	Static	Discrete	Multi
Poker	Partially	Stochastic	Sequential	Static	Discrete	Multi
Stock market	Partially	Stochastic	Sequential	Dynamic	Continuous	Multi
Robot navigation	Partially	Stochastic	Sequential	Dynamic	Continuous	Single
Email spam filter	Partially	Stochastic	Episodic	Static	Discrete	Single
Soccer	Partially	Stochastic	Sequential	Dynamic	Continuous	Multi
8-puzzle	Fully	Deterministic	Sequential	Static	Discrete	Single

Several answers are debatable: what matters is the reasoning. The 8-puzzle row is the setting for today's search algorithms.

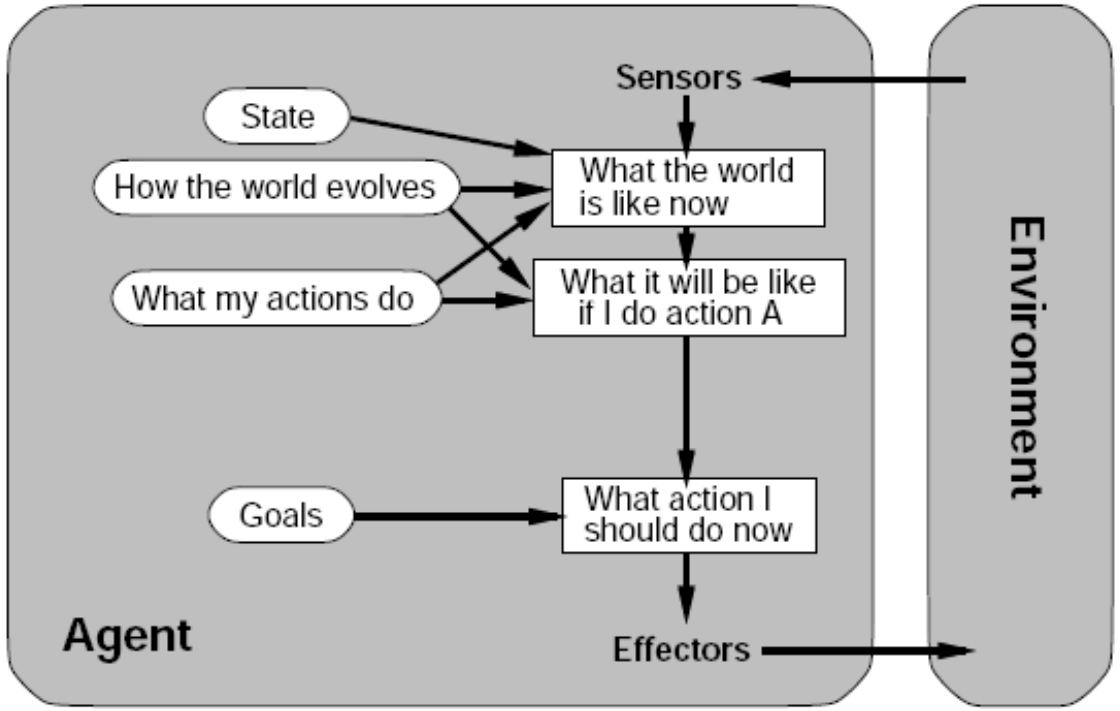
Types of Agents: Simple Reflex

- Choose an action using only the **current percept** and **condition–action rules**
  - "If the car in front is braking, then brake"
- Simple and fast, but only works if the environment is **fully observable**
- Example: a thermostat; a spam filter keyed on a list of banned words



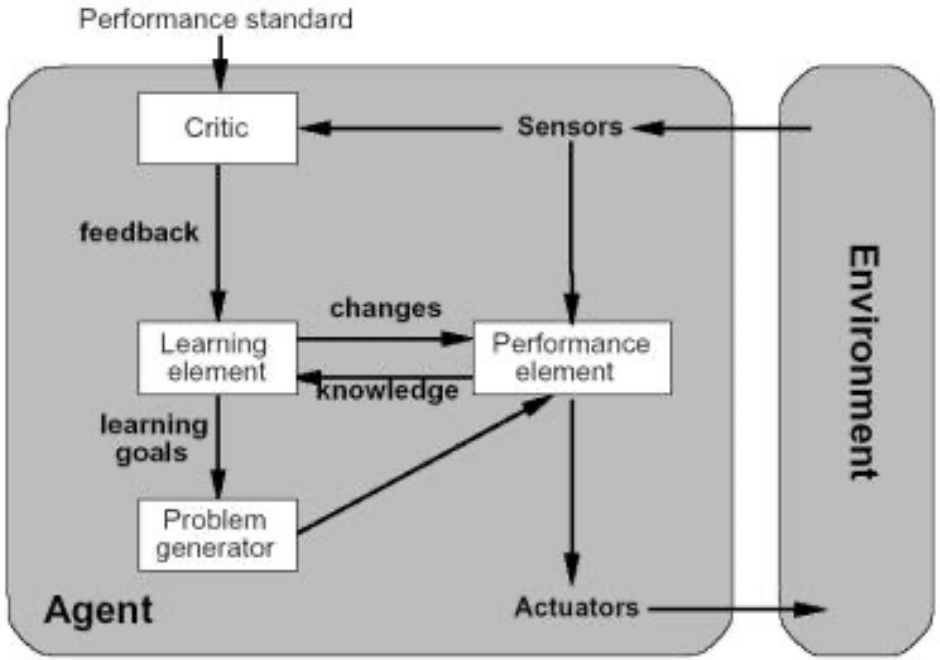
Model-Based and Goal-Based Agents

- **Model-based reflex:** keep an internal **state** that tracks the parts of the world it cannot currently see
  - Needs a model of how the world evolves and what its actions do
- **Goal-based:** ask "what will happen if I do action A?" and pick actions that reach a **goal**
  - This is where **search and planning** come in: today's topic
- Example: a navigation app planning a route to your destination



Utility-Based and Learning Agents

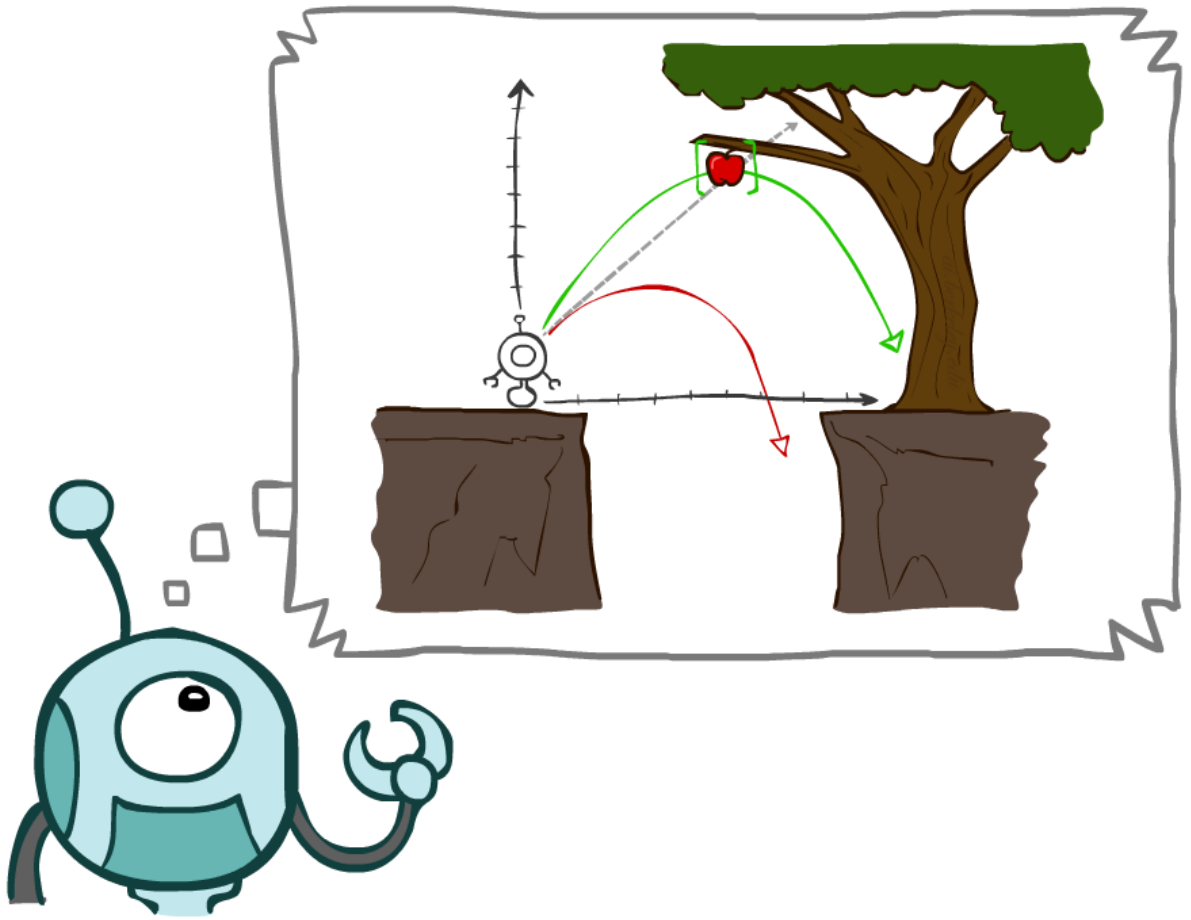
- **Utility-based:** goals are not enough when there are trade-offs. A **utility function** scores how good each outcome is
  - A self-driving car balancing safety, speed, and comfort
- **Learning agents** improve with experience:
  - Critic gives feedback against a performance standard
  - Learning element updates the agent; problem generator suggests exploration
- Course roadmap: goal-based (search) → utility-based (MDPs) → learning (RL, ML)



Problem-Solving Agents

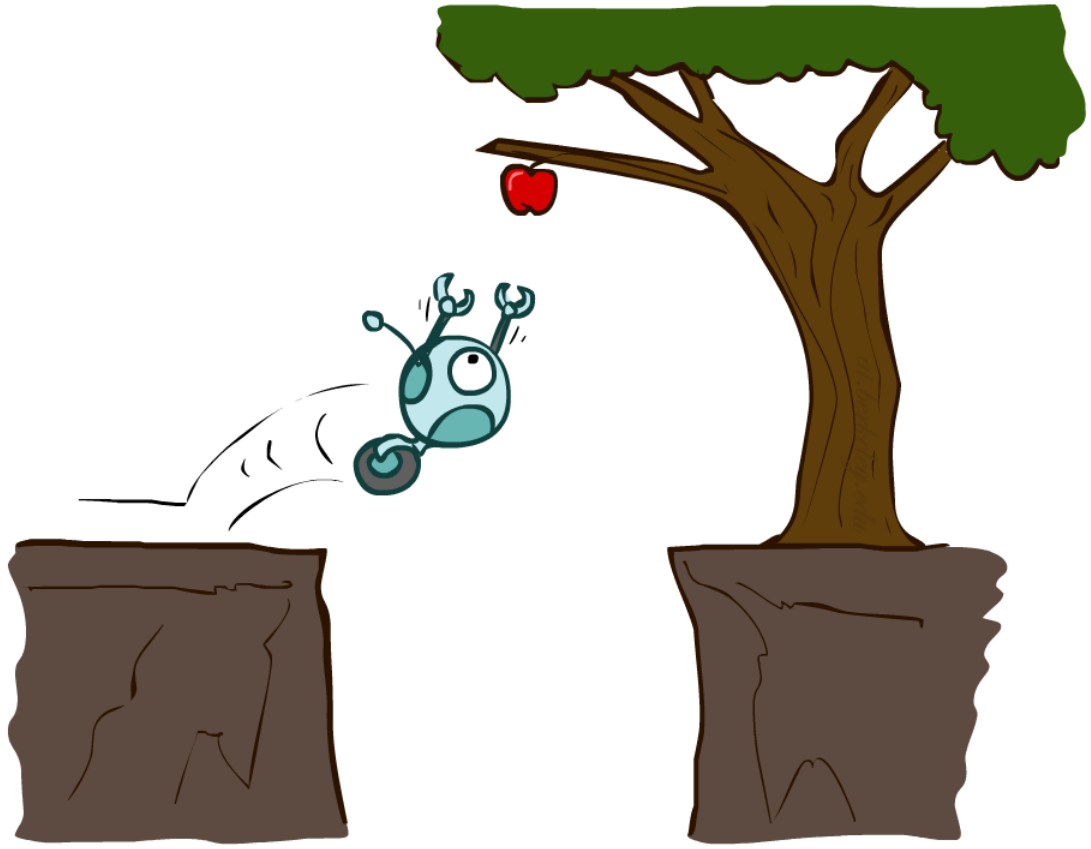
- When the right action is not obvious, a goal-based agent must **plan ahead**: consider a sequence of actions that leads to a goal
- Such an agent is a **problem-solving agent**; the computation it does is **search**
- We assume the environment is **fully observable, deterministic, static, discrete, and known**
  - Then a solution is simply a fixed sequence of actions
- Four steps: **formulate the goal** → **formulate the problem** → **search** for a solution → **execute** it

Agents that Plan



Reflex Agents

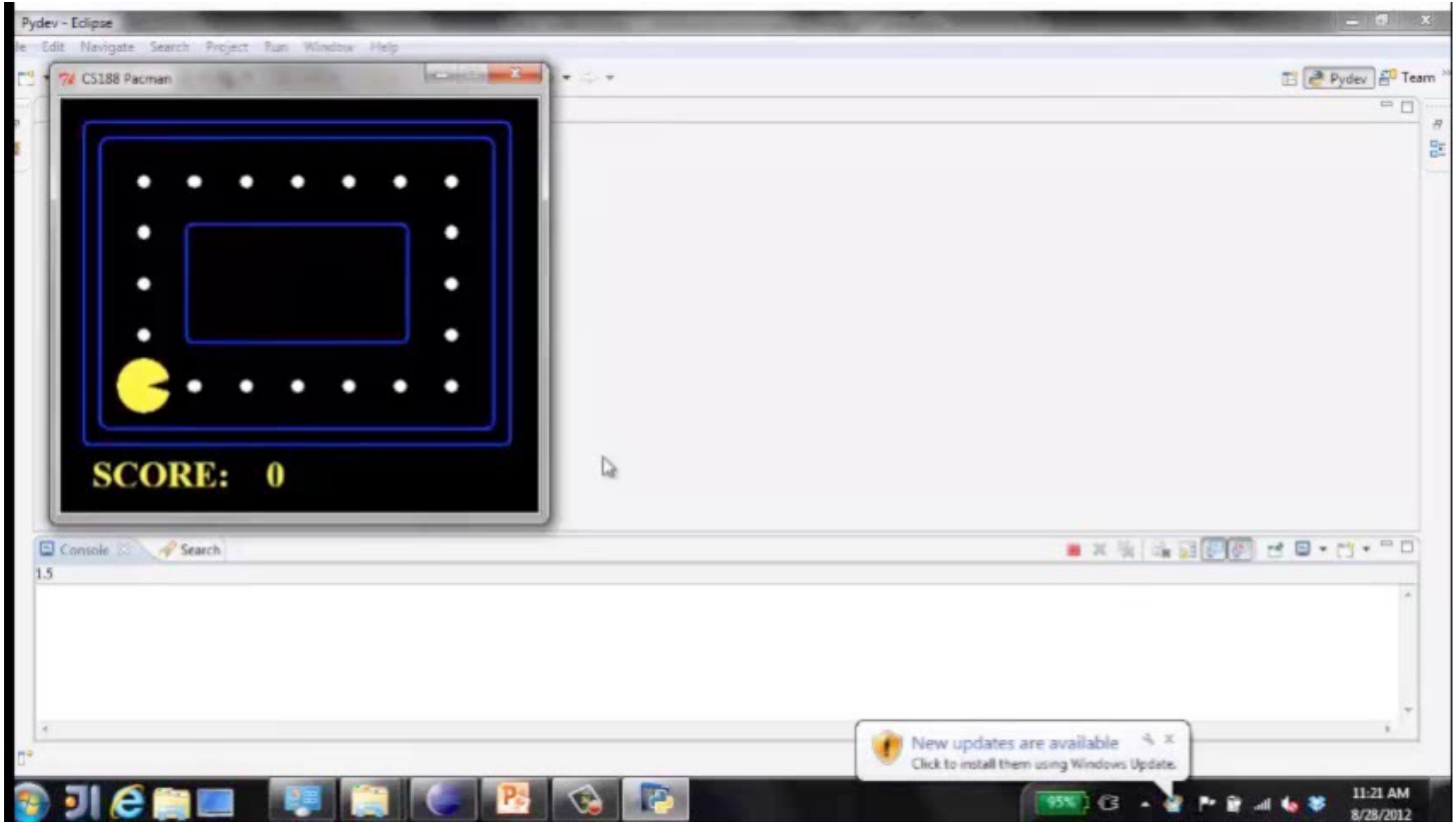
- Reflex agents:
  - Choose action based on current percept (and maybe memory)
  - May have memory or a model of the world's current state
  - Do not consider the future consequences of their actions
  - Consider how the world IS
- Can a reflex agent be rational?



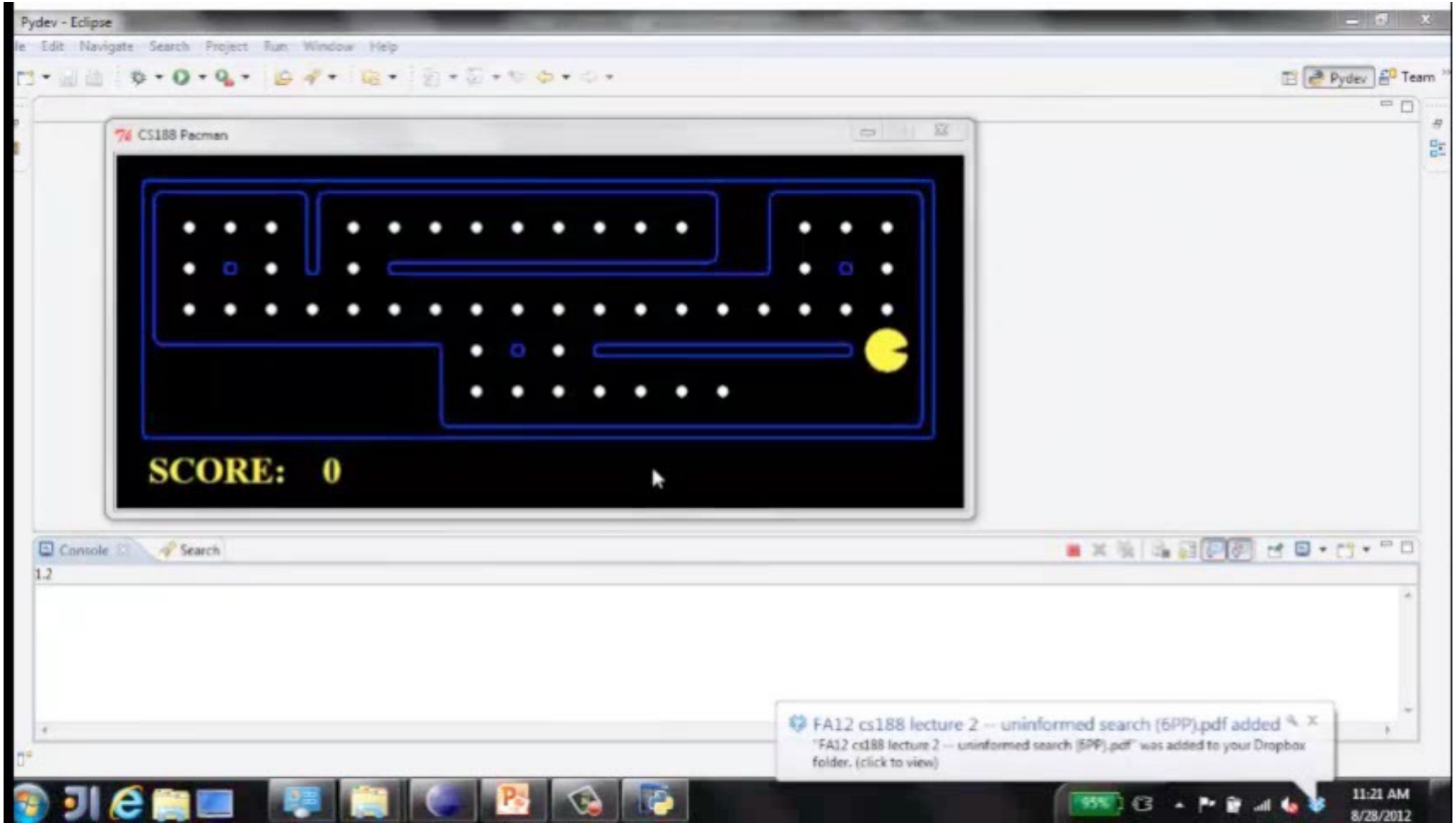
[Demo: reflex optimal (L2D1)]

[Demo: reflex optimal (L2D2)]

Video of Demo Reflex Optimal

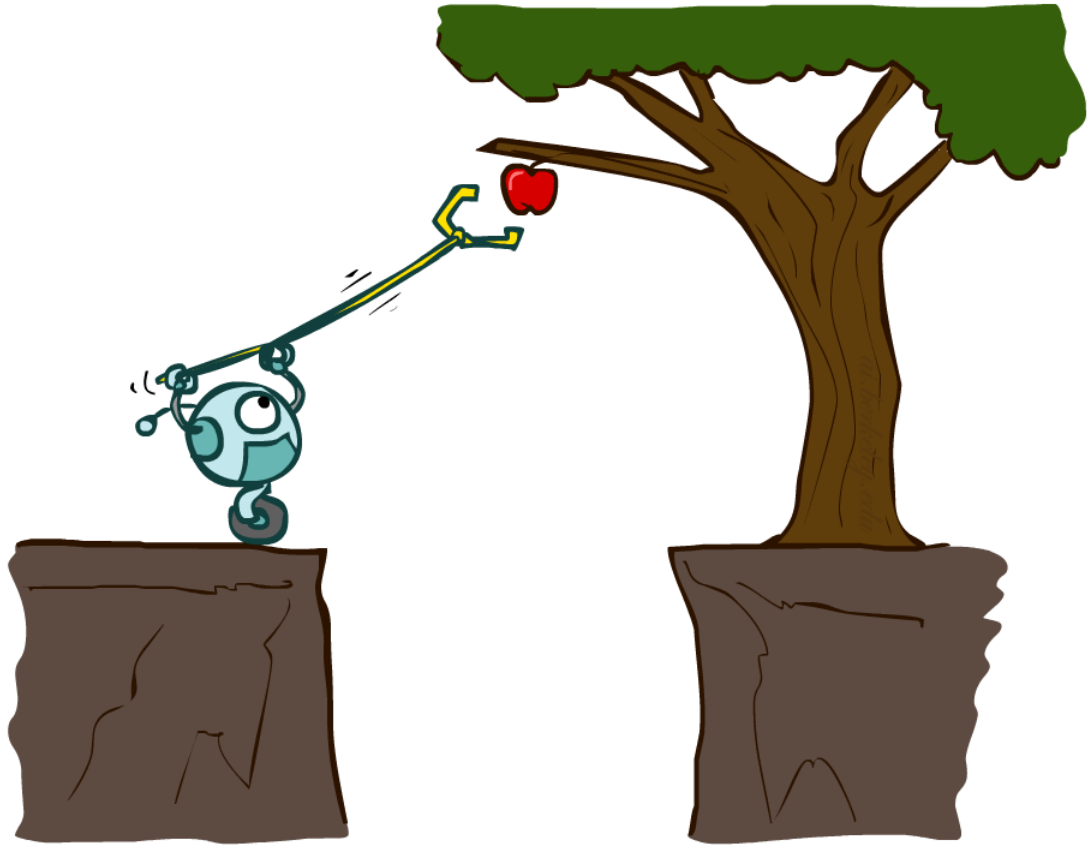


Video of Demo Reflex Odd



Planning Agents

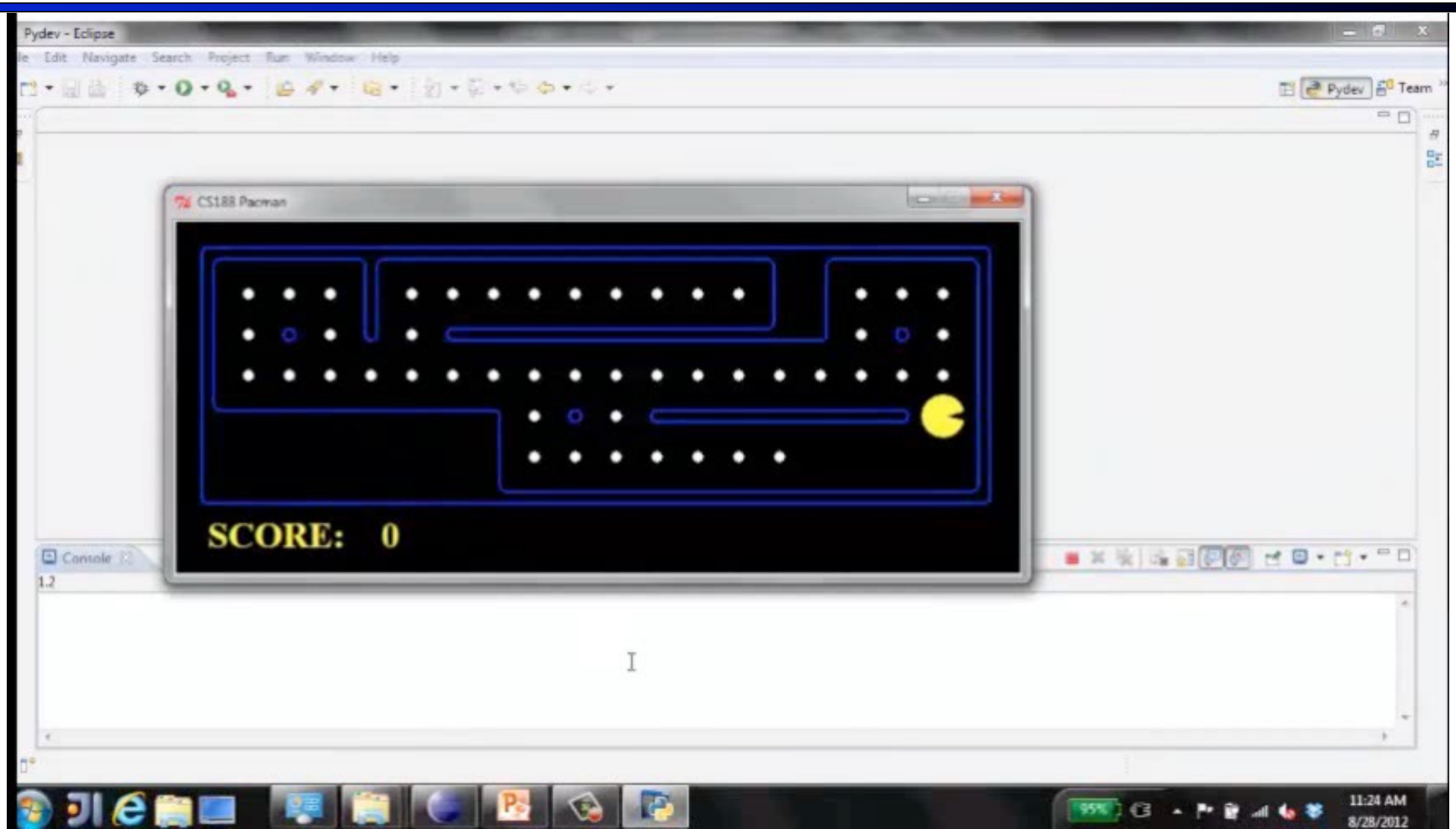
- Planning agents:
  - Ask “what if”
  - Decisions based on (hypothesized) consequences of actions
  - Must have a model of how the world evolves in response to actions
  - Must formulate a goal (test)
  - Consider how the world WOULD BE
- Optimal vs. complete planning
- Planning vs. replanning



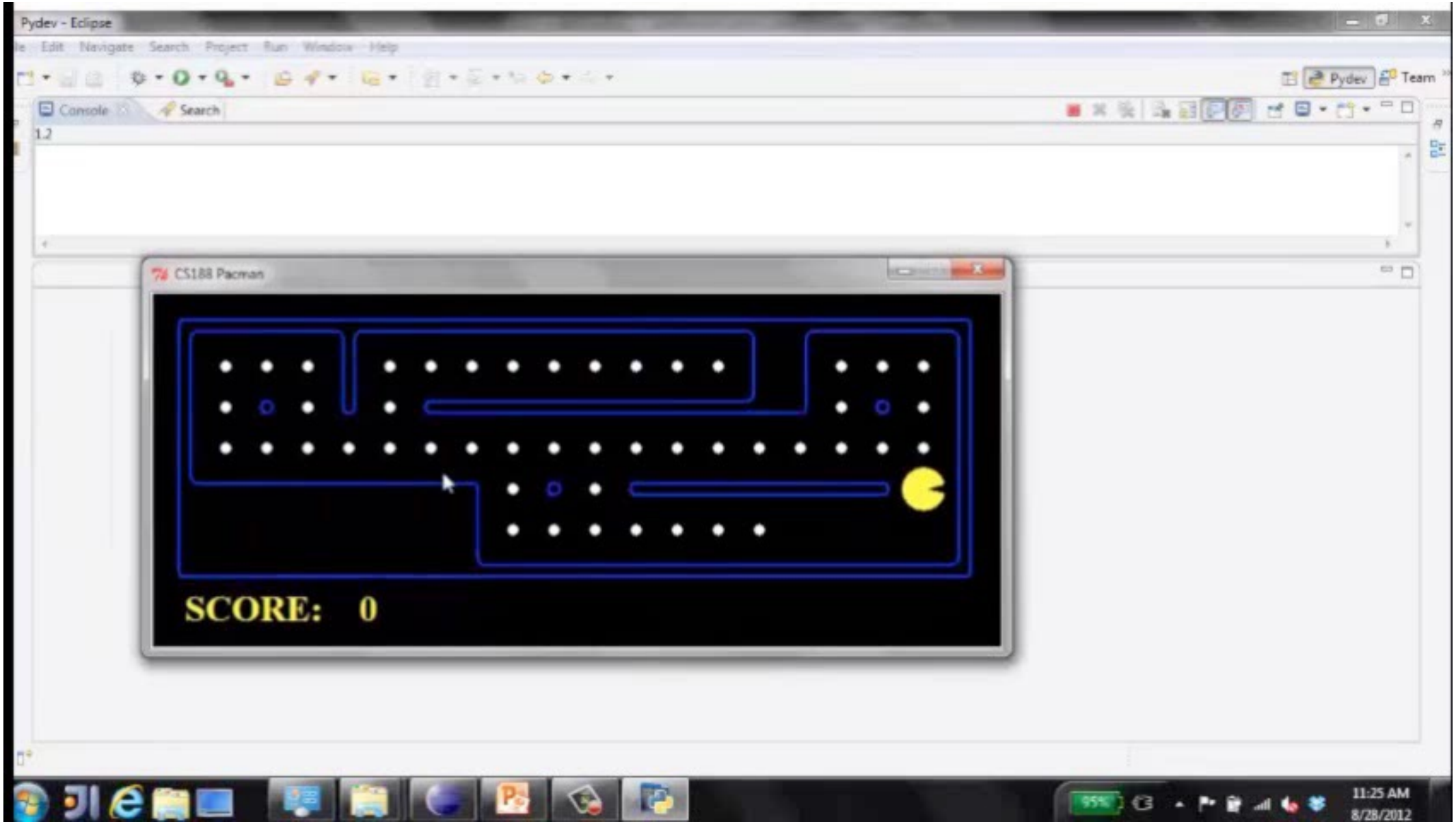
[Demo: replanning (L2D3)]

[Demo: mastermind (L2D4)]

Video of Demo Replanning



Video of Demo Mastermind

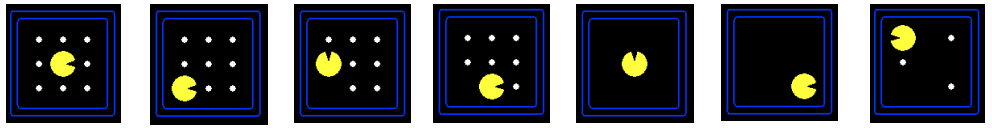


Search Problems

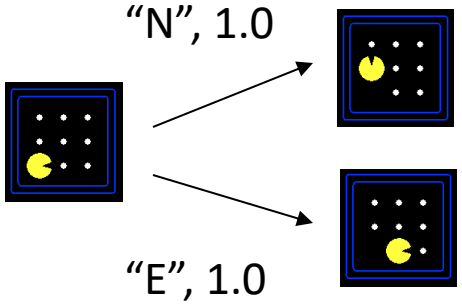


Search Problems

- A search problem consists of:
  - A state space



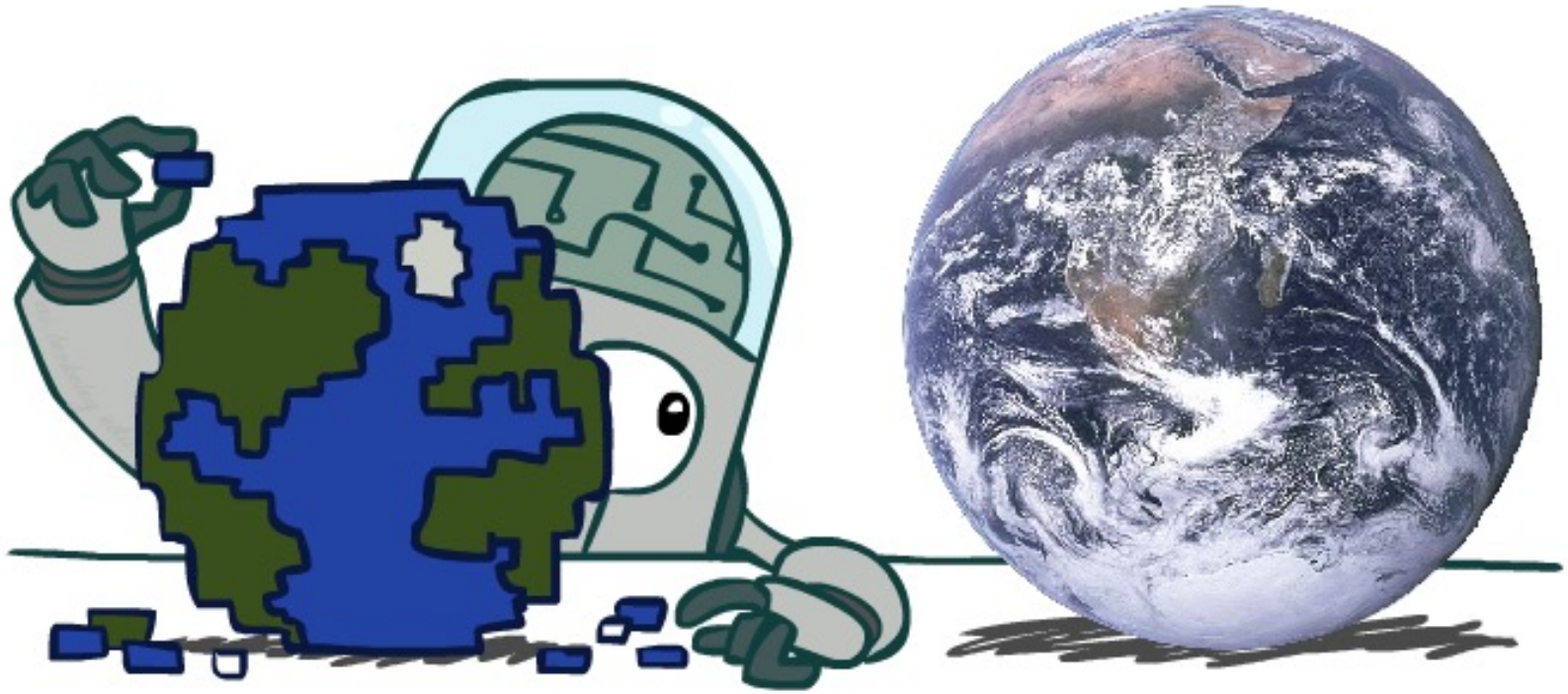
  - A successor function (with actions, costs)



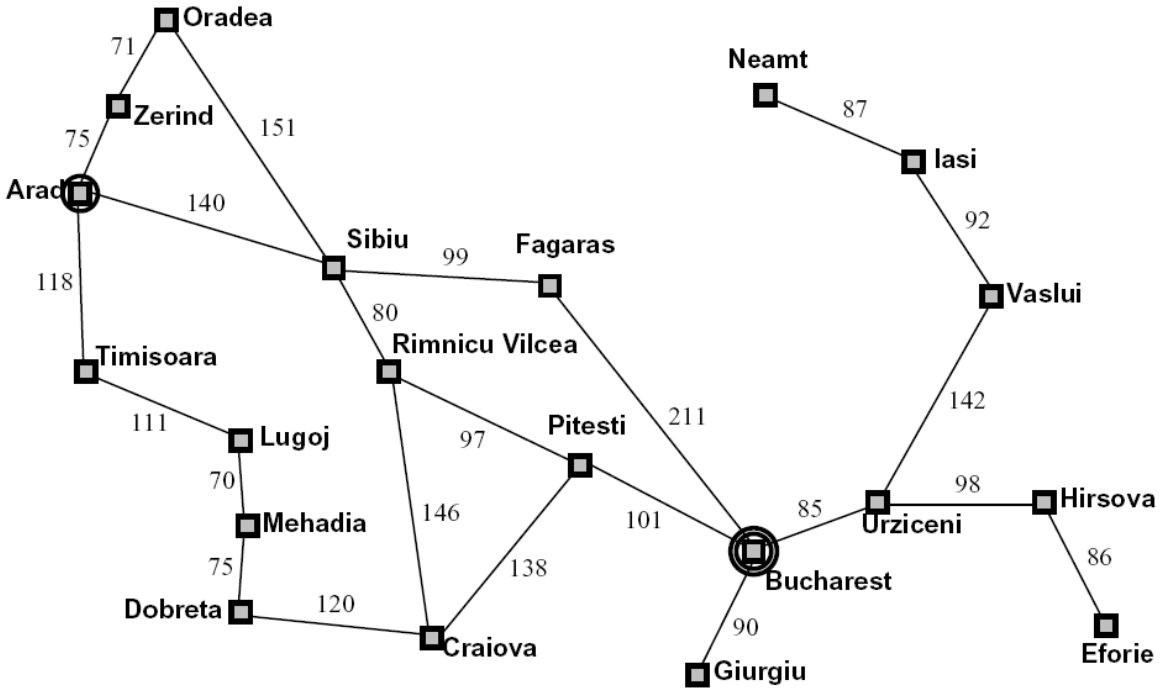
  - A start state and a goal test

- A solution is a sequence of actions (a plan) which transforms the start state to a goal state

Search Problems Are Models



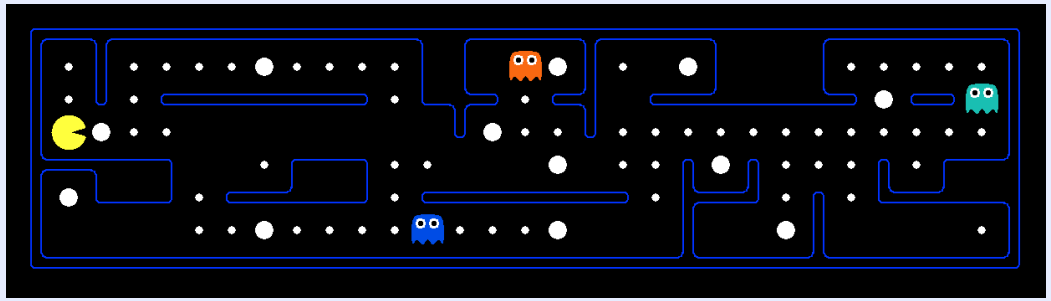
Example: Traveling in Romania



- State space:
  - Cities
- Successor function:
  - Roads: Go to adjacent city with cost = distance
- Start state:
  - Arad
- Goal test:
  - Is state == Bucharest?
- Solution?

What's in a State Space?

The world state includes every last detail of the environment



A search state keeps only the details needed for planning (abstraction)

- Problem: Pathing
  - States: (x,y) location
  - Actions: NSEW
  - Successor: update location only
  - Goal test: is (x,y)=END

- Problem: Eat-All-Dots
  - States: {(x,y), dot booleans}
  - Actions: NSEW
  - Successor: update location and possibly a dot boolean
  - Goal test: dots all false

State Space Sizes?

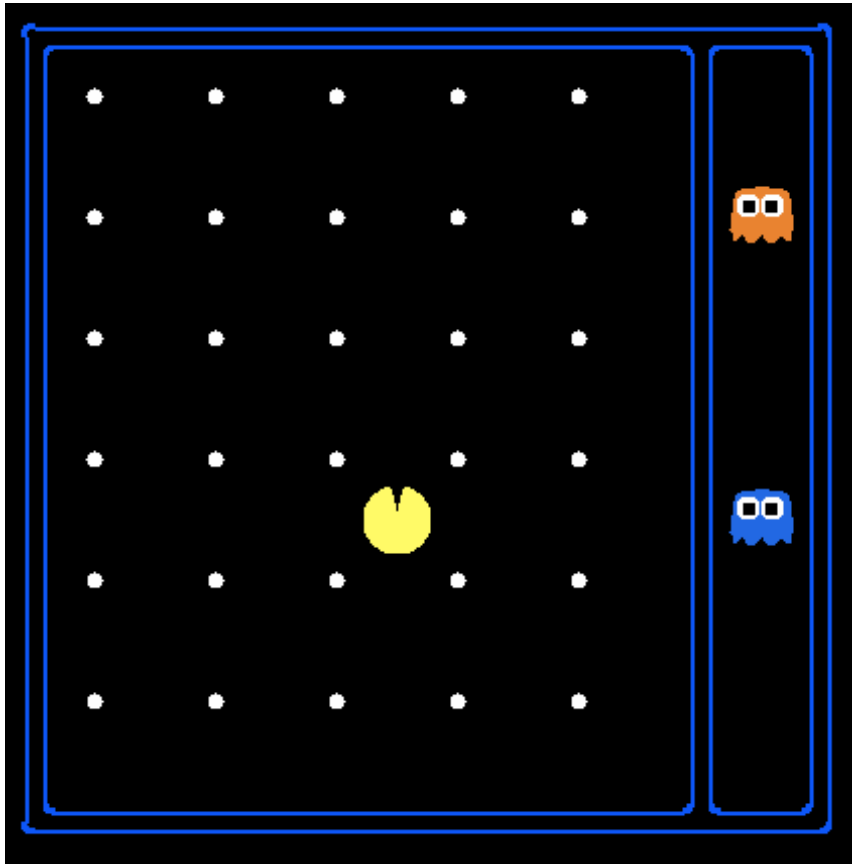
- World state:
  - Agent positions: 120
  - Food count: 30
  - Ghost positions: 12
  - Agent facing: NSEW

- How many
  - World states?

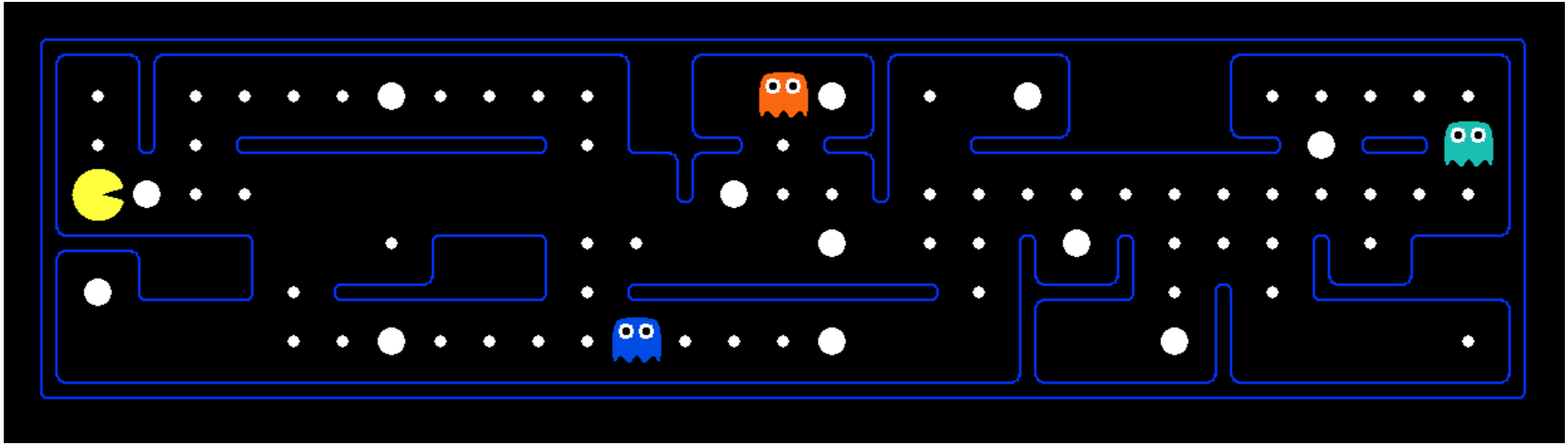
    $120x(2^{30})x(12^2)x4$
  - States for pathing?

    120
  - States for eat-all-dots?

    $120x(2^{30})$



Quiz: Safe Passage



- Problem: eat all dots while keeping the ghosts perma-scared
- What does the state space have to specify?
  - (agent position, dot booleans, power pellet booleans, remaining scared time)

Worked Example: Formulating a Search Problem

Two jugs (4 L and 3 L), a tap, and no markings. Measure exactly 2 L.

State space: (x, y) = litres in the 4 L and 3 L jugs

$x \in \{0..4\}$, $y \in \{0..3\}$ → 20 possible states

Start state: (0, 0)

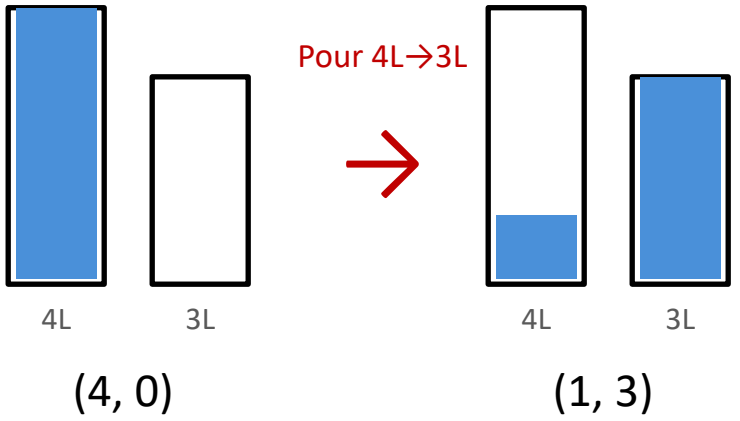
Goal test: x = 2

Actions (cost 1 each): Fill 4L, Fill 3L, Empty 4L, Empty 3L, Pour 4L→3L, Pour 3L→4L

Successor function: e.g. Pour 4L→3L moves min(x, 3 − y) litres

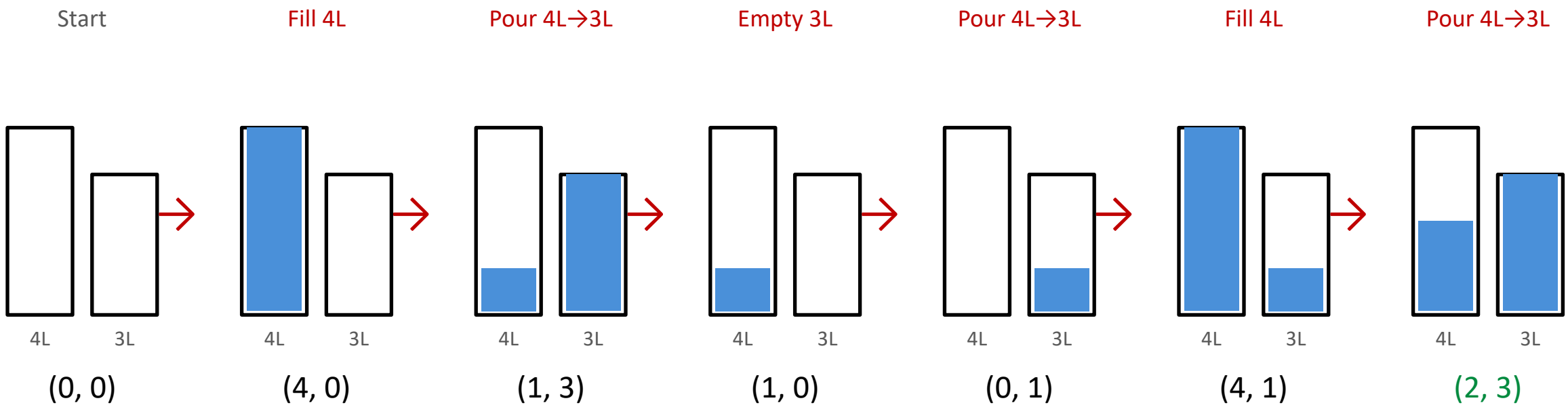
(4, 0) → (1, 3)

Solution: a sequence of actions from (0, 0) to a goal state



The search state keeps only what matters for planning: how much water is in each jug.

Water Jugs: Breadth-First Search Finds a Shortest Plan

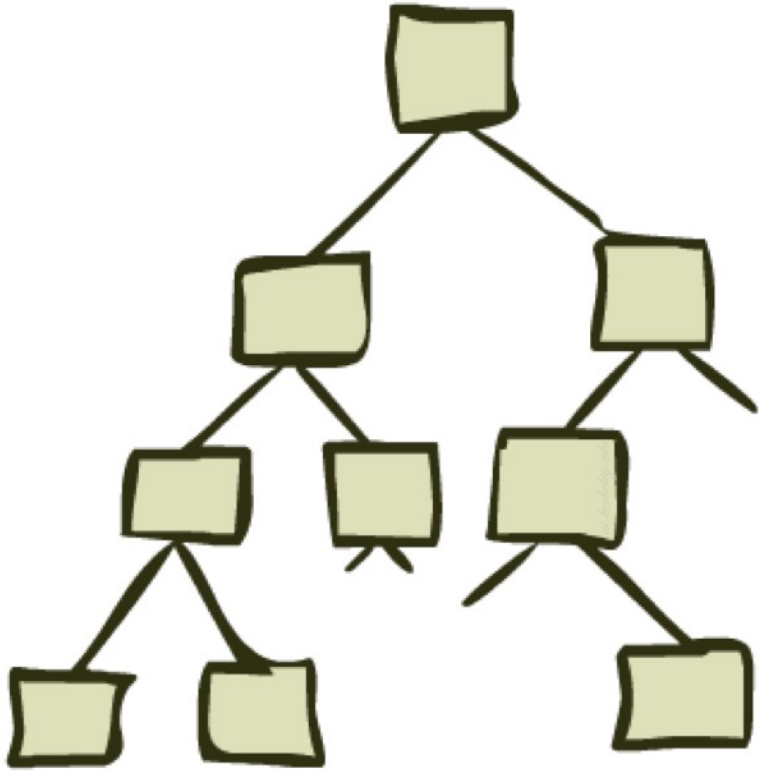


BFS explores states level by level: 1 state at depth 0, 2 at depth 1, 3 at depth 2, ... It reaches a goal at **depth 6**.

Because every action costs 1, the first goal BFS reaches uses the **fewest actions**. (Filling the 3 L jug first also takes 6 actions.)

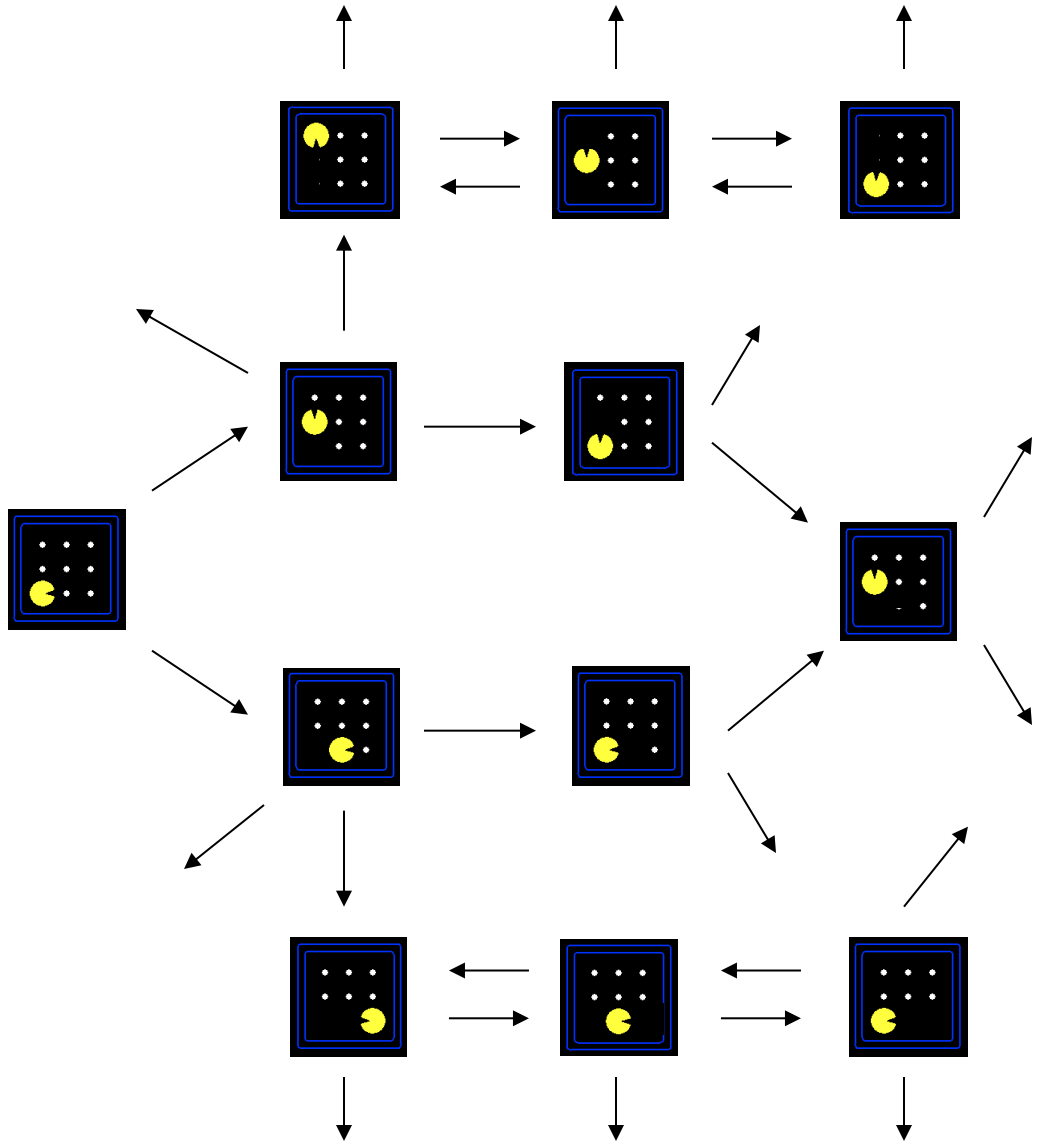
Think: would DFS also find a 6-action plan? Would it always terminate without a closed set?

State Space Graphs and Search Trees



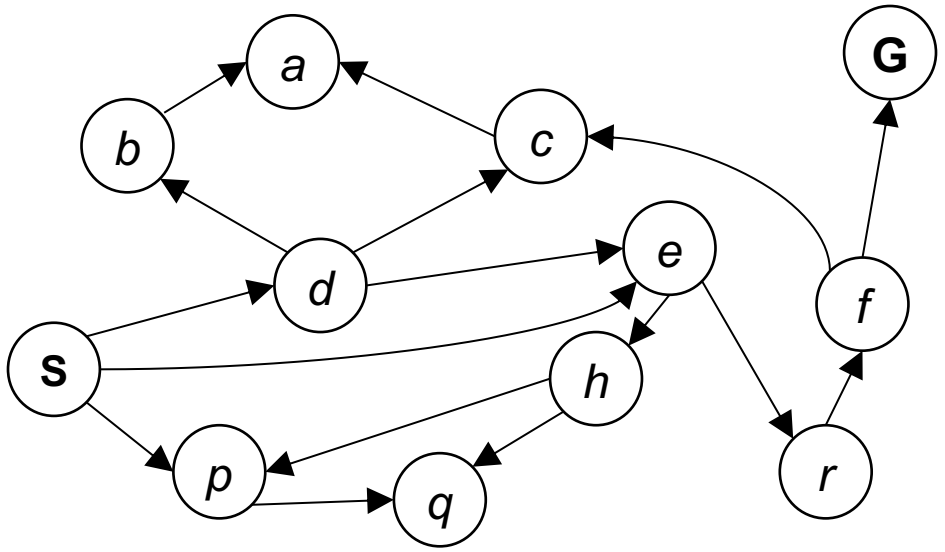
State Space Graphs

- State space graph: A mathematical representation of a search problem
  - Nodes are (abstracted) world configurations
  - Arcs represent successors (action results)
  - The goal test is a set of goal nodes (maybe only one)
- In a state space graph, each state occurs only once!
- We can rarely build this full graph in memory (it's too big), but it's a useful idea



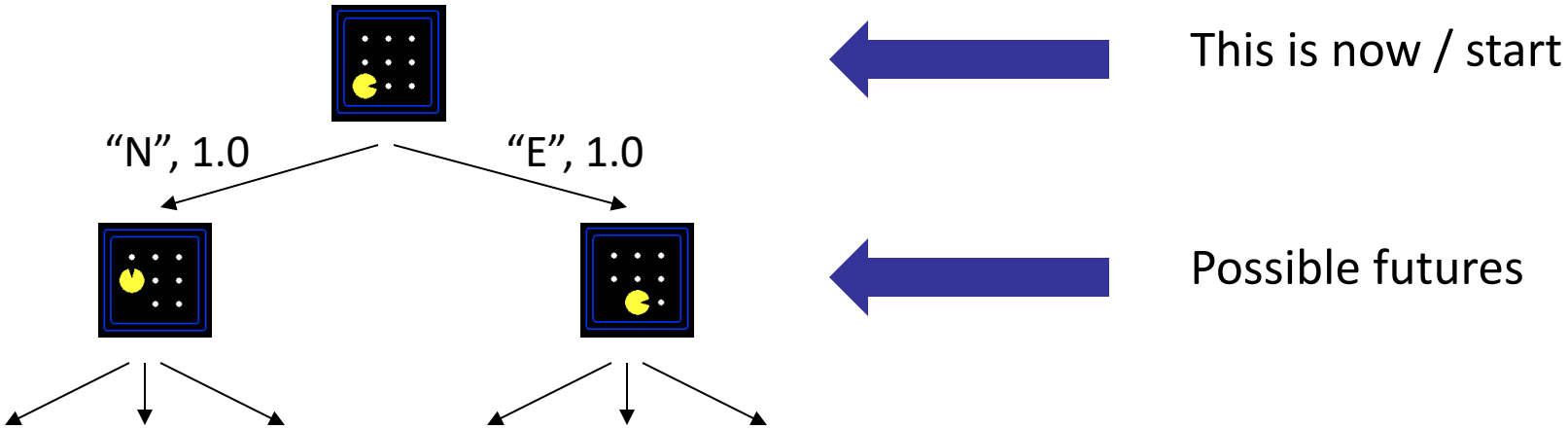
State Space Graphs

- State space graph: A mathematical representation of a search problem
  - Nodes are (abstracted) world configurations
  - Arcs represent successors (action results)
  - The goal test is a set of goal nodes (maybe only one)
- In a search graph, each state occurs only once!
- We can rarely build this full graph in memory (it's too big), but it's a useful idea



Tiny search graph for a tiny search problem

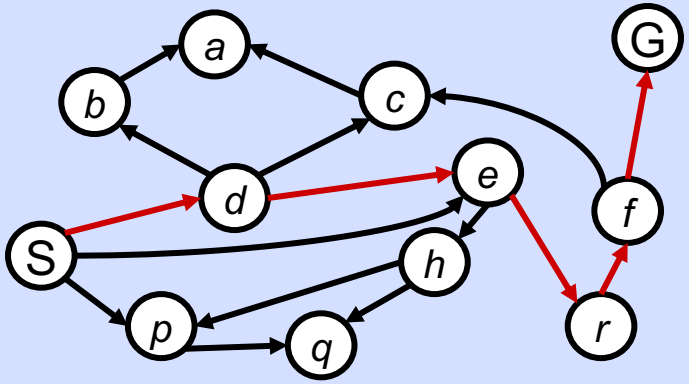
Search Trees



- A search tree:
  - A "what if" tree of plans and their outcomes
  - The start state is the root node
  - Children correspond to successors
  - Nodes show states, but correspond to PLANS that achieve those states
  - For most problems, we can never actually build the whole tree

State Space Graphs vs. Search Trees

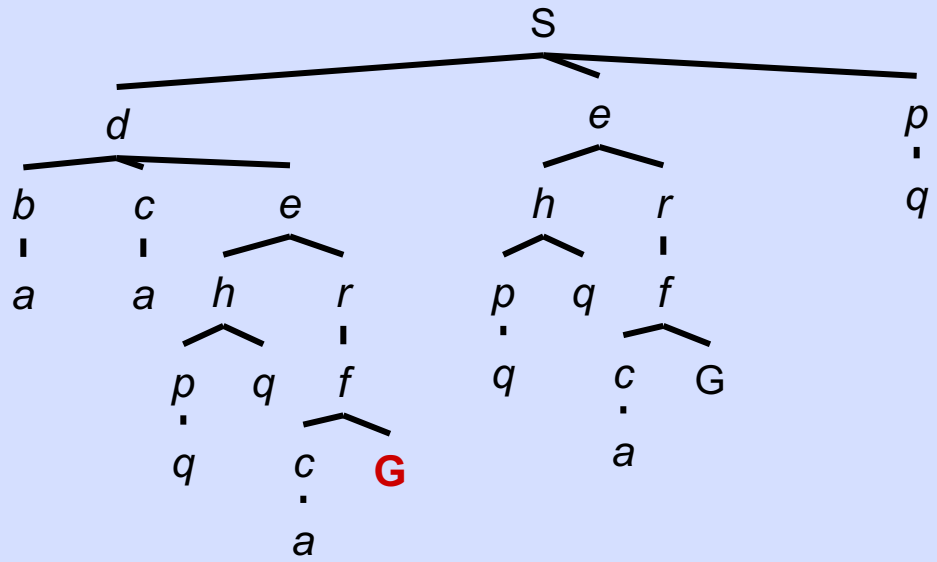
State Space Graph



Each NODE in in the search tree is an entire PATH in the state space graph.

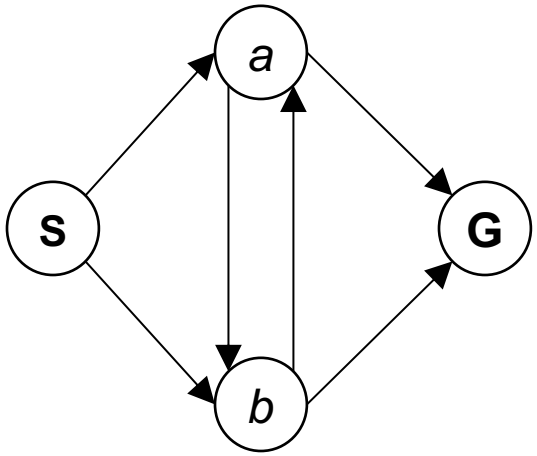
We construct both on demand – and we construct as little as possible.

Search Tree



Quiz: State Space Graphs vs. Search Trees

Consider this 4-state graph:

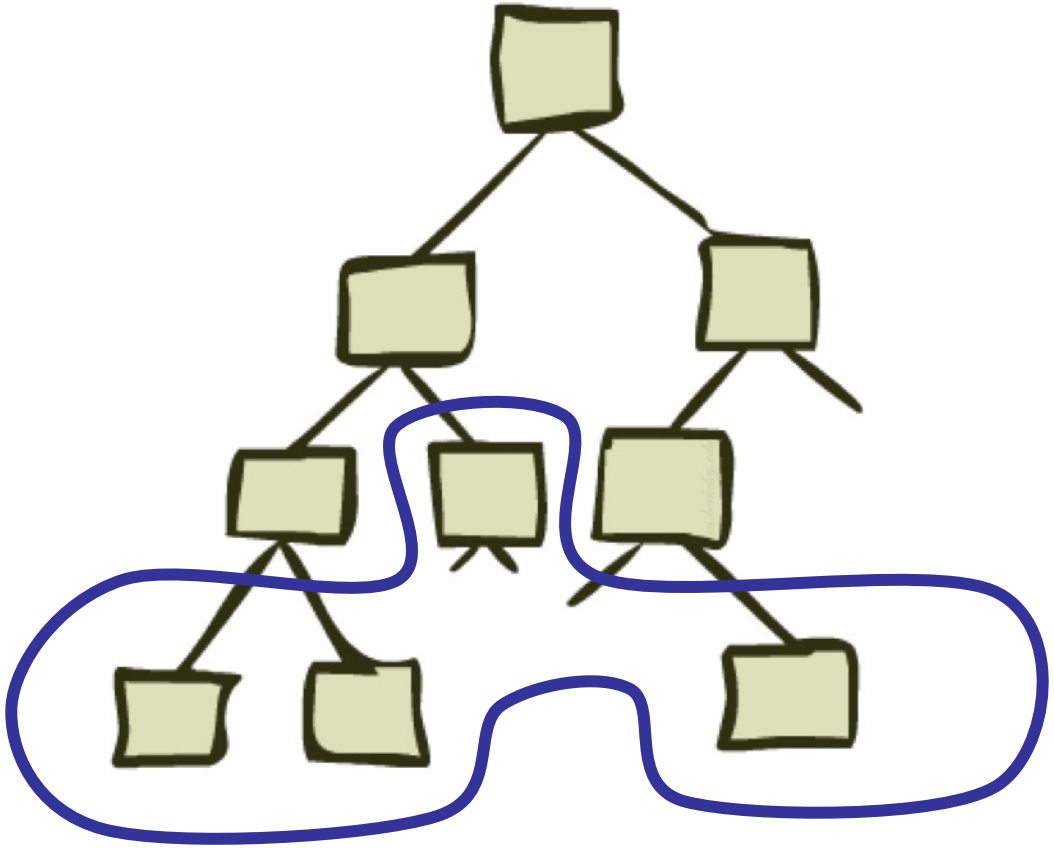


How big is its search tree (from S)?

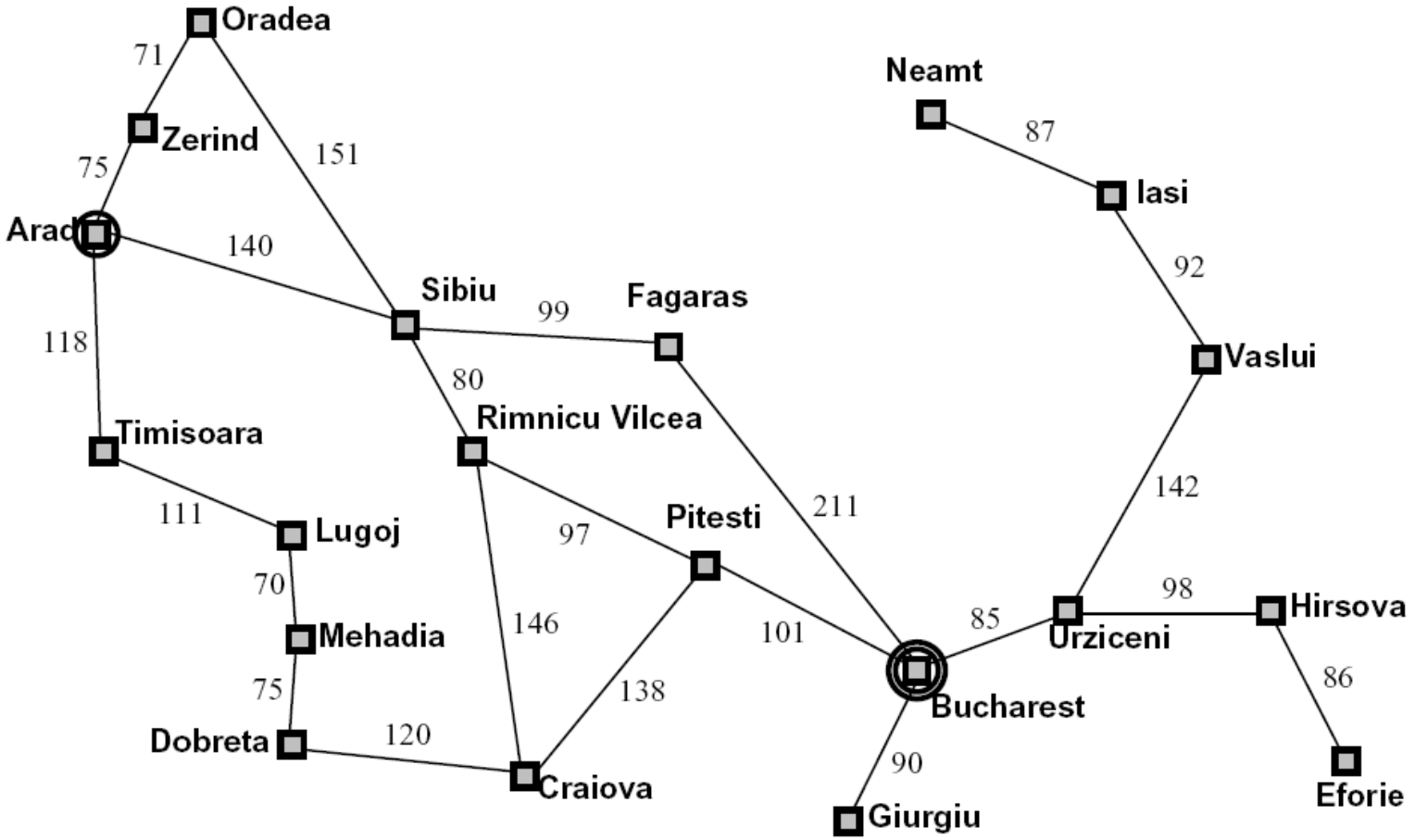


Important: Lots of repeated structure in the search tree!

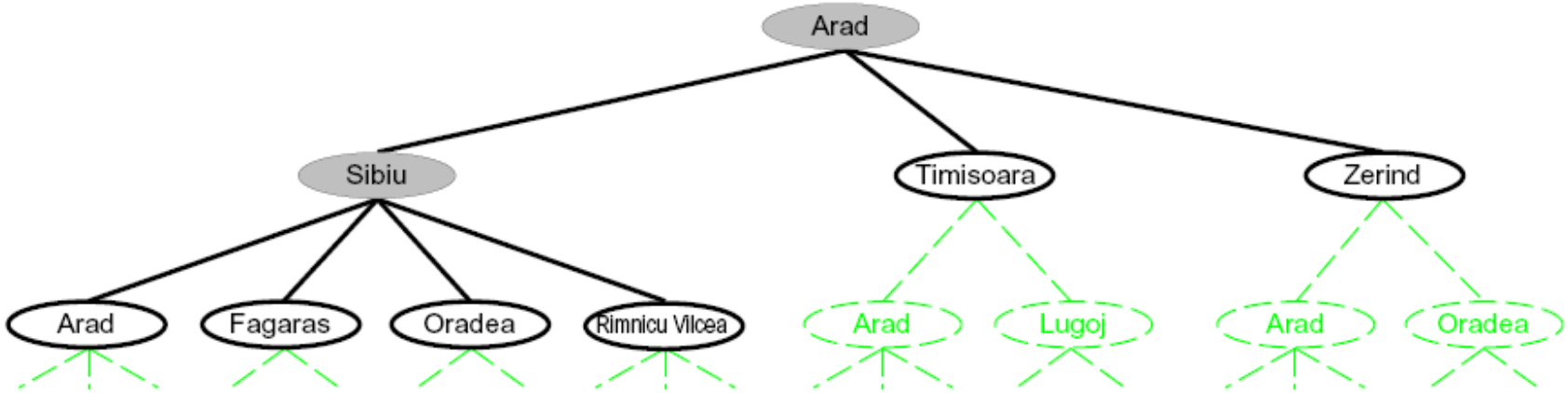
Tree Search



Search Example: Romania



Searching with a Search Tree



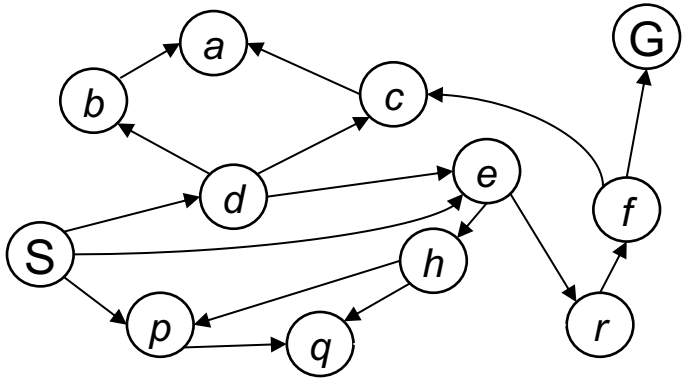
- Search:
  - Expand out potential plans (tree nodes)
  - Maintain a fringe of partial plans under consideration
  - Try to expand as few tree nodes as possible

General Tree Search

```
function TREE-SEARCH( problem, strategy) returns a solution, or failure
    initialize the search tree using the initial state of problem
    loop do
        if there are no candidates for expansion then return failure
        choose a leaf node for expansion according to strategy
        if the node contains a goal state then return the corresponding solution
        else expand the node and add the resulting nodes to the search tree
    end
```

- Important ideas:
  - Fringe
  - Expansion
  - Exploration strategy
- Main question: which fringe nodes to explore?

Example: Tree Search



Tree Search vs. Graph Search

Tree search can expand the same state many times: the search tree has lots of repeated structure, and can be infinite when the graph has cycles.

Graph search = tree search + a **closed set**: never expand a state twice.

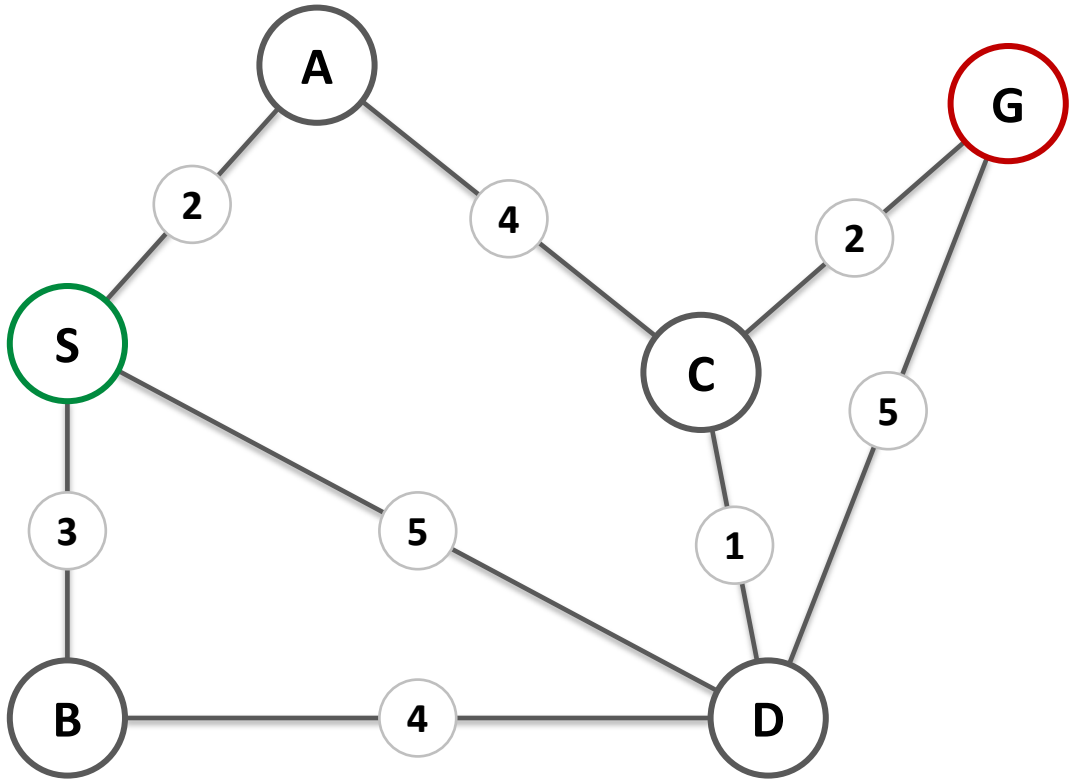
When a popped node's state is already closed, skip it.

All of today's worked examples (and HW0) use **graph search**.

```
fringe ← [start];  closed ← {}
while fringe is not empty:
    node ← fringe.pop()
    if node.state in closed:
        continue           # skip
    if is_goal(node.state):
        return node.path
    closed.add(node.state)
    for child in successors(node):
        if child.state not in closed:
            fringe.push(child)
return failure
```

The fringe type is the only difference between DFS, BFS, and UCS.

Worked Example Graph



Find a path from **S** (start) to **G** (goal). Edge labels are step costs.

We will run **DFS, BFS, and UCS** on this same graph, one step at a time.

Rules for every trace:

- Graph search: never expand a state twice
- Goal test when a node is **popped**, not when it is added
- Ties are broken **alphabetically**

Try predicting each answer before we step through it!

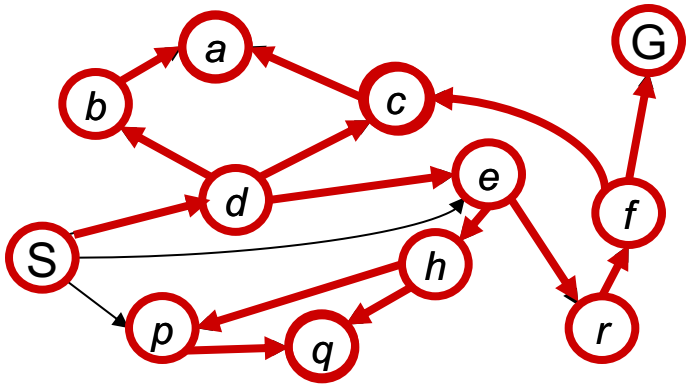
Depth-First Search

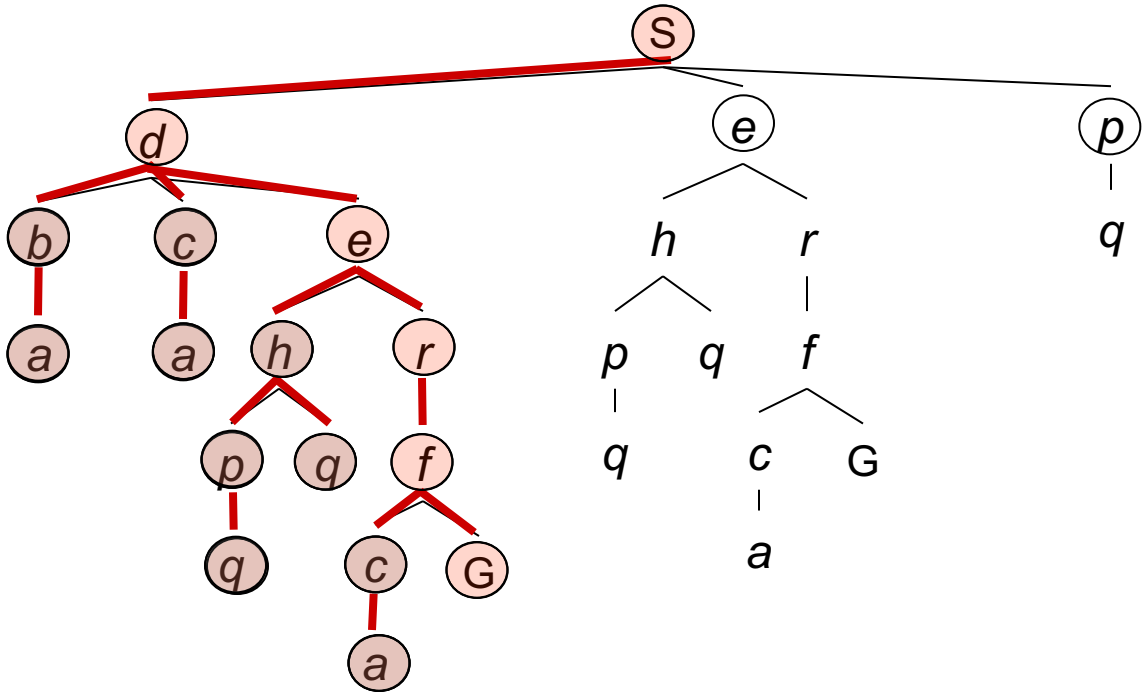


Depth-First Search

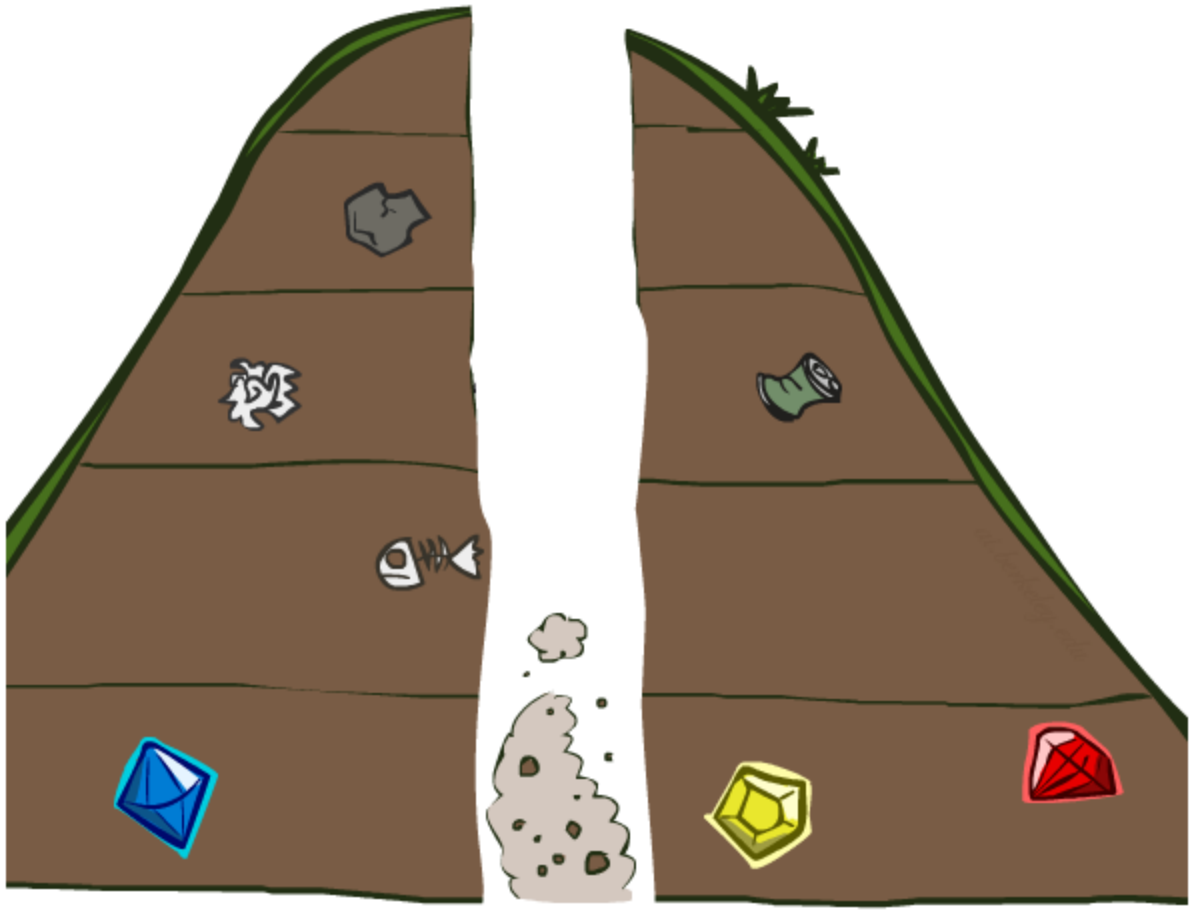
Strategy: expand a deepest node first

Implementation: Fringe is a LIFO stack





Search Algorithm Properties

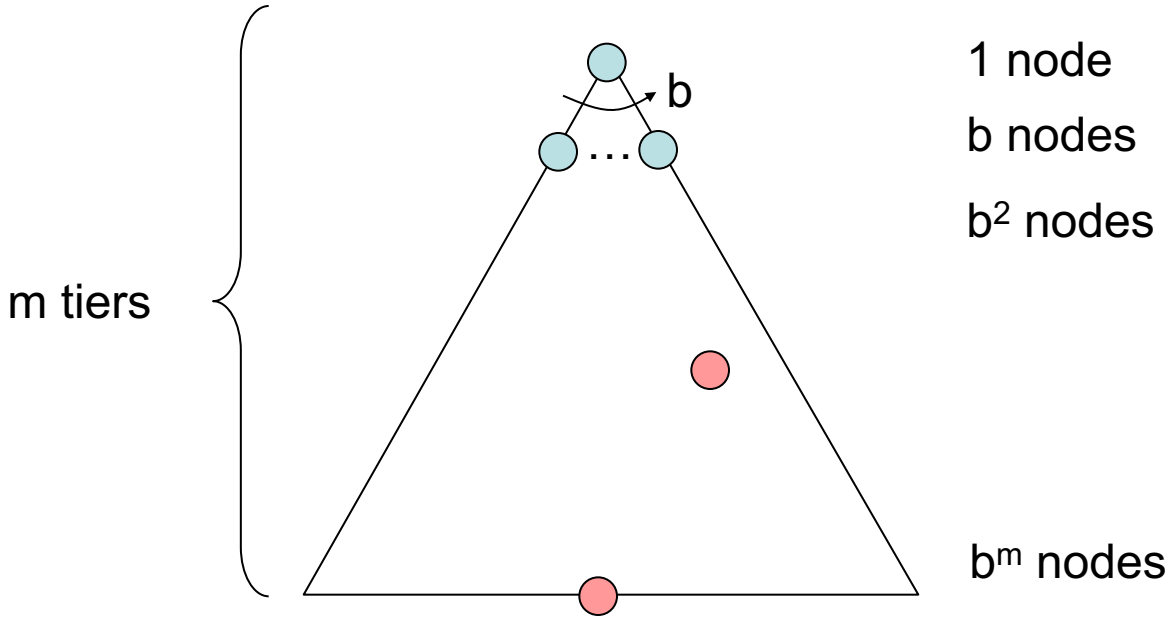


Search Algorithm Properties

- Complete: Guaranteed to find a solution if one exists?
- Optimal: Guaranteed to find the least cost path?
- Time complexity?
- Space complexity?

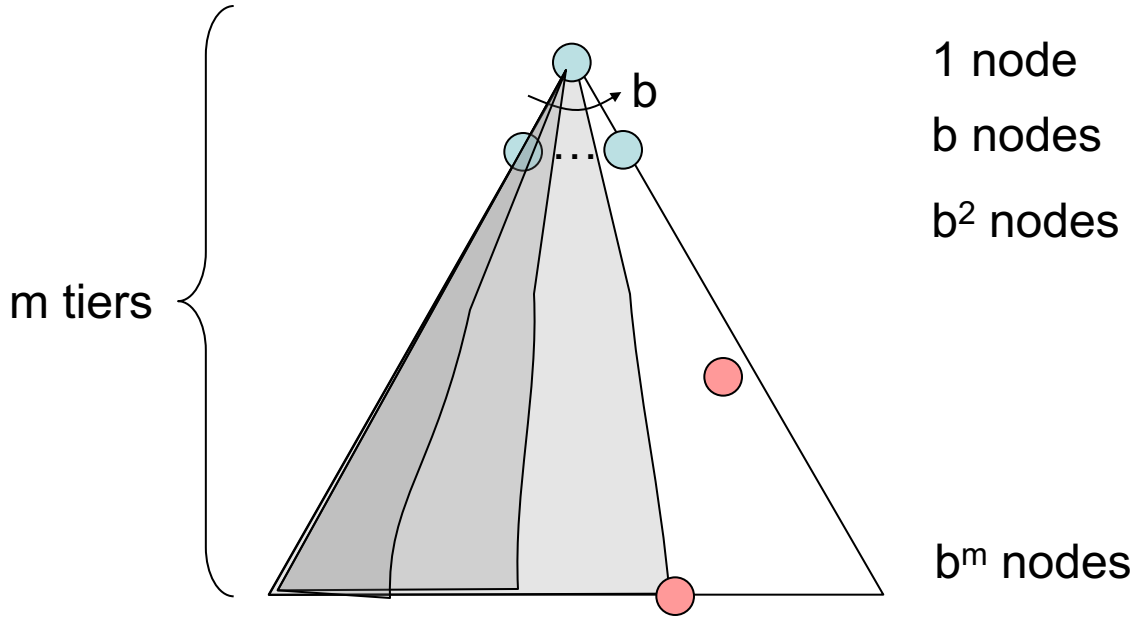
- Cartoon of search tree:
  - b is the branching factor
  - m is the maximum depth
  - solutions at various depths

- Number of nodes in entire tree?
  - $1 + b + b^2 + \ldots. b^m = O(b^m)$

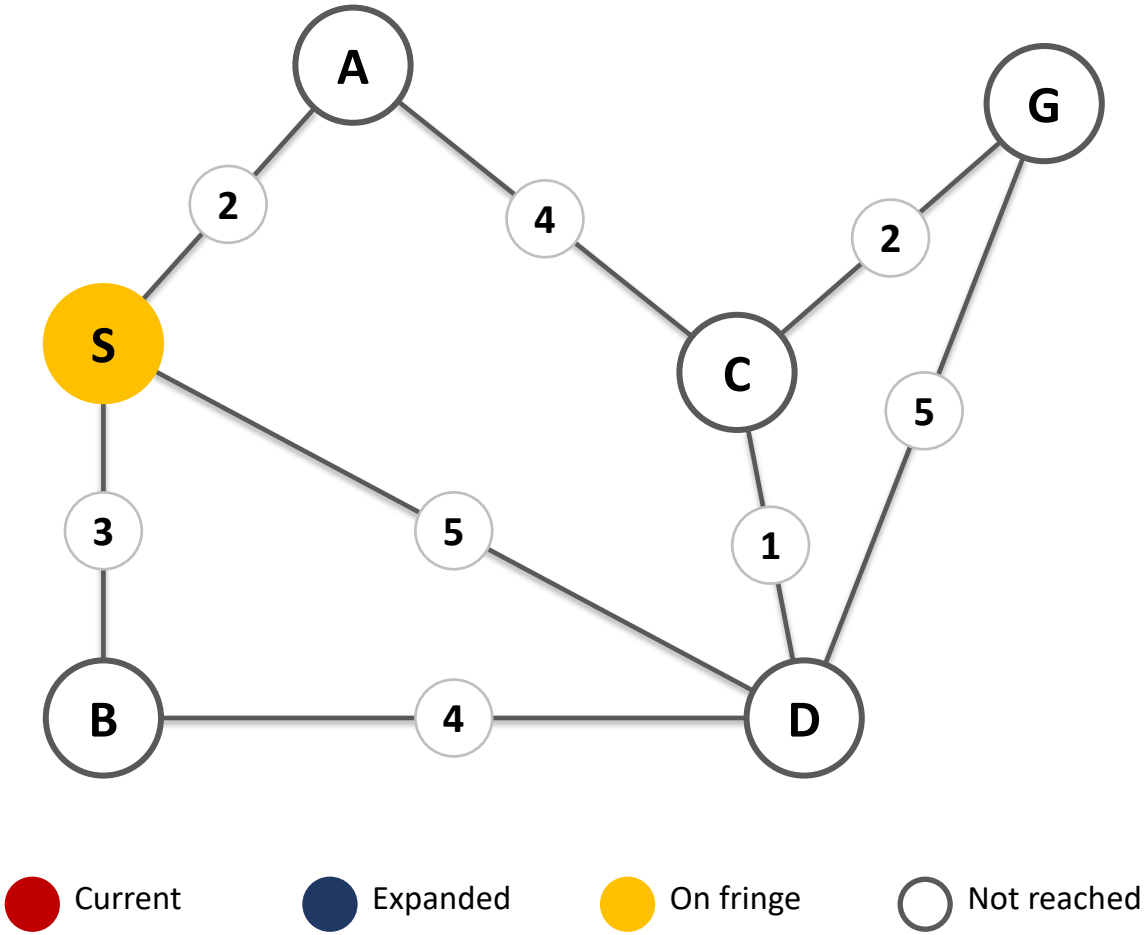


Depth-First Search (DFS) Properties

- What nodes DFS expand?
  - Some left prefix of the tree.
  - Could process the whole tree!
  - If m is finite, takes time $O(b^m)$
- How much space does the fringe take?
  - Only has siblings on path to root, so O(bm)
- Is it complete?
  - m could be infinite, so only if we prevent cycles (more later)
- Is it optimal?
  - No, it finds the "leftmost" solution, regardless of depth or cost



DFS Worked Example (step 0 of 6)



Current Expanded On fringe Not reached

Initial state

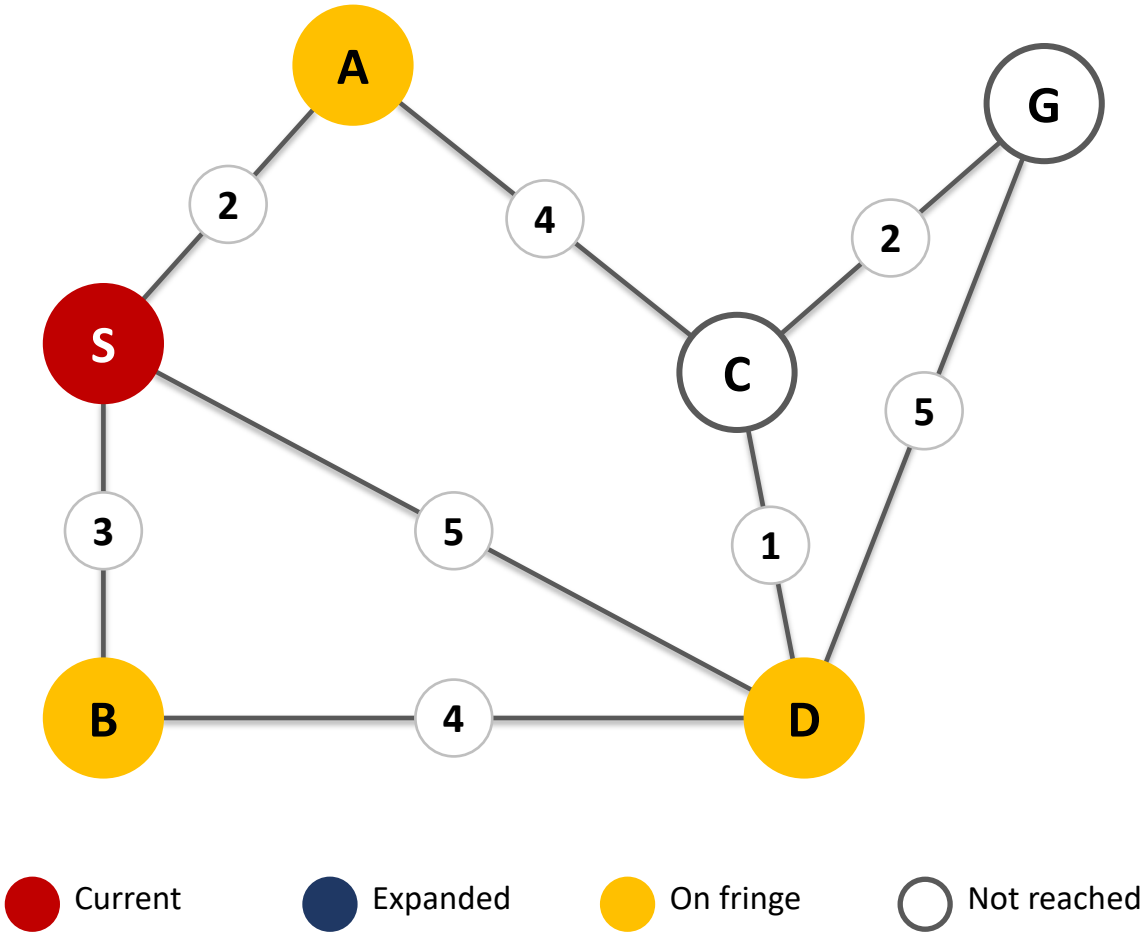
Fringe: Stack (LIFO): top first

top ▸ **S**

Expanded: (none)

Start: the fringe holds only the start state S. Nothing has been expanded yet.

DFS Worked Example (step 1 of 6)



Popped: S

Fringe: Stack (LIFO): top first

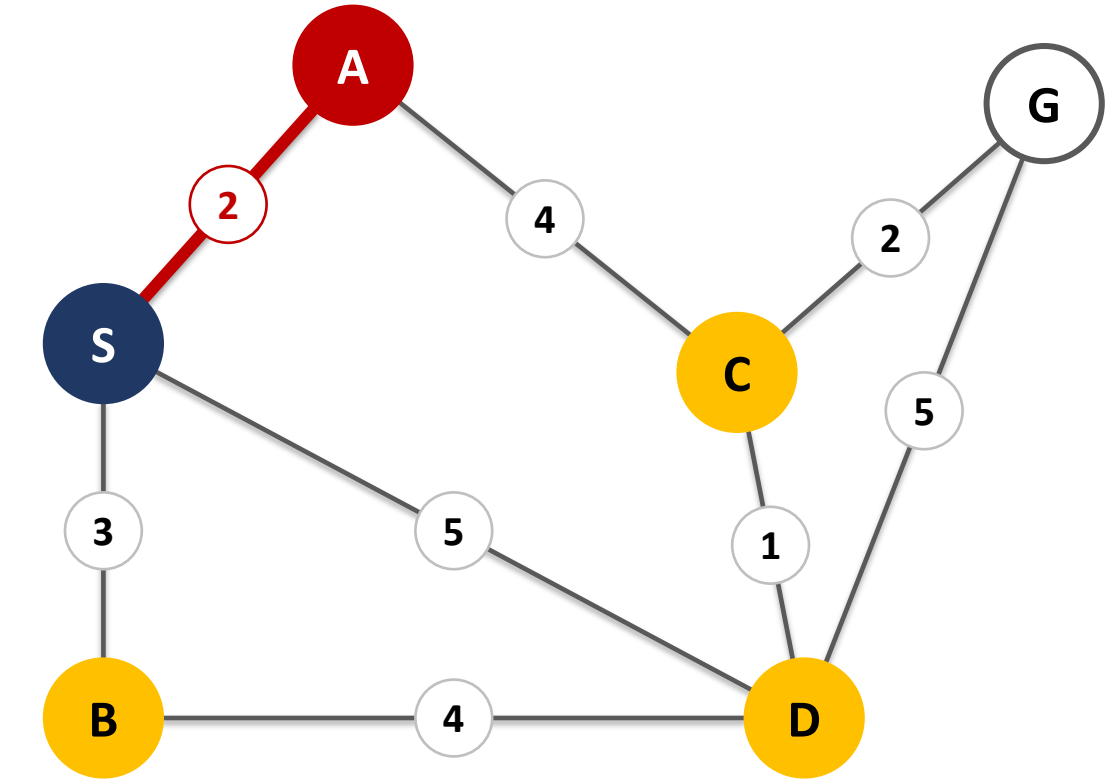
top ▸

- **S → A (new)**
- S → B (new)
- S → D (new)

Expanded: S

Pop S. S is not the goal, so **expand S**. Add S → A, S → B, S → D. Pushed in reverse order so A ends on top.

DFS Worked Example (step 2 of 6)



Current Expanded On fringe Not reached

Popped: S → A

Fringe: Stack (LIFO): top first

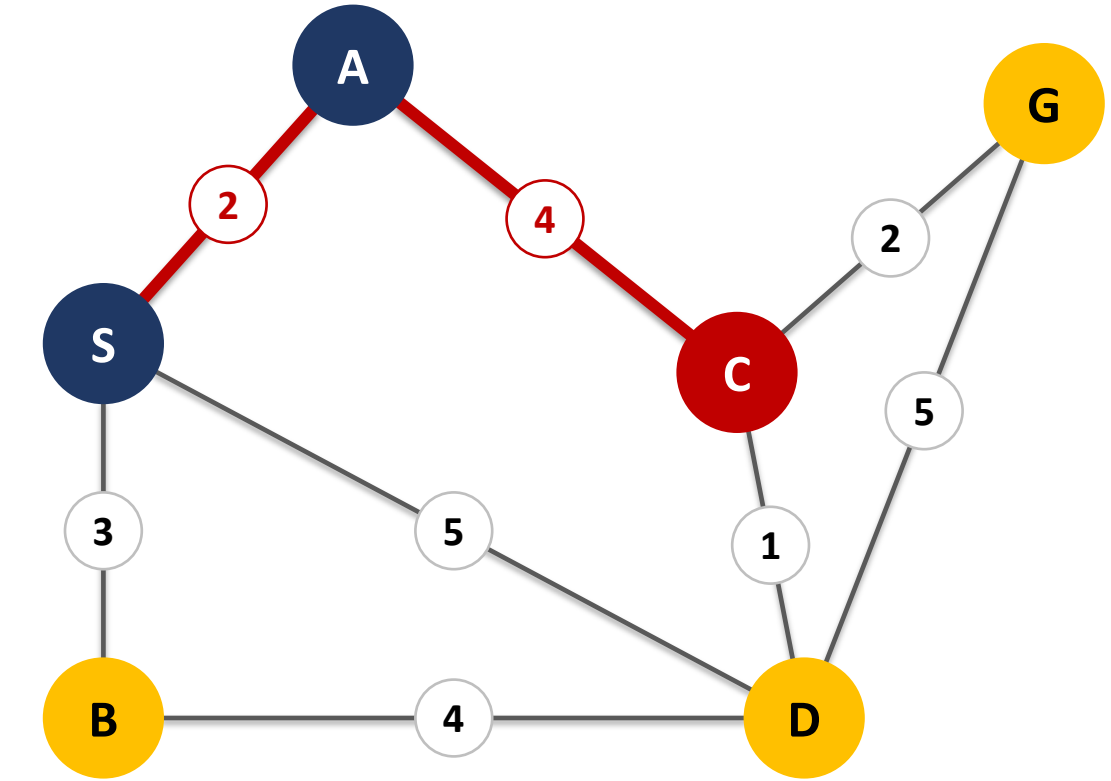
top ▸

- **S → A → C (new)**
- S → B
- S → D

Expanded: S, A

Pop S → A. A is not the goal, so **expand A**. Add S → A → C. (S already expanded: not added.)

DFS Worked Example (step 3 of 6)



Current Expanded On fringe Not reached

Popped: S → A → C

Fringe: Stack (LIFO): top first

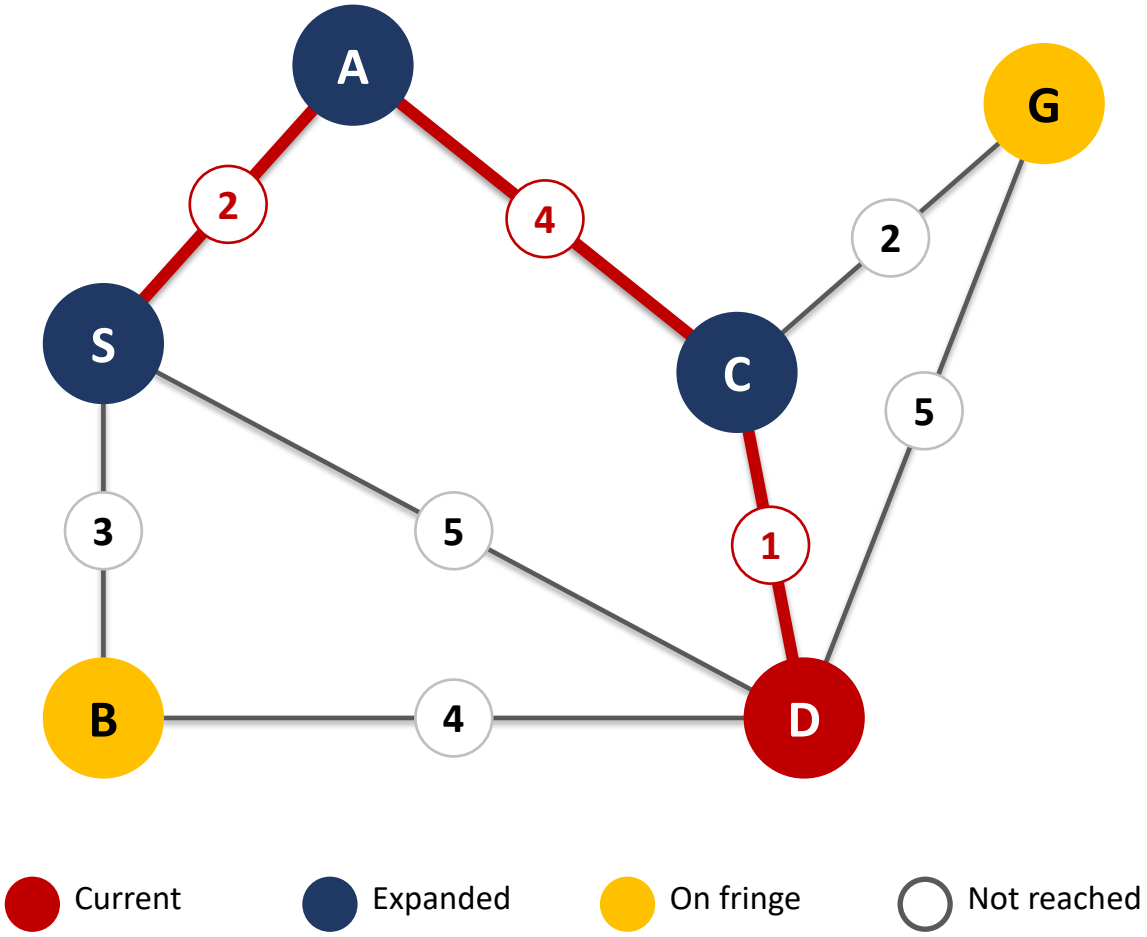
top ▸

- **S → A → C → D (new)**
- S → A → C → G (new)
- S → B
- S → D

Expanded: S, A, C

Pop S → A → C. C is not the goal, so **expand C**. Add S → A → C → D, S → A → C → G. (A already expanded: not added.) Pushed in reverse order so D ends on top.

DFS Worked Example (step 4 of 6)



Popped: S → A → C → D

Fringe: Stack (LIFO): top first

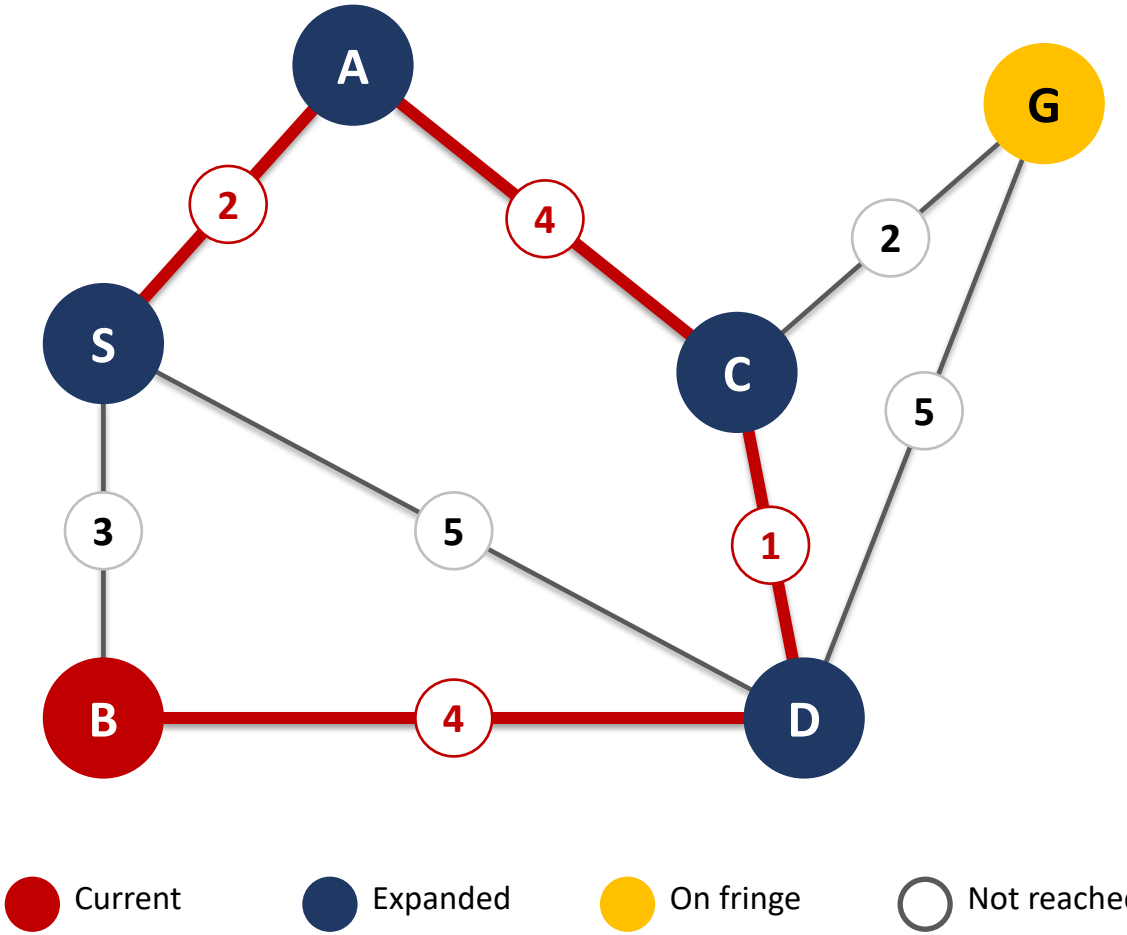
top ▸

S → A → C → D → B (new)
S → A → C → D → G (new)
S → A → C → G
S → B
S → D

Expanded: S, A, C, D

Pop S → A → C → D. D is not the goal, so **expand D**. Add S → A → C → D → B, S → A → C → D → G. (C, S already expanded: not added.) Pushed in reverse order so B ends on top.

DFS Worked Example (step 5 of 6)



Popped: S → A → C → D → B

Fringe: Stack (LIFO): top first

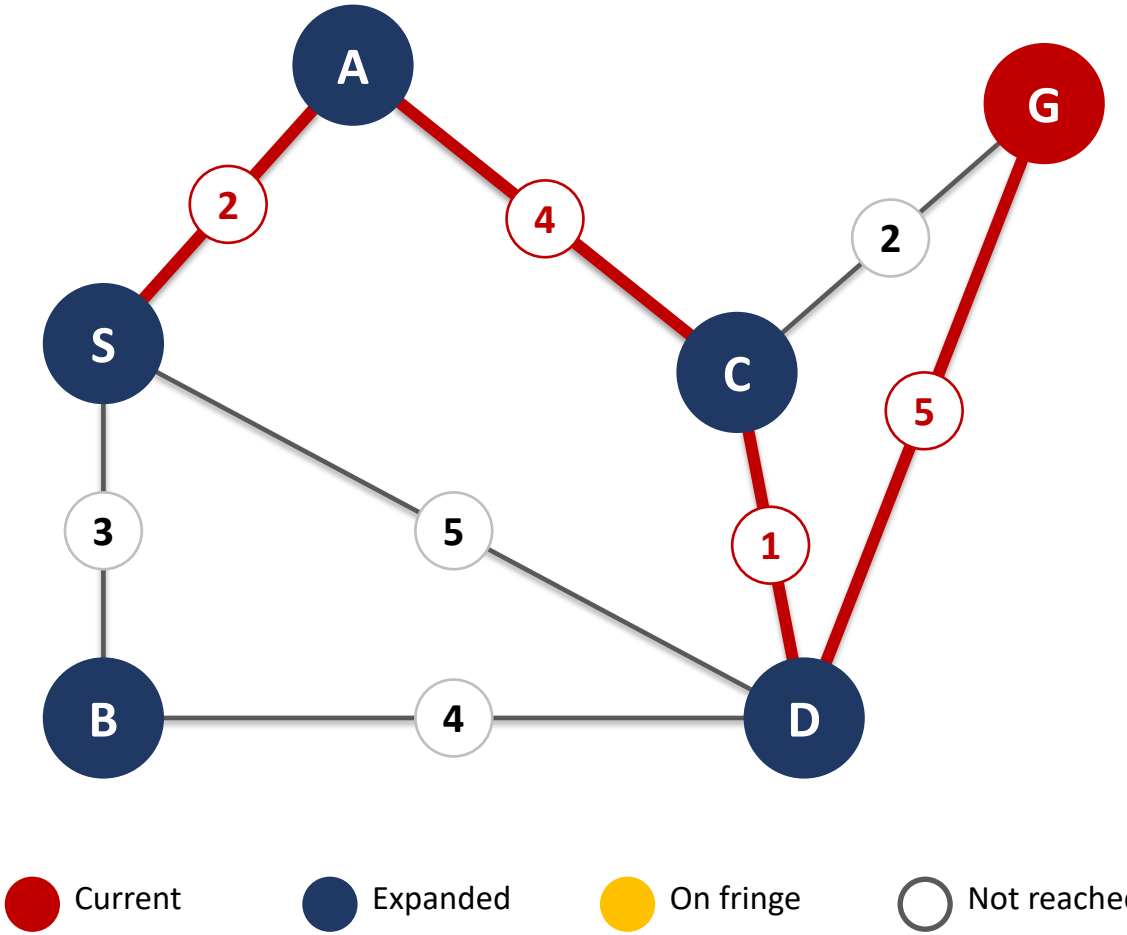
top ▸

- **S → A → C → D → G**
- S → A → C → G
- S → B
- S → D

Expanded: S, A, C, D, B

Pop S → A → C → D → B. B is not the goal, so **expand B**. Every neighbor is already expanded, so nothing is added. (D, S already expanded: not added.)

DFS Worked Example (step 6 of 6)



Popped (goal!): **S → A → C → D → G**

Fringe: Stack (LIFO): top first

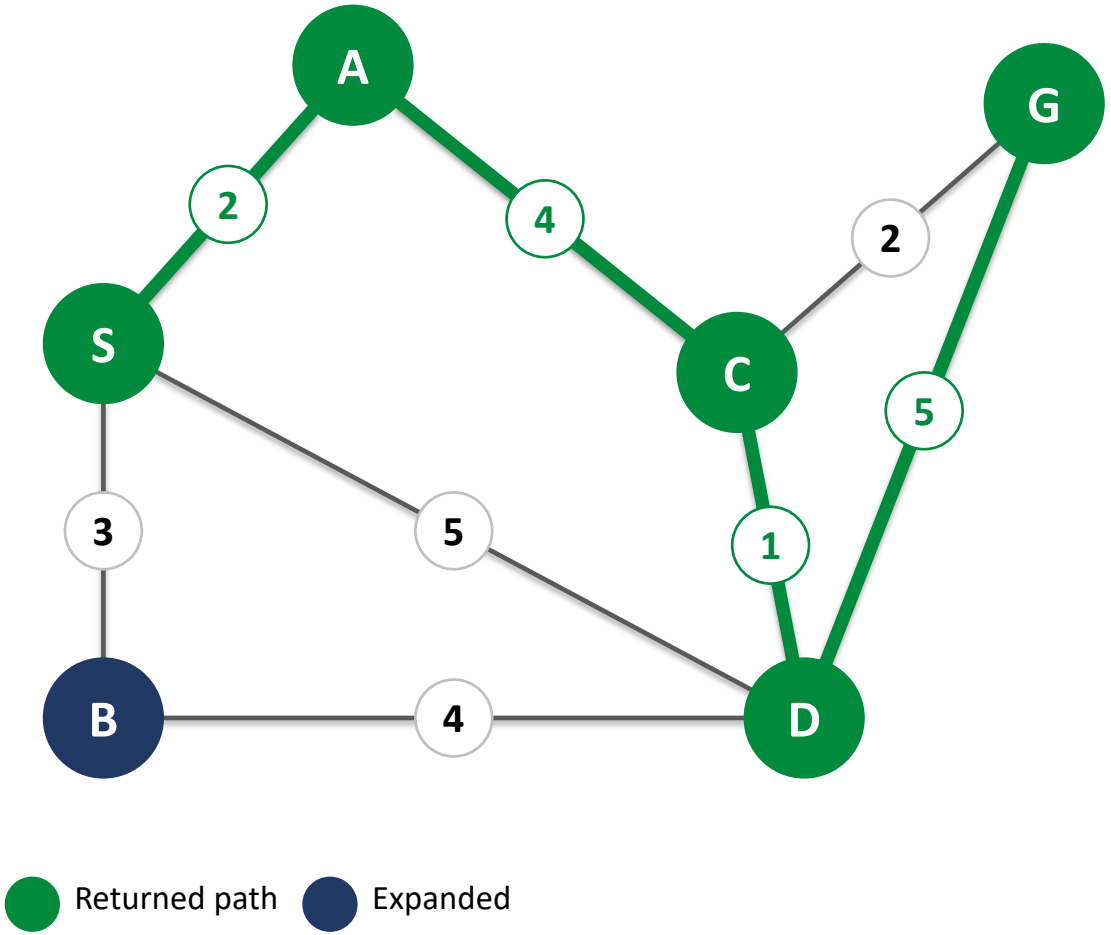
top ▸

- **S → A → C → G**
- S → B
- S → D

Expanded: S, A, C, D, B, G

Pop S → A → C → D → G. **G is the goal**, so return this path. Total cost = 12.

DFS Result



Expansion order: S, A, C, D, B, G

Path returned: S → A → C → D → G

Cost: 2 + 4 + 1 + 5 = 12

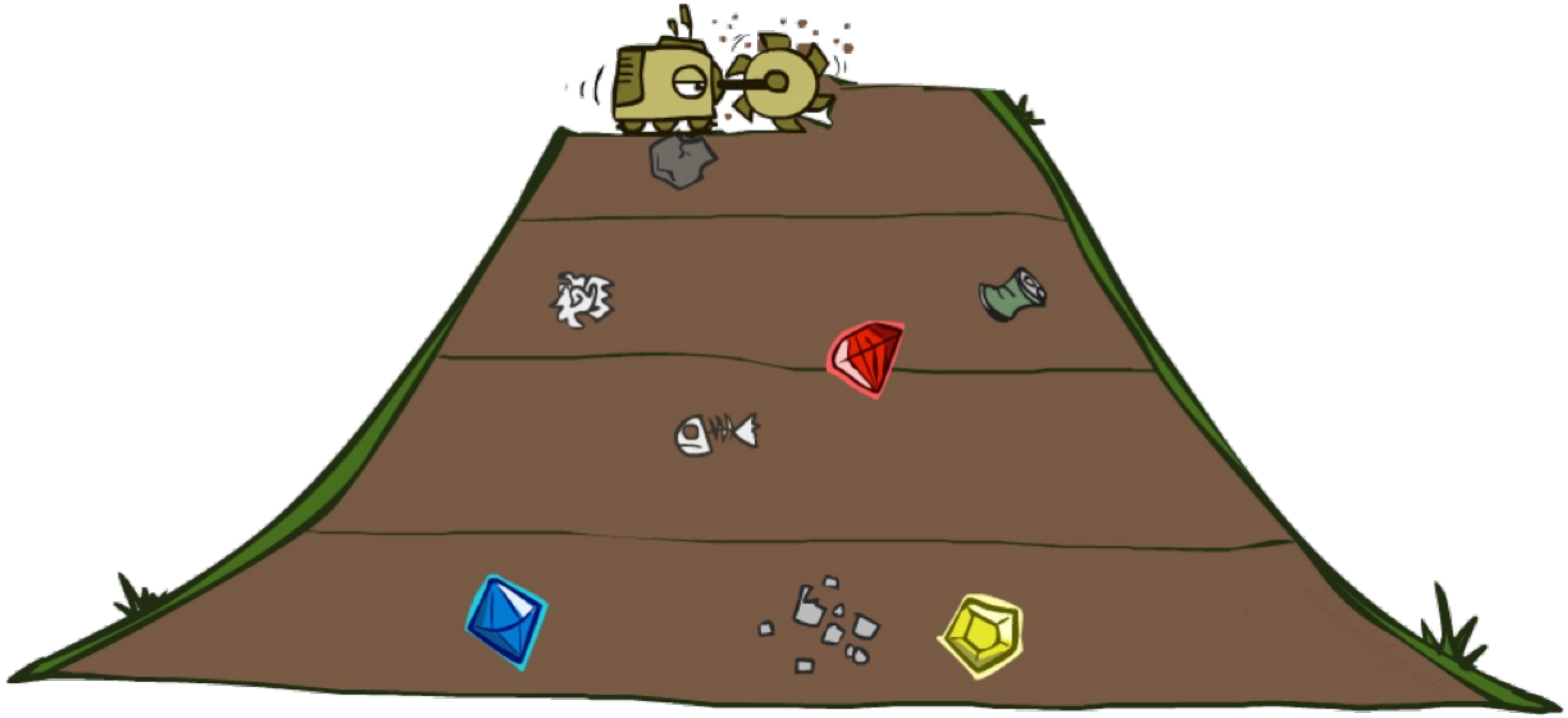
Actions: 4

DFS dives deep along S → A → C → D before backing up.

It found a path quickly, but **not the cheapest one**. UCS will find a path of cost 8.

DFS returns the “leftmost” solution, regardless of depth or cost.

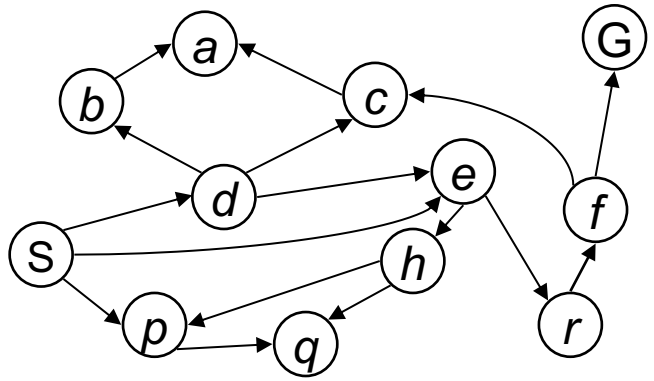
Breadth-First Search

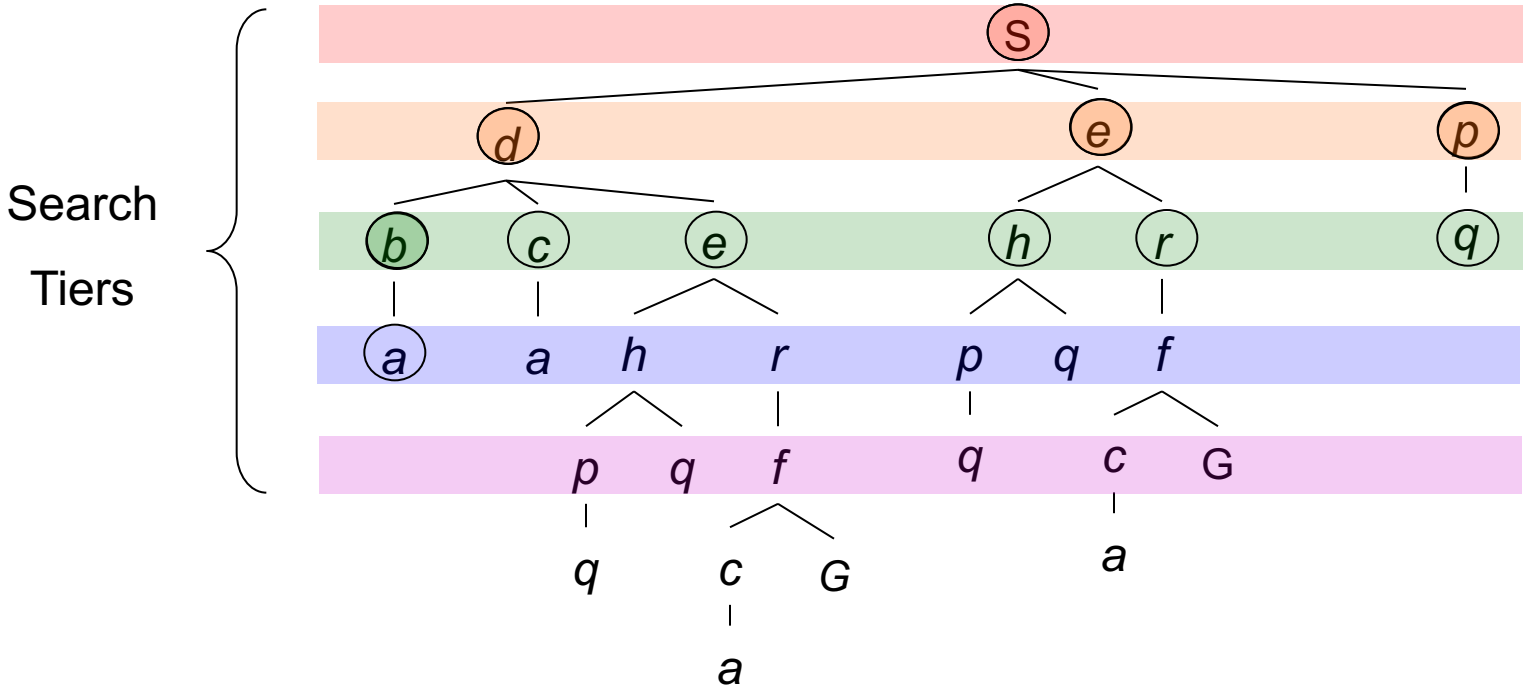


Breadth-First Search

Strategy: expand a shallowest node first

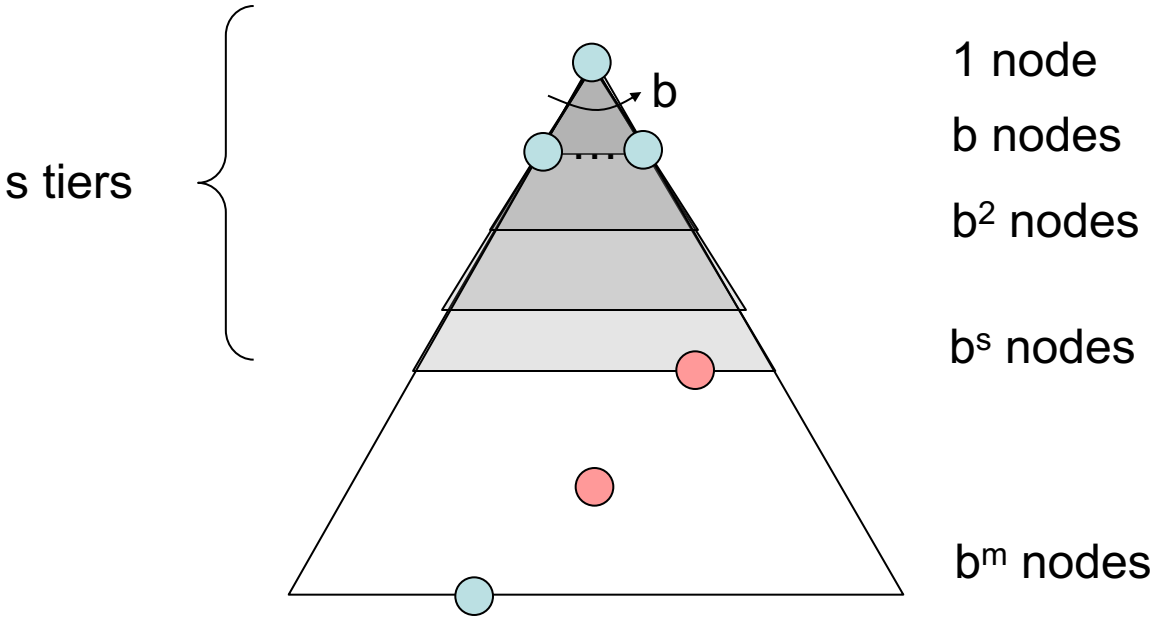
Implementation: Fringe is a FIFO queue



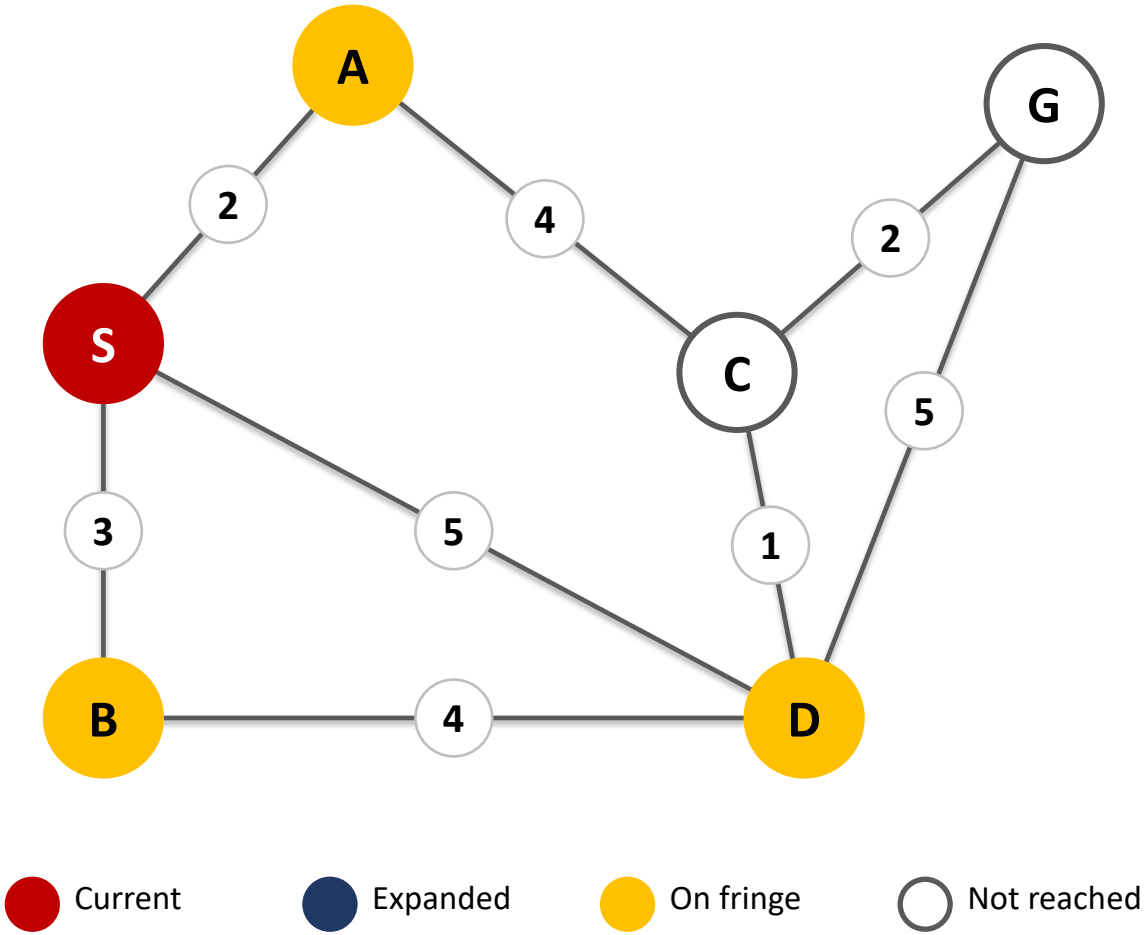


Breadth-First Search (BFS) Properties

- What nodes does BFS expand?
  - Processes all nodes above shallowest solution
  - Let depth of shallowest solution be s
  - Search takes time $O(b^s)$
- How much space does the fringe take?
  - Has roughly the last tier, so $O(b^s)$
- Is it complete?
  - s must be finite if a solution exists, so yes!
- Is it optimal?
  - Only if costs are all 1 (more on costs later)



BFS Worked Example (step 1 of 8)



Popped: S

Fringe: Queue (FIFO): front first

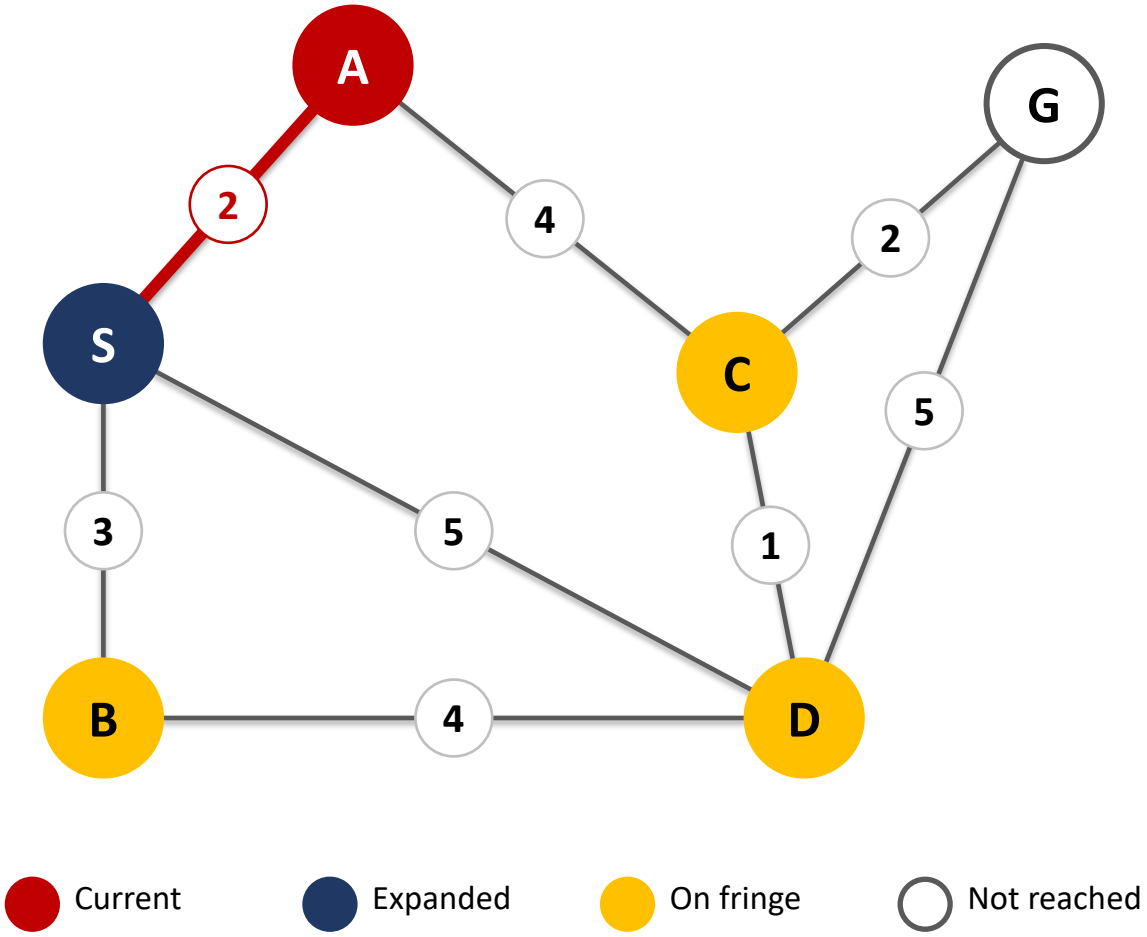
front ▸

- **S → A (new)**
- S → B (new)
- S → D (new)

Expanded: S

Pop S. S is not the goal, so **expand S**. Add S → A, S → B, S → D.

BFS Worked Example (step 2 of 8)



Popped: S → A

Fringe: Queue (FIFO): front first

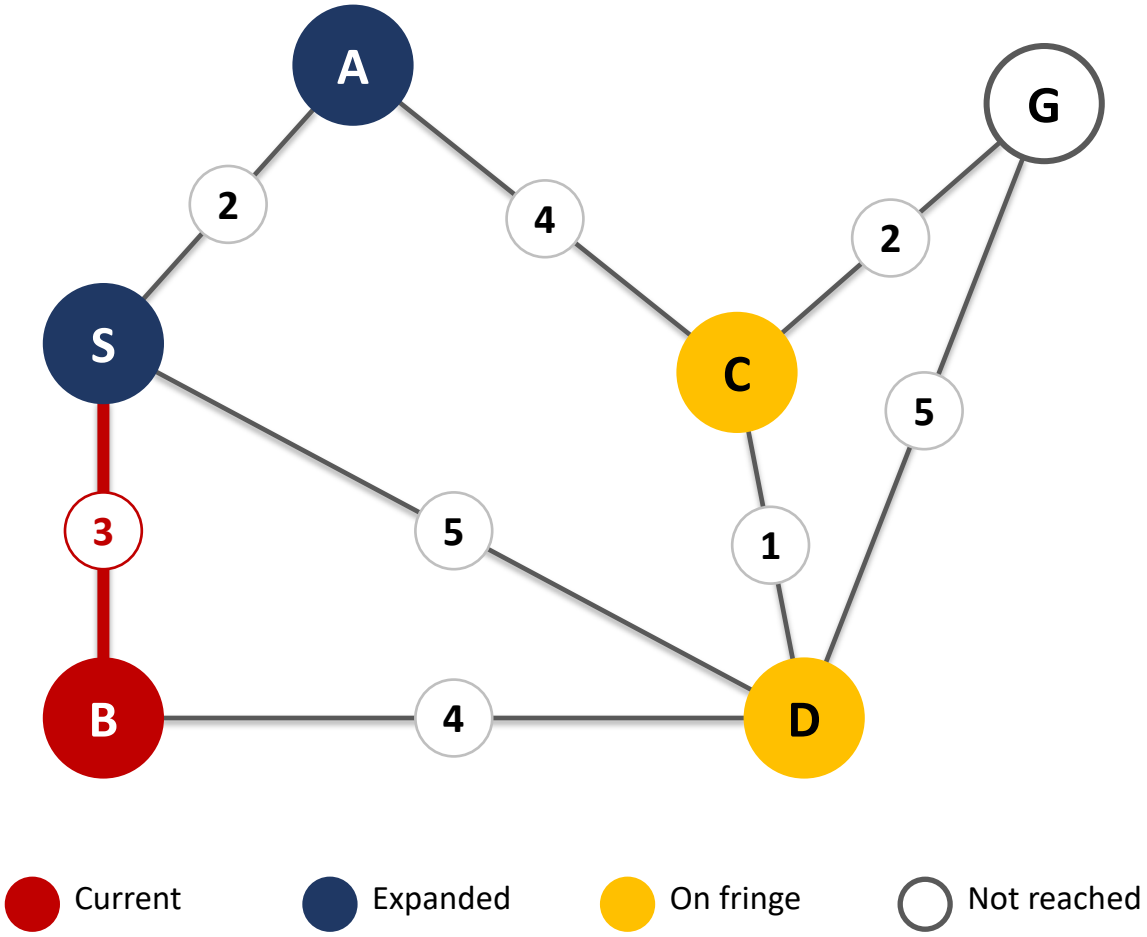
front ▸

- **S → B**
- S → D
- S → A → C (new)

Expanded: S, A

Pop S → A. A is not the goal, so **expand A**. Add S → A → C. (S already expanded: not added.)

BFS Worked Example (step 3 of 8)



Popped: S → B

Fringe: Queue (FIFO): front first

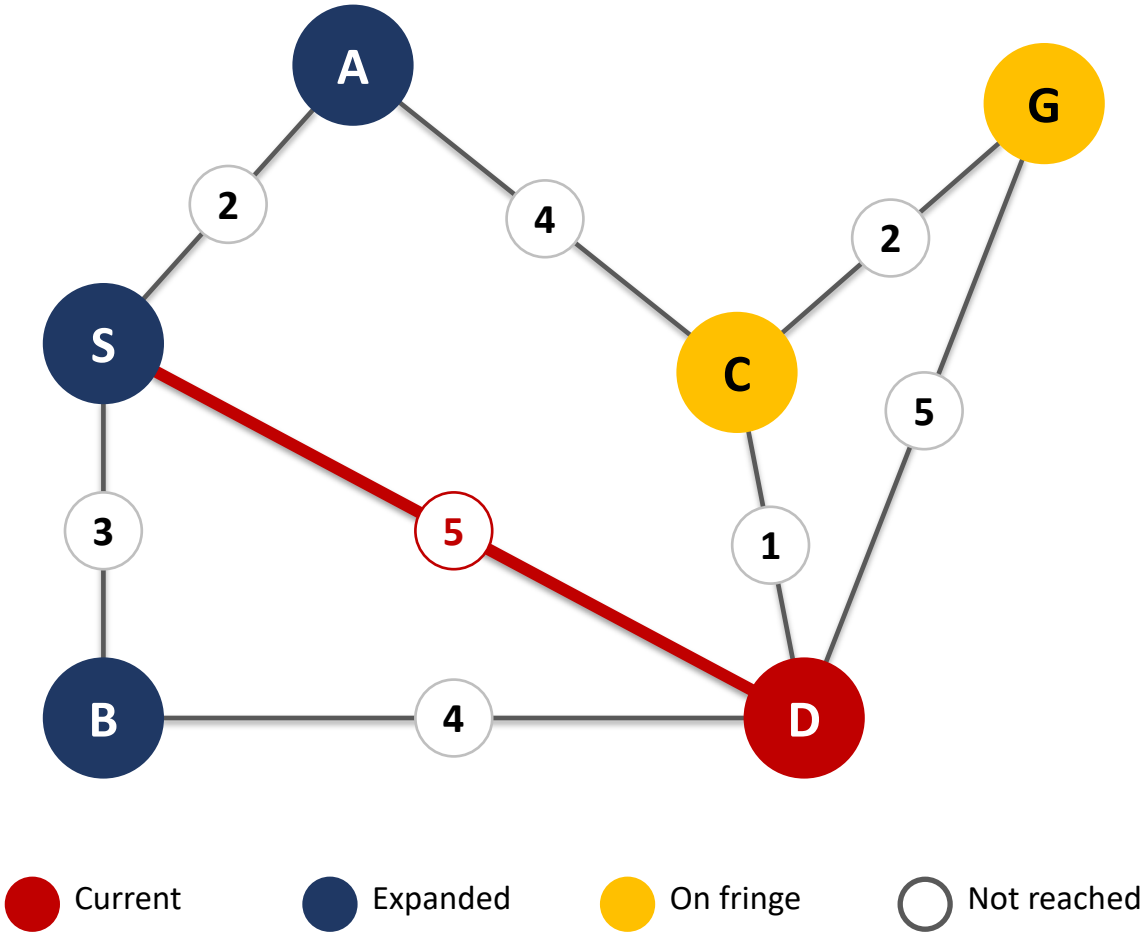
front ▸

- **S → D**
- S → A → C
- S → B → D (new)

Expanded: S, A, B

Pop S → B. B is not the goal, so **expand B**. Add S → B → D. (S already expanded: not added.)

BFS Worked Example (step 4 of 8)



Popped: S → D

Fringe: Queue (FIFO): front first

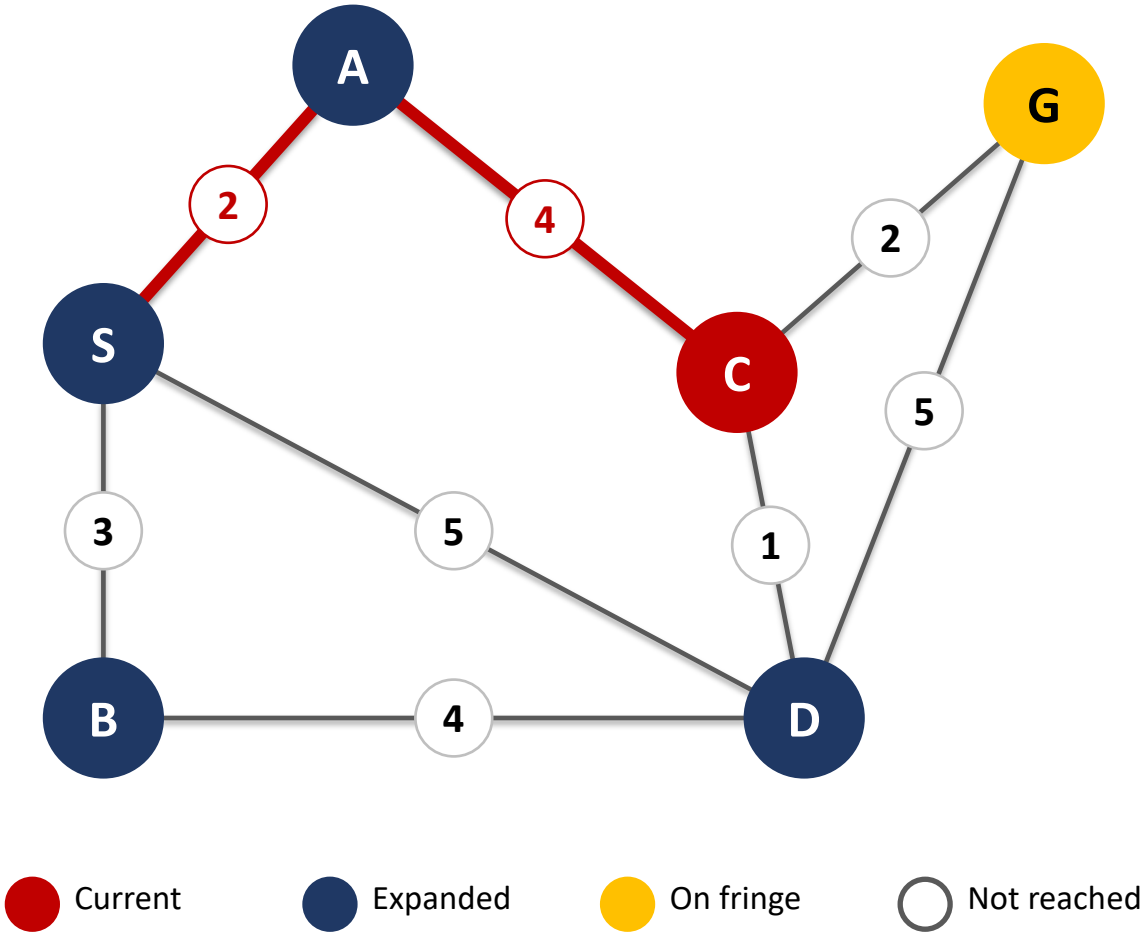
front ▸

- **S → A → C**
- S → B → D
- S → D → C (new)
- S → D → G (new)

Expanded: S, A, B, D

Pop S → D. D is not the goal, so **expand D**. Add S → D → C, S → D → G. (B, S already expanded: not added.)

BFS Worked Example (step 5 of 8)



Popped: S → A → C

Fringe: Queue (FIFO): front first

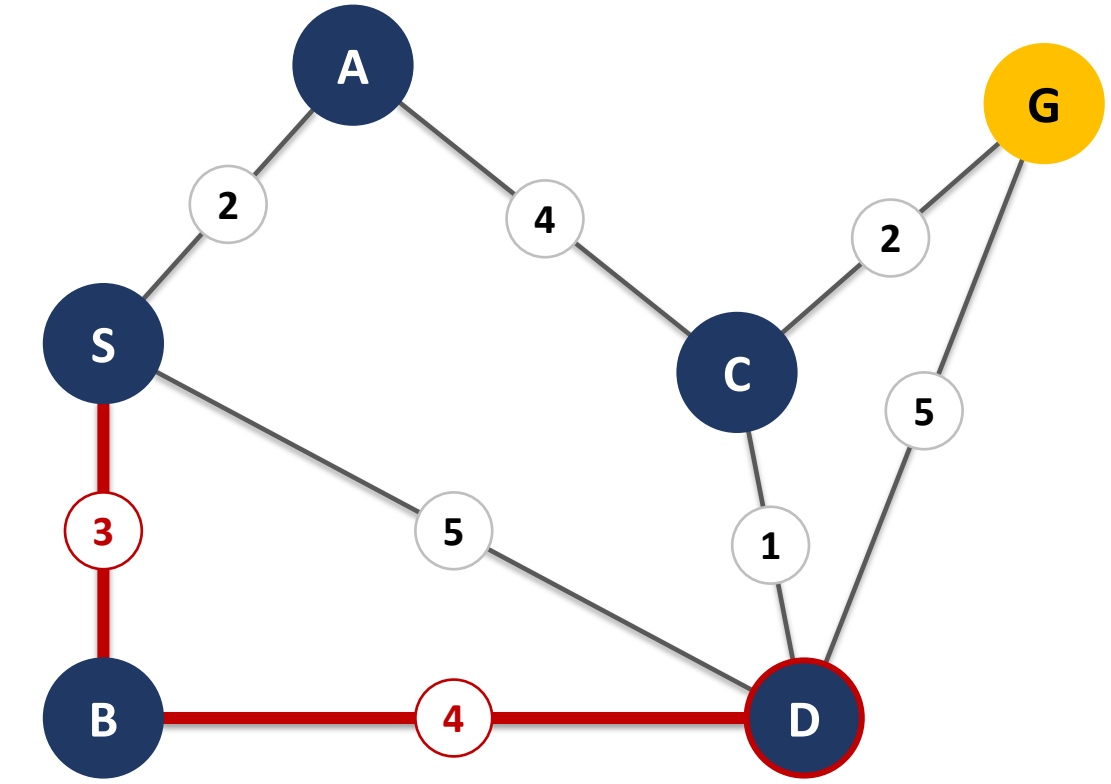
front ▸

- **S → B → D**
- S → D → C
- S → D → G
- S → A → C → G (new)

Expanded: S, A, B, D, C

Pop S → A → C. C is not the goal, so **expand C**. Add S → A → C → G. (A, D already expanded: not added.)

BFS Worked Example (step 6 of 8)



Current Expanded On fringe Not reached

Popped (skip): S → B → D

Fringe: Queue (FIFO): front first

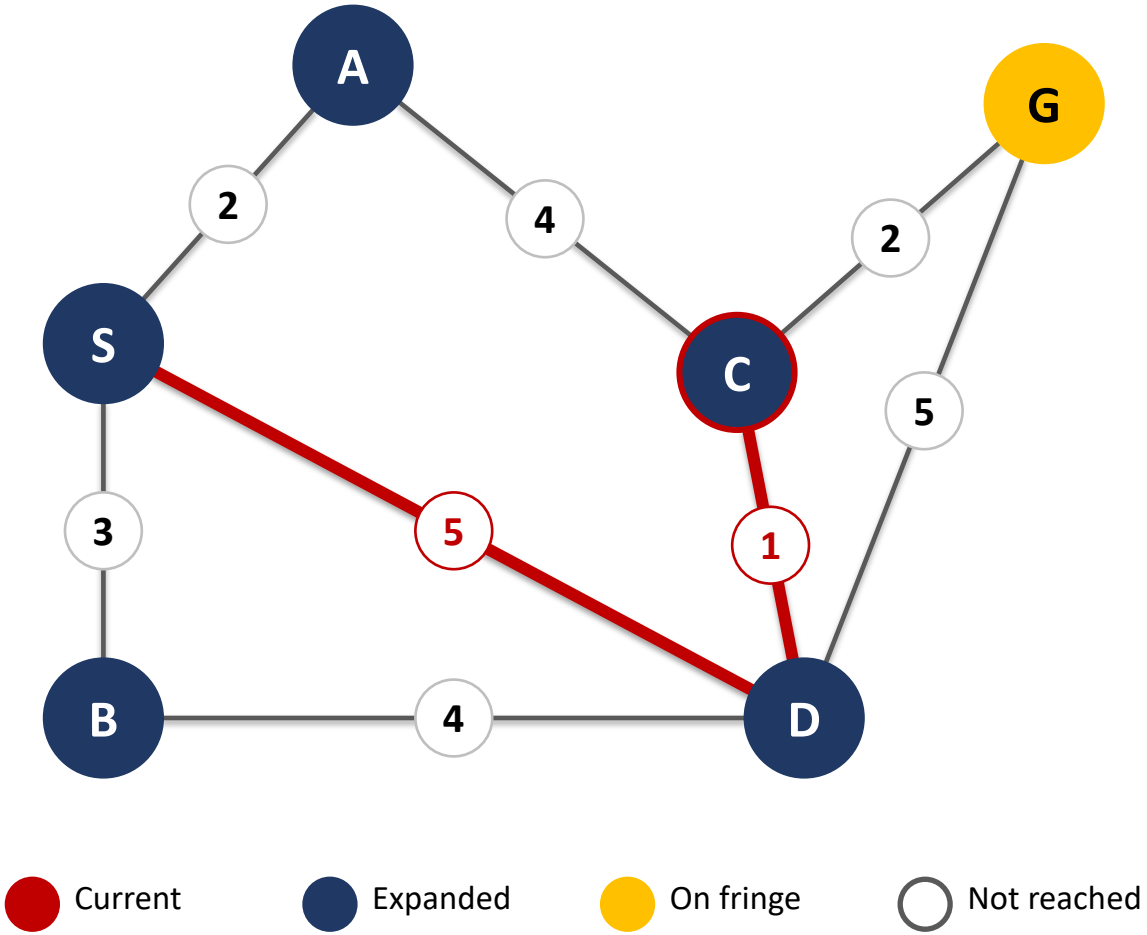
front ▸

S → D → C
S → D → G
S → A → C → G

Expanded: S, A, B, D, C

Pop S → B → D. **D was already expanded**, so graph search discards this entry.

BFS Worked Example (step 7 of 8)



Popped (skip): S → D → C

Fringe: Queue (FIFO): front first

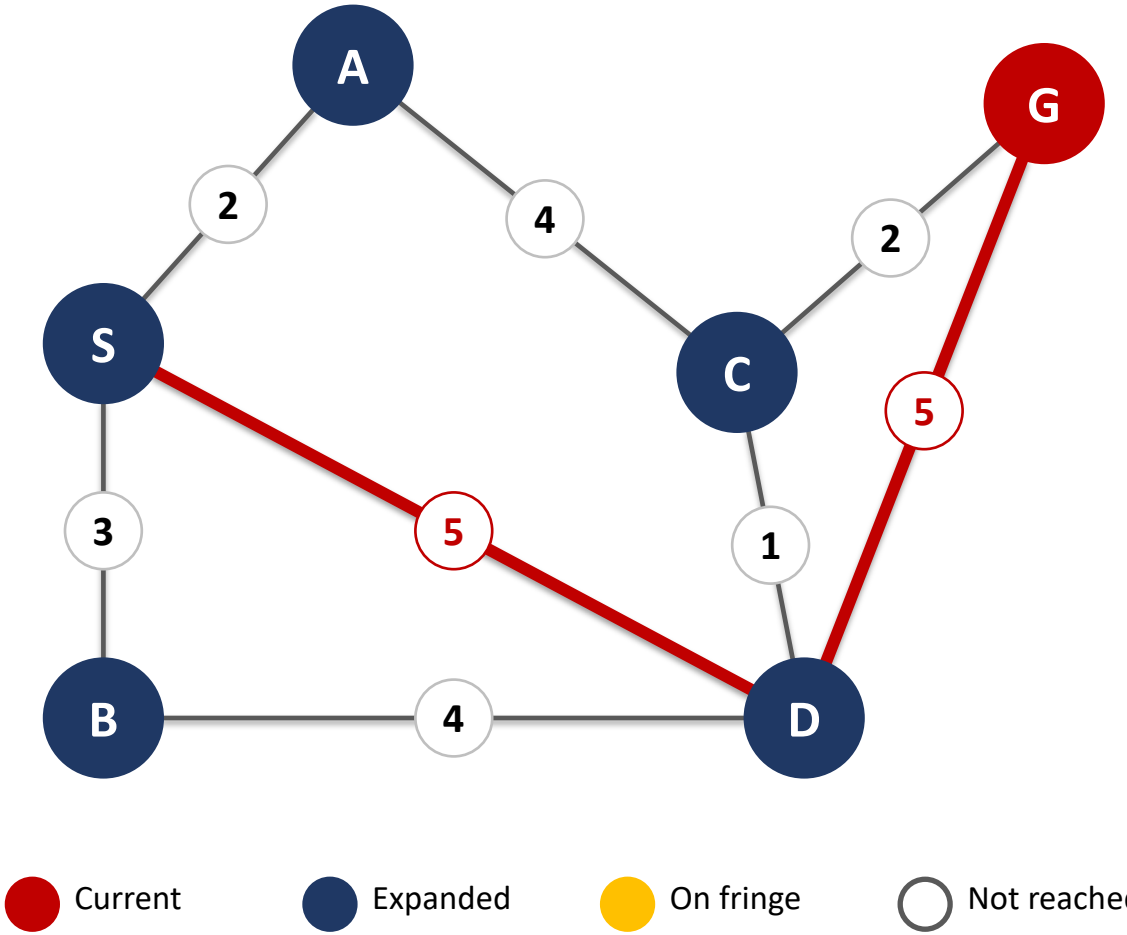
front ▸ **S → D → G**

S → A → C → G

Expanded: S, A, B, D, C

Pop S → D → C. **C was already expanded**, so graph search discards this entry.

BFS Worked Example (step 8 of 8)



Popped (goal!): S → D → G

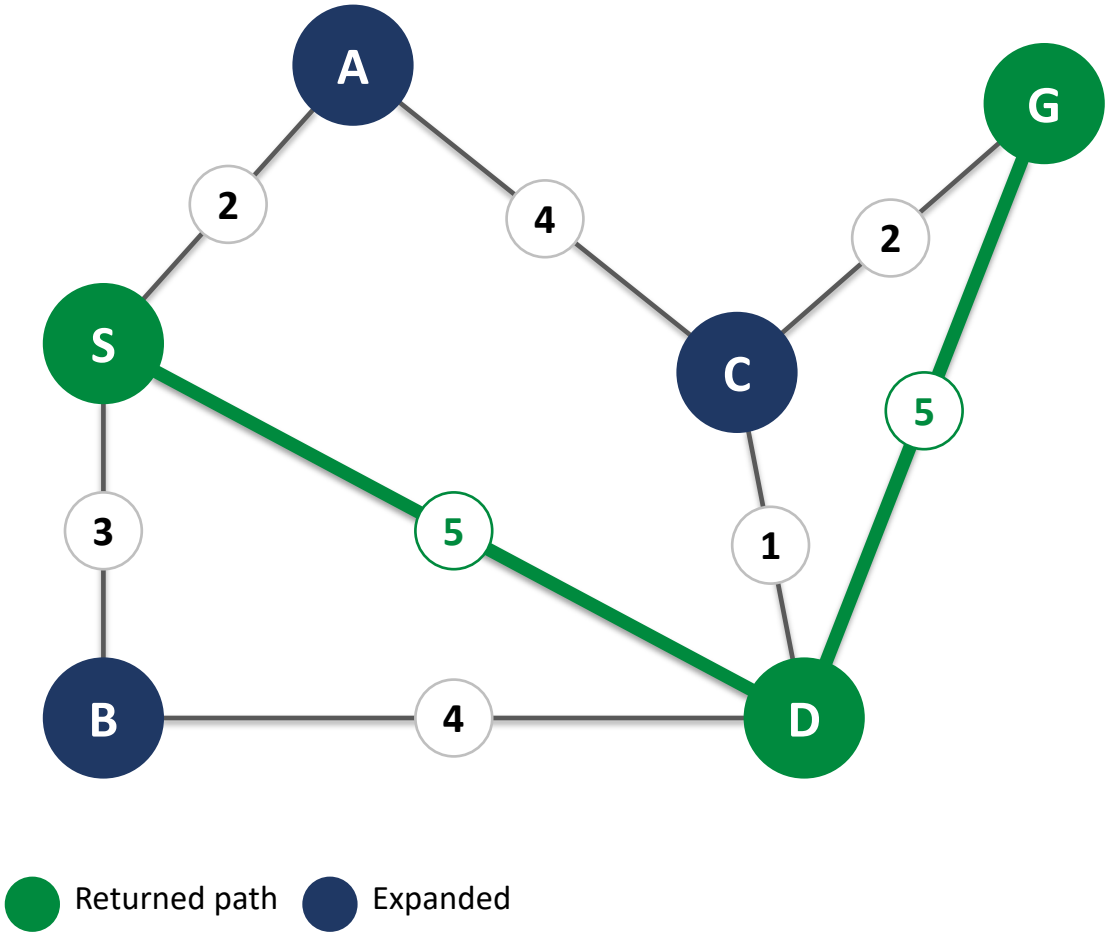
Fringe: Queue (FIFO): front first

front ▸ **S → A → C → G**

Expanded: S, A, B, D, C, G

Pop S → D → G. **G is the goal**, so return this path. Total cost = 10.

BFS Result



Expansion order: S, A, B, D, C, G

Path returned: S → D → G

Cost: 5 + 5 = 10

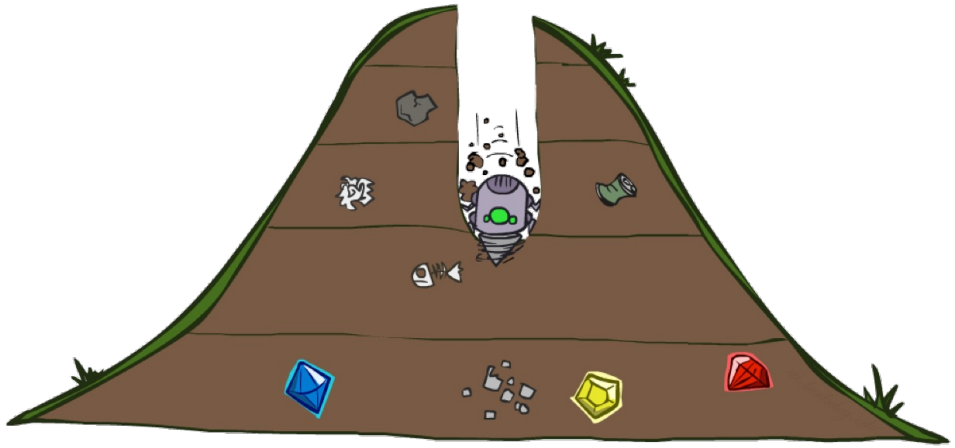
Actions: 2

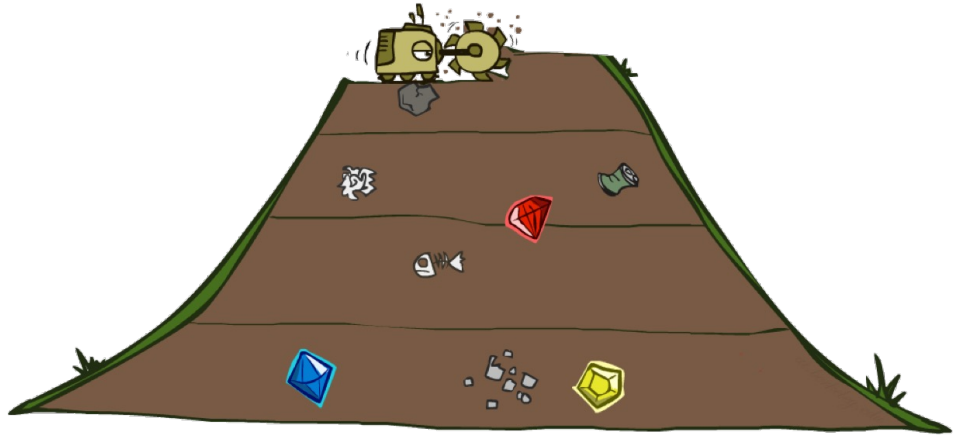
BFS returns the path with the **fewest actions** (2).

But it ignores edge costs: S → D → G costs 10, while S → A → C → G costs 8.

Two entries were skipped because their states were already expanded.

Quiz: DFS vs BFS





Quiz: DFS vs BFS

- When will BFS outperform DFS?

- When will DFS outperform BFS?

[Demo: dfs/bfs maze water (L2D6)]

Video of Demo Maze Water DFS/BFS (part 1)

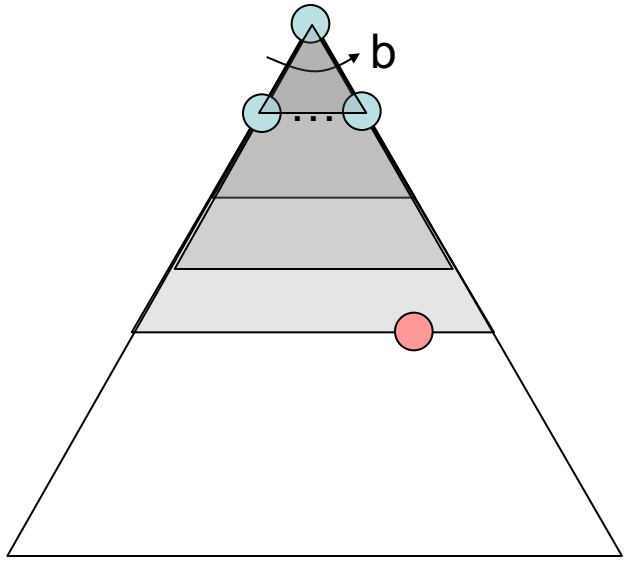


Video of Demo Maze Water DFS/BFS (part 2)



Iterative Deepening

- Idea: get DFS's space advantage with BFS's time / shallow-solution advantages
  - Run a DFS with depth limit 1. If no solution…
  - Run a DFS with depth limit 2. If no solution…
  - Run a DFS with depth limit 3. …..
- Isn't that wastefully redundant?
  - Generally most work happens in the lowest level searched, so not so bad!



Iterative Deepening: Worked Example

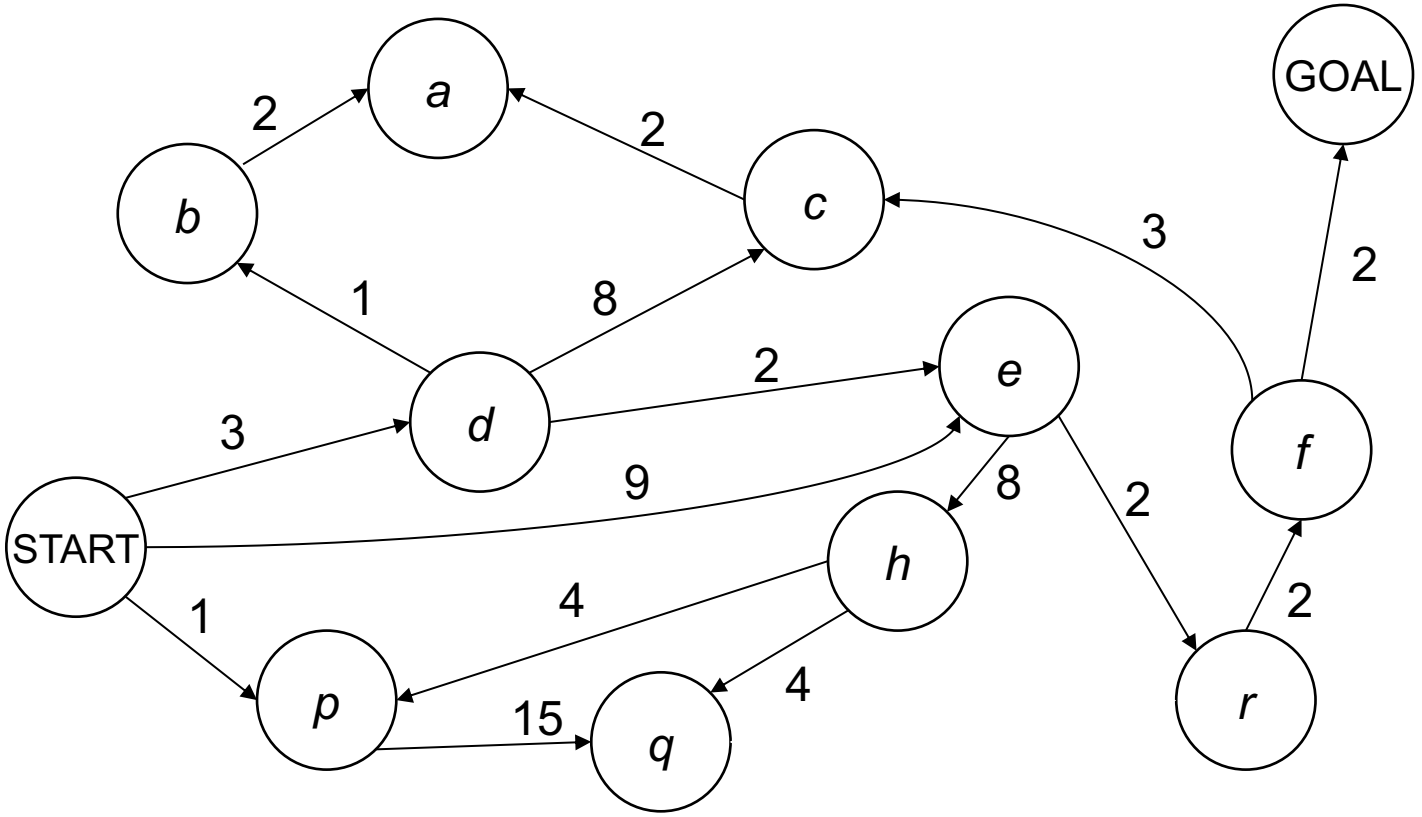
Depth limit	Nodes visited (in order)	Result
0	S	no goal
1	S, A, B, D	no goal
2	S, A, C, B, D, D, B, C, G	**found S → D → G**

Each iteration runs a depth-limited DFS from scratch on the same graph (avoiding cycles along the current path).

IDS finds the **shallowest** goal, the same 2-action path as BFS, while storing only one path at a time like DFS.

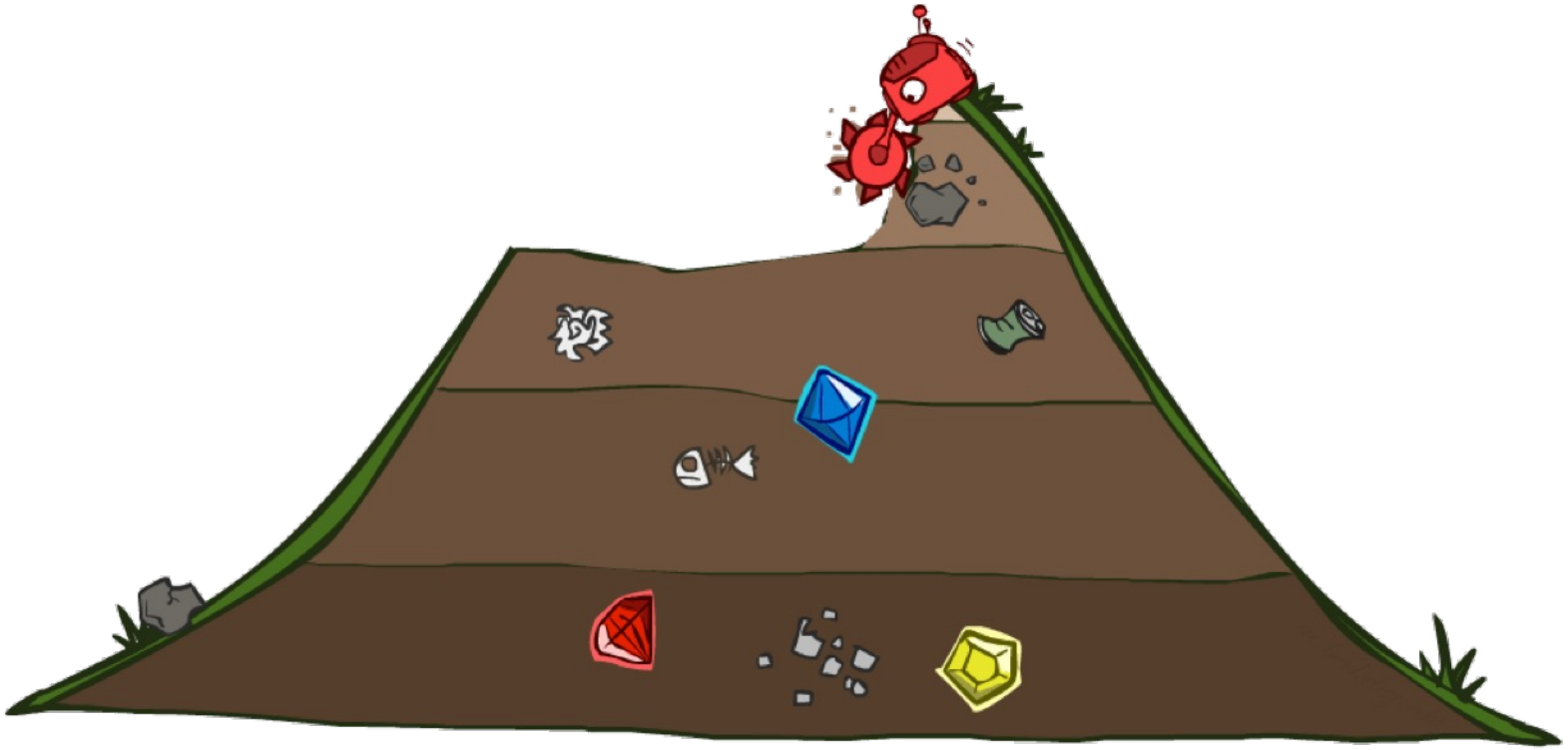
Repeating the shallow levels is cheap: most of the work happens at the deepest level searched.

Cost-Sensitive Search



BFS finds the shortest path in terms of number of actions.
It does not find the least-cost path. We will now cover
a similar algorithm which does find the least-cost path.

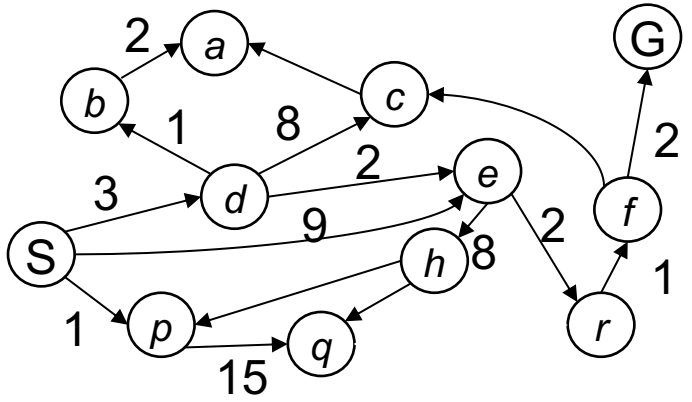
Uniform Cost Search



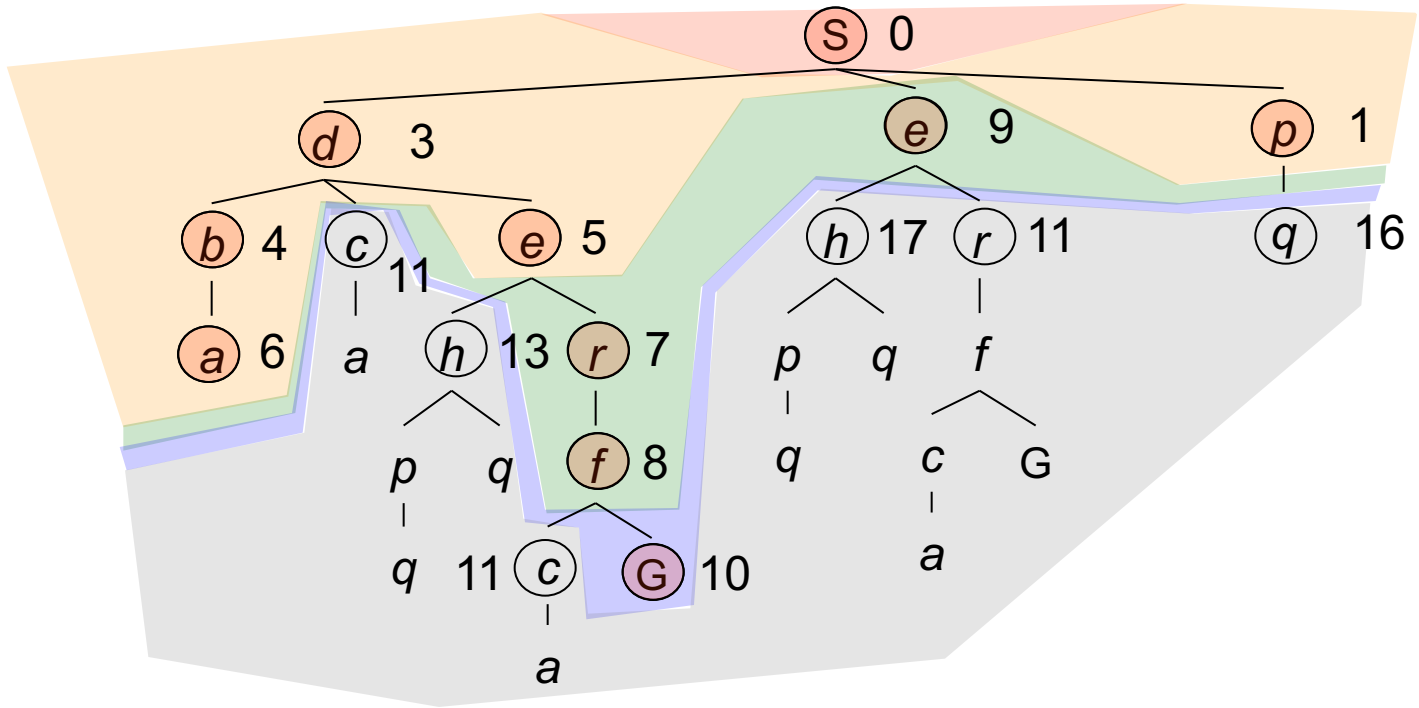
Uniform Cost Search

Strategy: expand a cheapest node first:

Fringe is a priority queue (priority: cumulative cost)

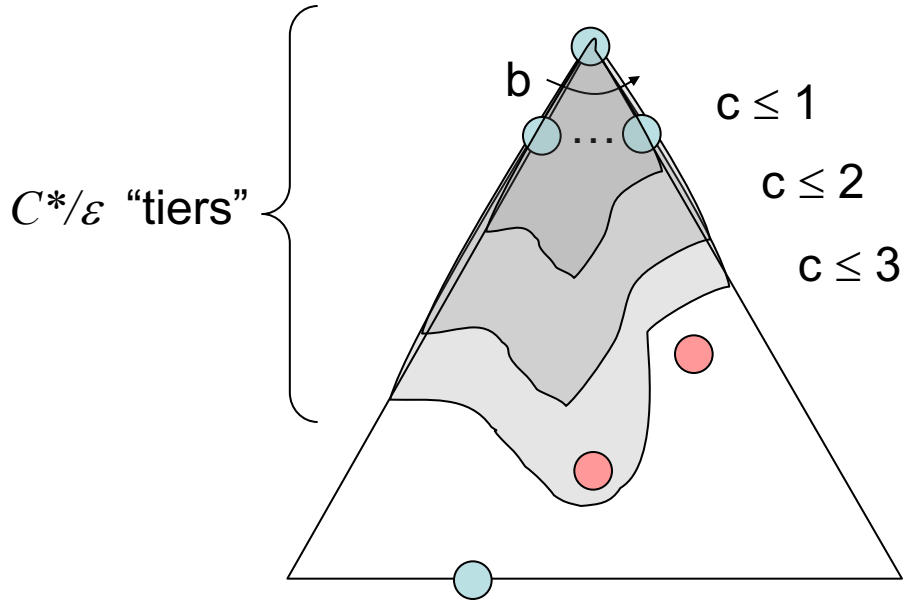


Cost contours



Uniform Cost Search (UCS) Properties

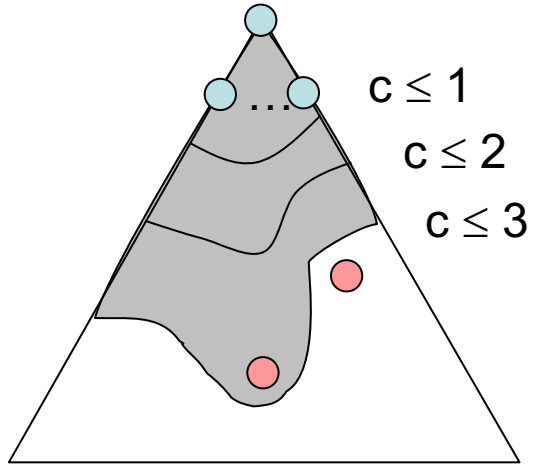
- What nodes does UCS expand?
  - Processes all nodes with cost less than cheapest solution!
  - If that solution costs C^* and arcs cost at least ε, then the "effective depth" is roughly C^*/ε
  - Takes time $O(b^{C^*/\varepsilon})$ (exponential in effective depth)



- How much space does the fringe take?
  - Has roughly the last tier, so $O(b^{C^*/\varepsilon})$
- Is it complete?
  - Assuming best solution has a finite cost and minimum arc cost is positive, yes!
- Is it optimal?
  - Yes! (Proof next lecture via A*)

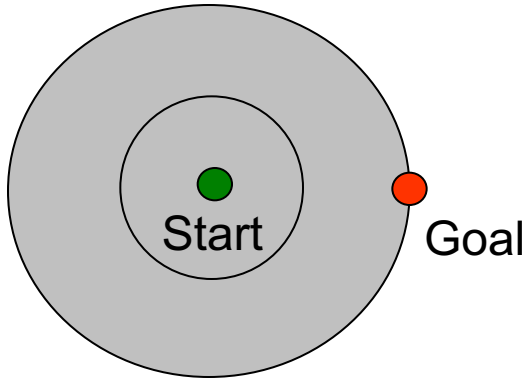
Uniform Cost Issues

- Remember: UCS explores increasing cost contours



- The good: UCS is complete and optimal!

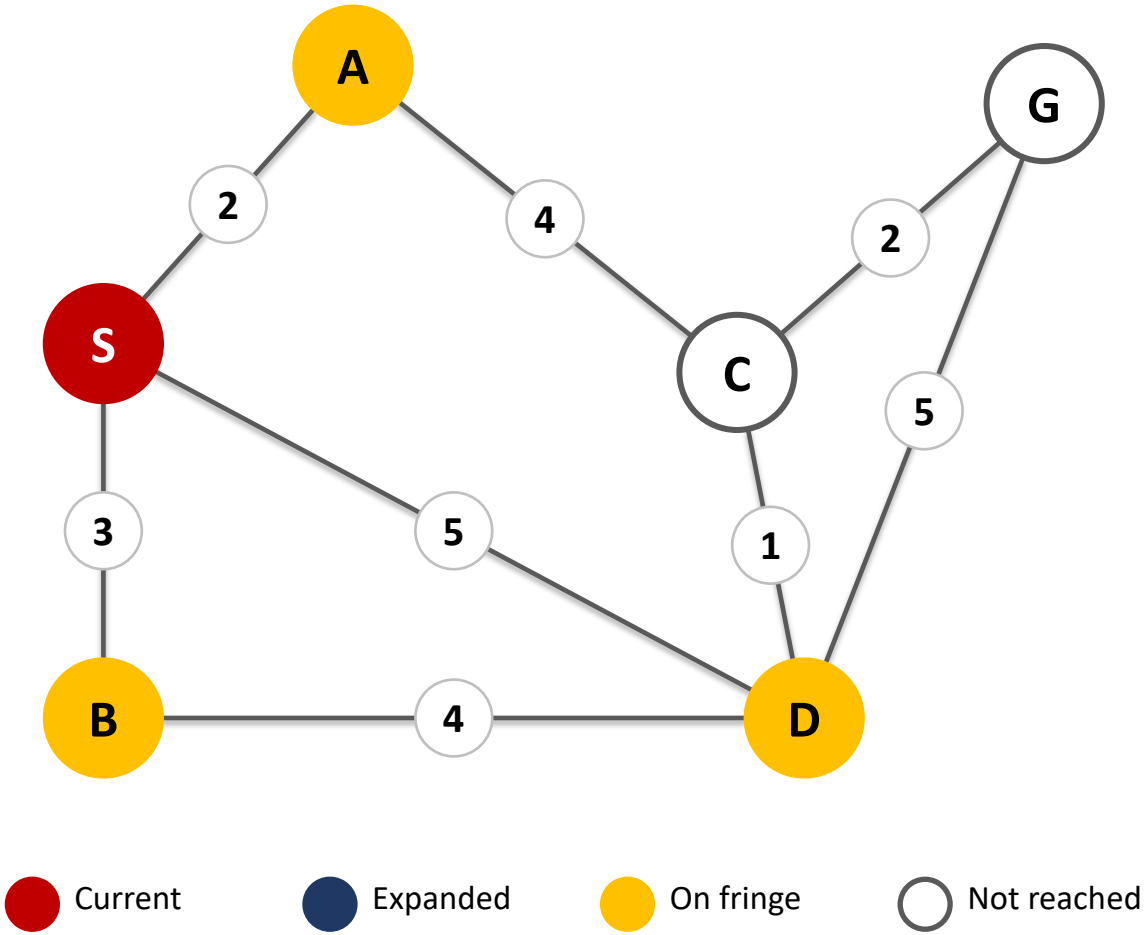
- The bad:
  - Explores options in every “direction”
  - No information about goal location



- We’ll fix that soon!

[Demo: empty grid UCS (L2D5)]
[Demo: maze with deep/shallow water DFS/BFS/UCS (L2D7)]

UCS Worked Example (step 1 of 8)



Popped: S (g = 0)

Fringe: Priority queue: cheapest first

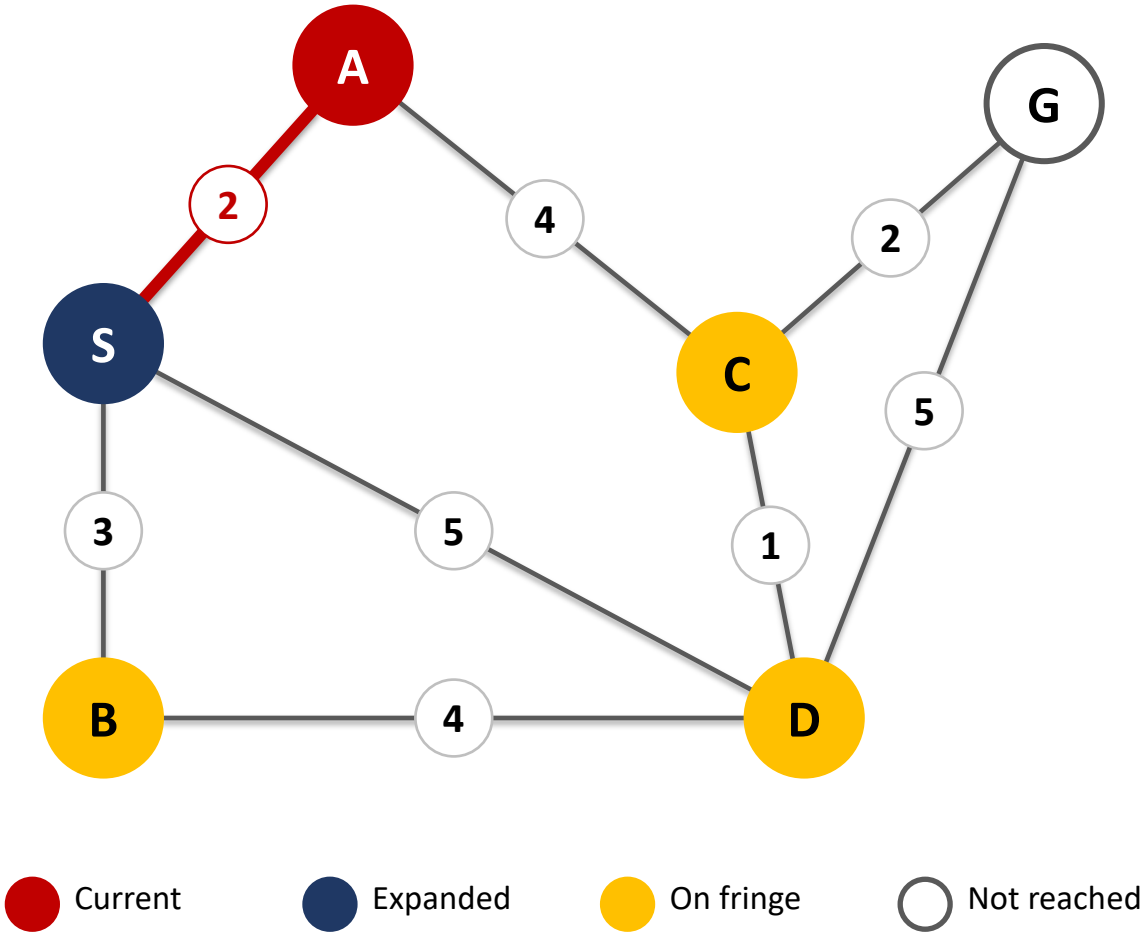
next ▸

S → A g = 2 (new)
S → B g = 3 (new)
S → D g = 5 (new)

Expanded: S

Pop S (cost 0). S is not the goal, so **expand S**. Add S → A (2), S → B (3), S → D (5).

UCS Worked Example (step 2 of 8)



Popped: S → A (g = 2)

Fringe: Priority queue: cheapest first

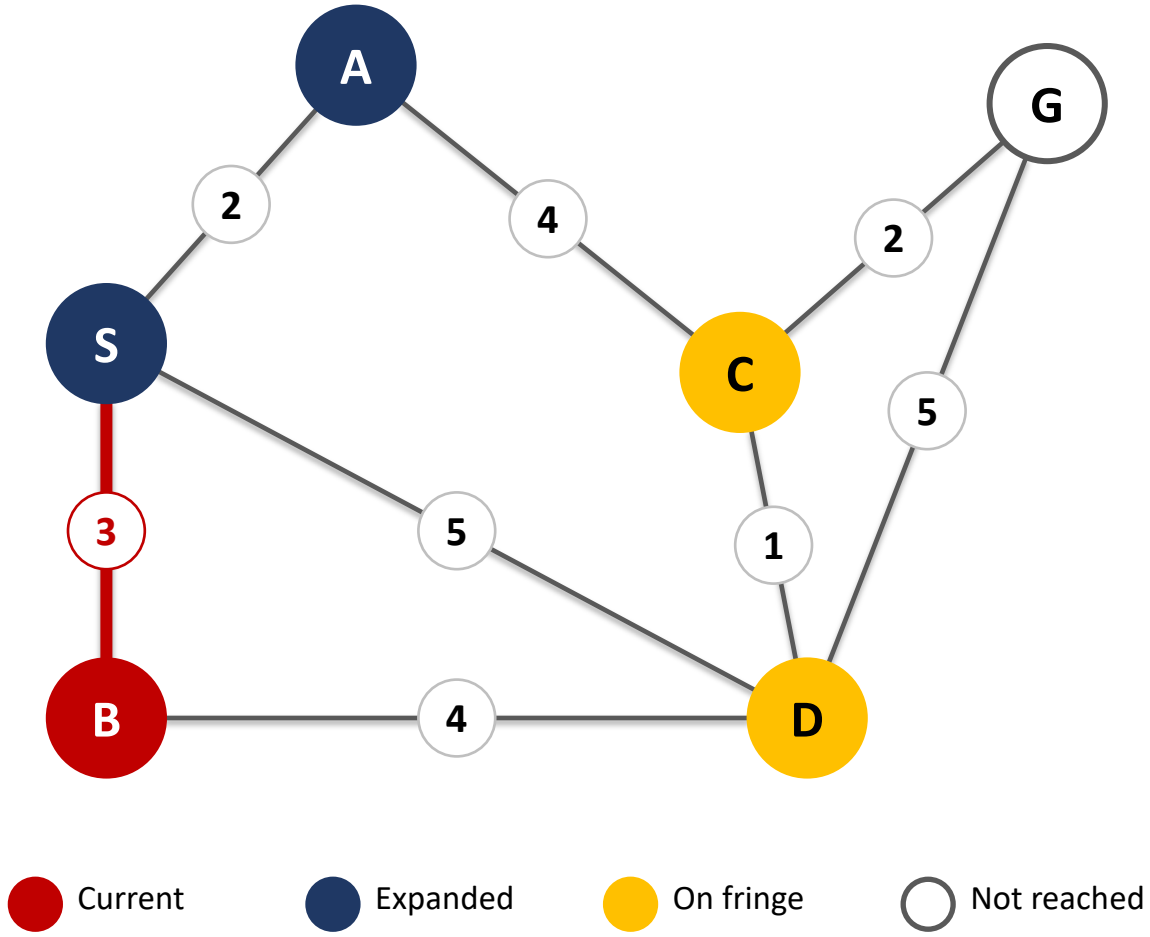
next ▸

S → B g = 3
S → D g = 5
S → A → C g = 6 (new)

Expanded: S, A

Pop S → A (cost 2). A is not the goal, so **expand A**. Add S → A → C (6). (S already expanded: not added.)

UCS Worked Example (step 3 of 8)



Popped: S → B (g = 3)

Fringe: Priority queue: cheapest first

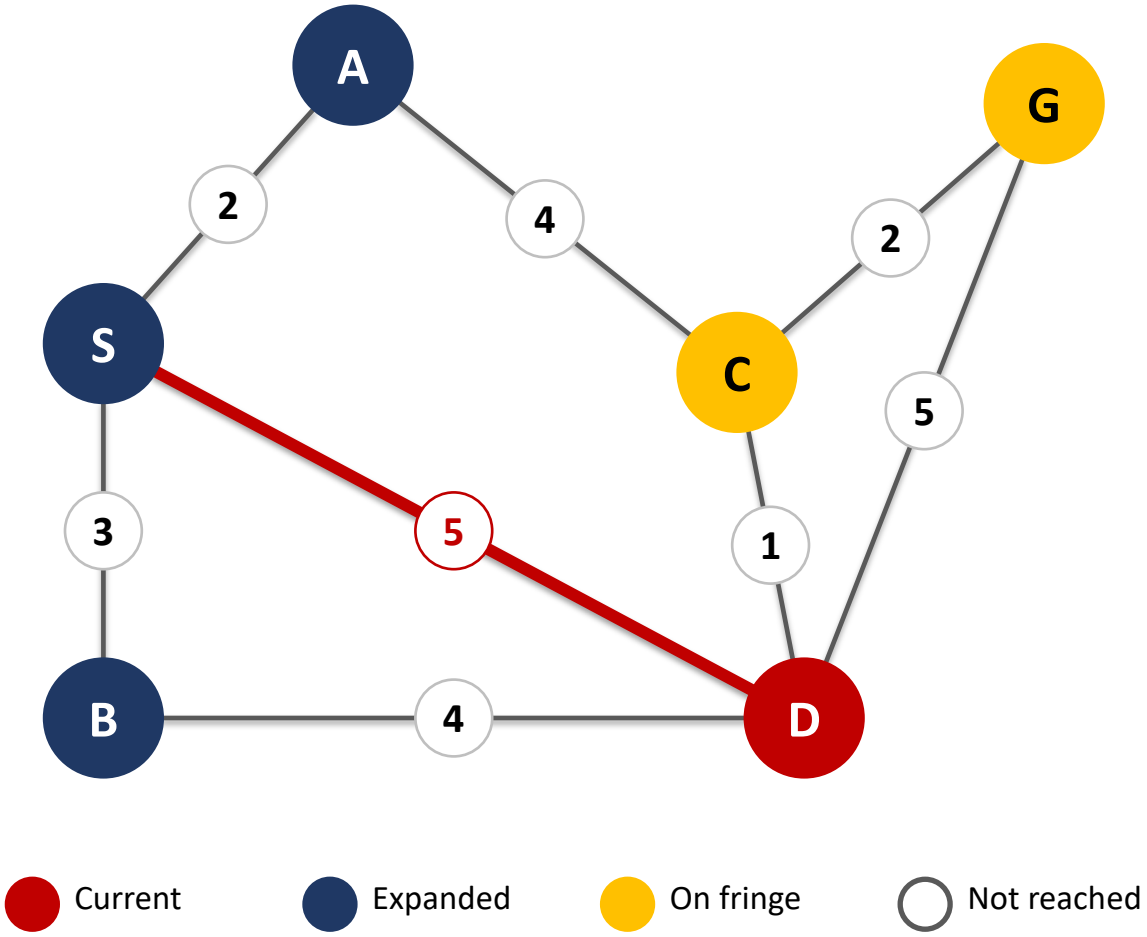
next ▸

S → D g = 5
S → A → C g = 6
S → B → D g = 7 (new)

Expanded: S, A, B

Pop S → B (cost 3). B is not the goal, so **expand B**. Add S → B → D (7). (S already expanded: not added.)

UCS Worked Example (step 4 of 8)



Popped: S → D (g = 5)

Fringe: Priority queue: cheapest first

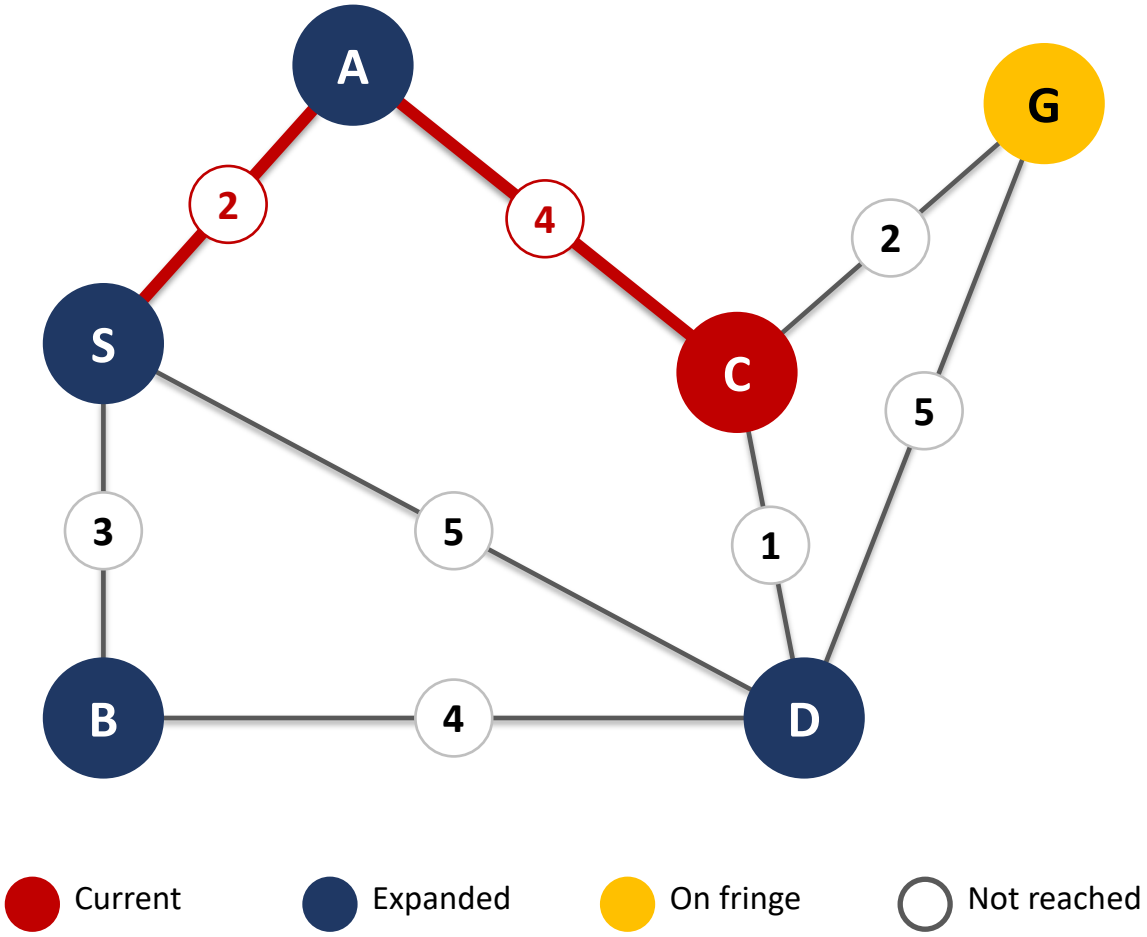
next ▸

S → A → C g = 6
S → D → C g = 6 (new)
S → B → D g = 7
S → D → G g = 10 (new)

Expanded: S, A, B, D

Pop S → D (cost 5). D is not the goal, so **expand D**. Add S → D → C (6), S → D → G (10). (B, S already expanded: not added.)

UCS Worked Example (step 5 of 8)



Popped: S → A → C (g = 6)

Fringe: Priority queue: cheapest first

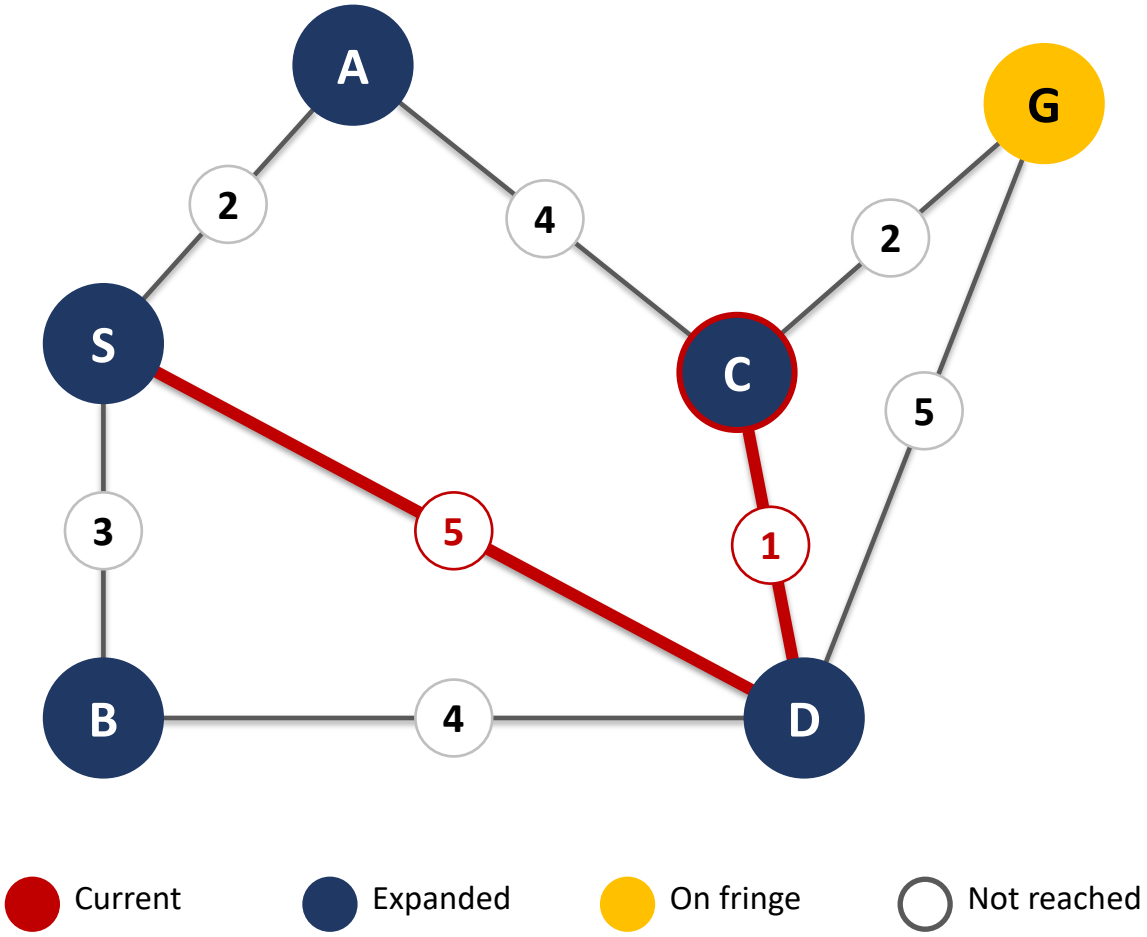
next ▸

S → D → C g = 6
S → B → D g = 7
S → A → C → G g = 8 (new)
S → D → G g = 10

Expanded: S, A, B, D, C

Pop S → A → C (cost 6). C is not the goal, so **expand C**. Add S → A → C → G (8). (A, D already expanded: not added.)

UCS Worked Example (step 6 of 8)



Popped (skip): S → D → C (g = 6)

Fringe: Priority queue: cheapest first

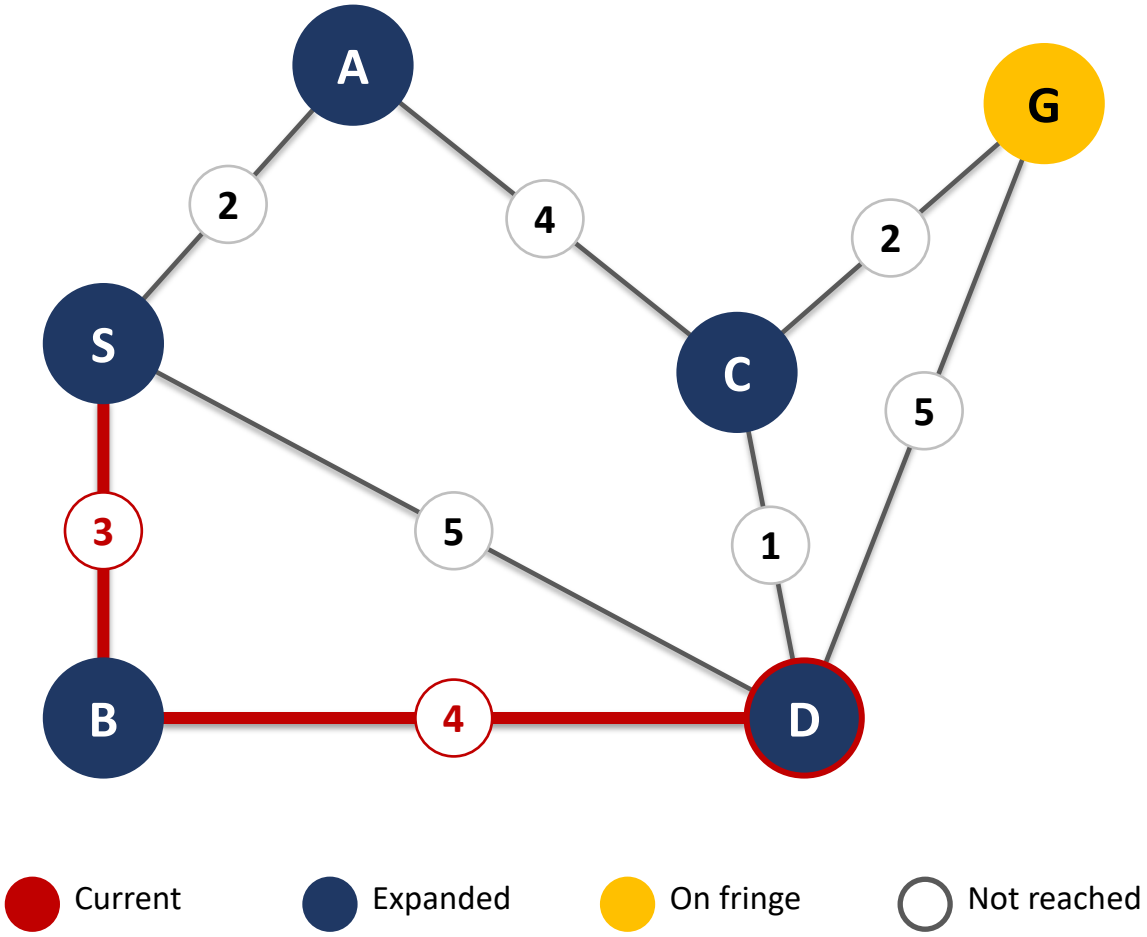
next ▸

S → B → D g = 7
S → A → C → G g = 8
S → D → G g = 10

Expanded: S, A, B, D, C

Pop S → D → C (cost 6). **C was already expanded**, so graph search discards this entry.

UCS Worked Example (step 7 of 8)



Popped (skip): S → B → D (g = 7)

Fringe: Priority queue: cheapest first

next ▸

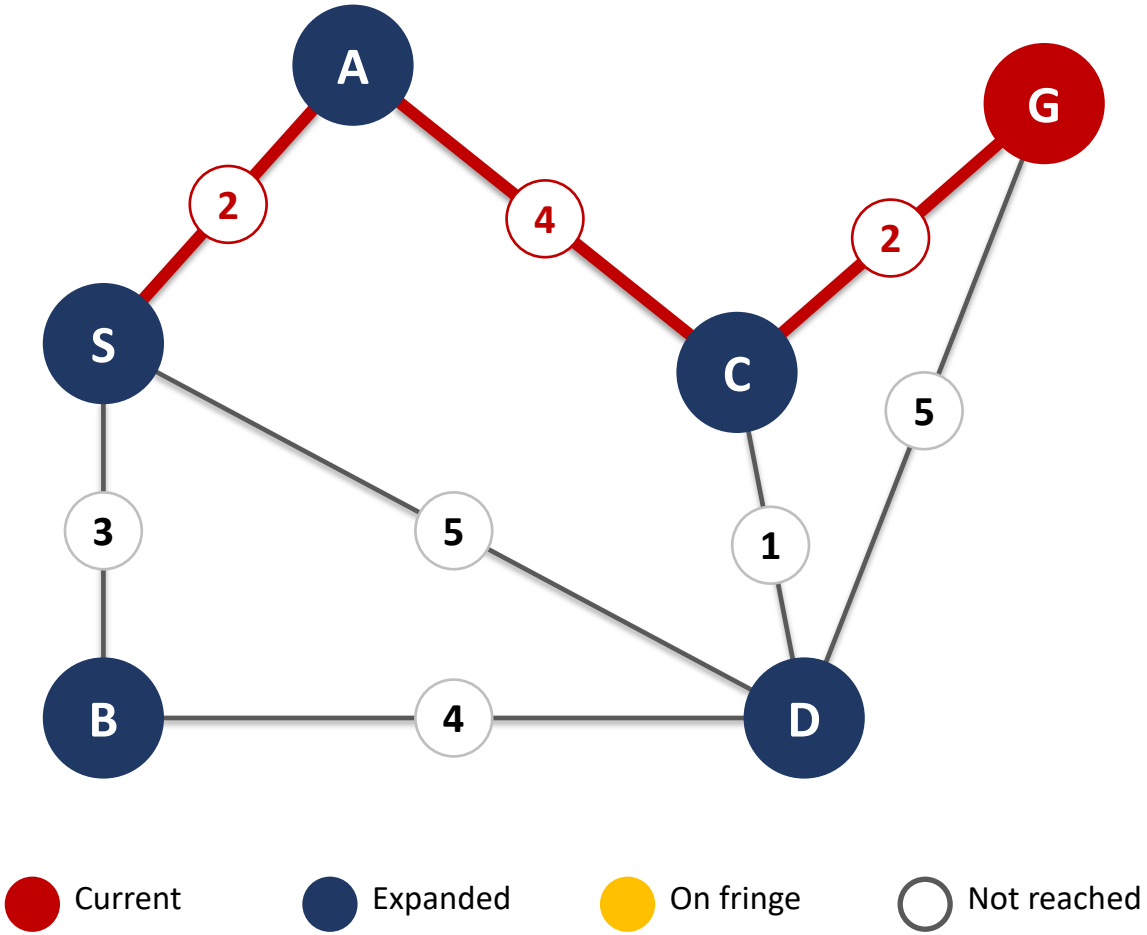
S → A → C → G g = 8

S → D → G g = 10

Expanded: S, A, B, D, C

Pop S → B → D (cost 7). **D was already expanded**, so graph search discards this entry.

UCS Worked Example (step 8 of 8)



Popped (goal!): S → A → C → G (g = 8)

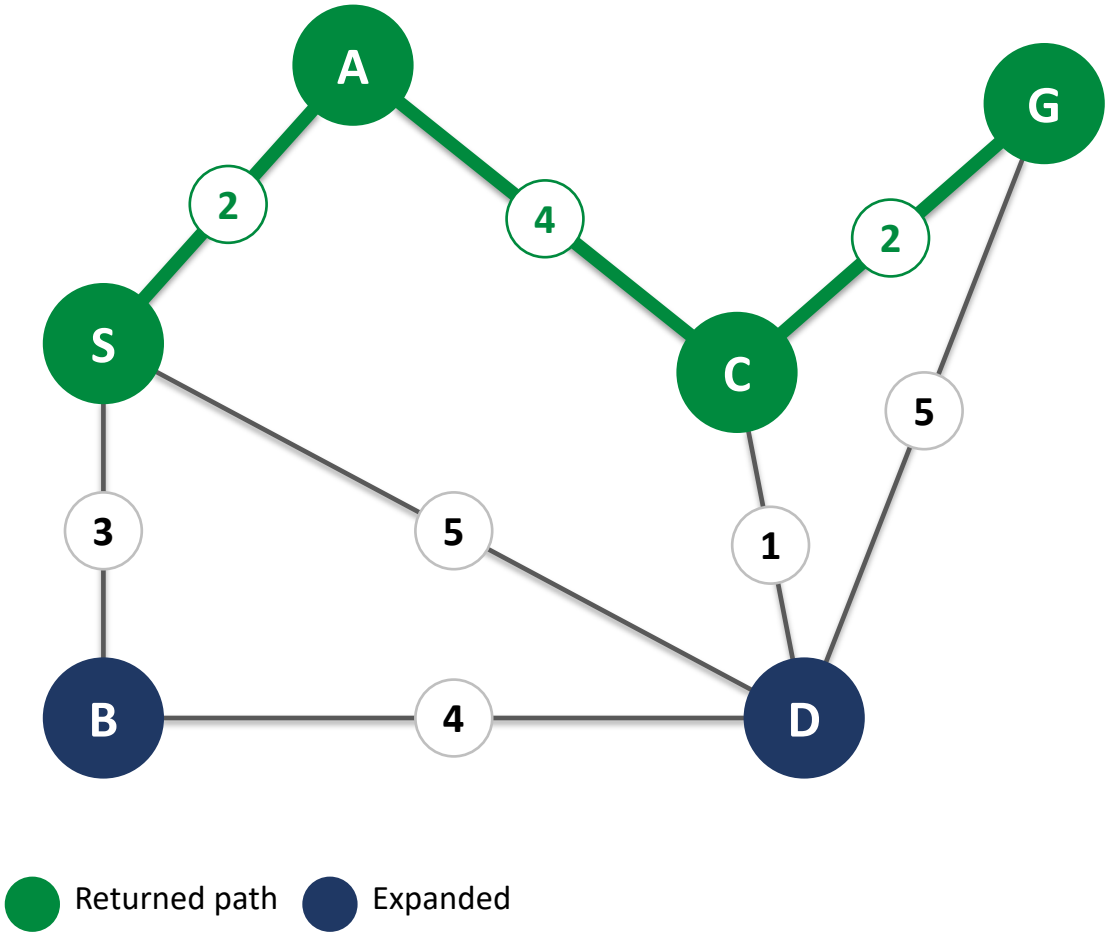
Fringe: Priority queue: cheapest first

next ▸ **S → D → G g = 10**

Expanded: S, A, B, D, C, G

Pop S → A → C → G (cost 8). **G is the goal**, so return this path. Total cost = 8.

UCS Result



Expansion order: S, A, B, D, C, G

Path returned: S → A → C → G

Cost: 2 + 4 + 2 = 8

Actions: 3

UCS returns the **cheapest path** (cost 8).

It expanded every state reachable for less than 8 before reaching G.

Notice S → D → G (cost 10) sat on the fringe, but a cheaper path reached G first.

Video of Demo Empty UCS



Video of Demo Maze with Deep/Shallow Water --- DFS, BFS, or UCS? (part 1)



Video of Demo Maze with Deep/Shallow Water --- DFS, BFS, or UCS? (part 2)



Video of Demo Maze with Deep/Shallow Water --- DFS, BFS, or UCS? (part 3)



Same Graph, Three Strategies

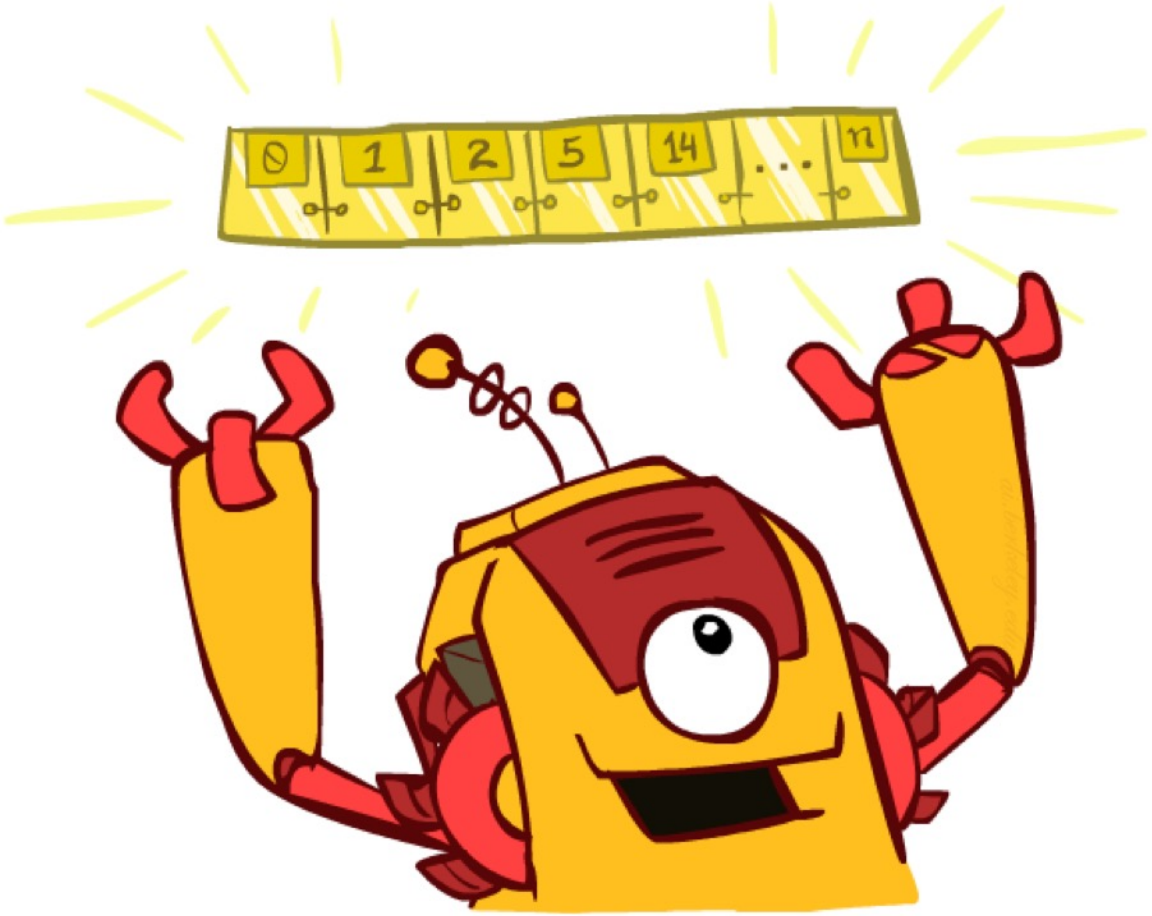
Algorithm	Fringe	Expansion order	Path returned	Cost
DFS	Stack	S, A, C, D, B, G	S → A → C → D → G	12
BFS	Queue	S, A, B, D, C, G	S → D → G	10
UCS	Priority queue	S, A, B, D, C, G	S → A → C → G	8

	Complete?	Optimal?	Time	Space
DFS	Only if depth is finite	No	O(b^m)	O(bm)
BFS	Yes	Only if costs are equal	O(b^s)	O(b^s)
IDS	Yes	Only if costs are equal	O(b^s)	O(bs)
UCS	Yes*	Yes*	O(b^(C*/ε))	O(b^(C*/ε))

b = branching factor, m = max depth, s = depth of shallowest goal, C* = optimal cost, ε = smallest step cost. * if every step cost ≥ ε > 0

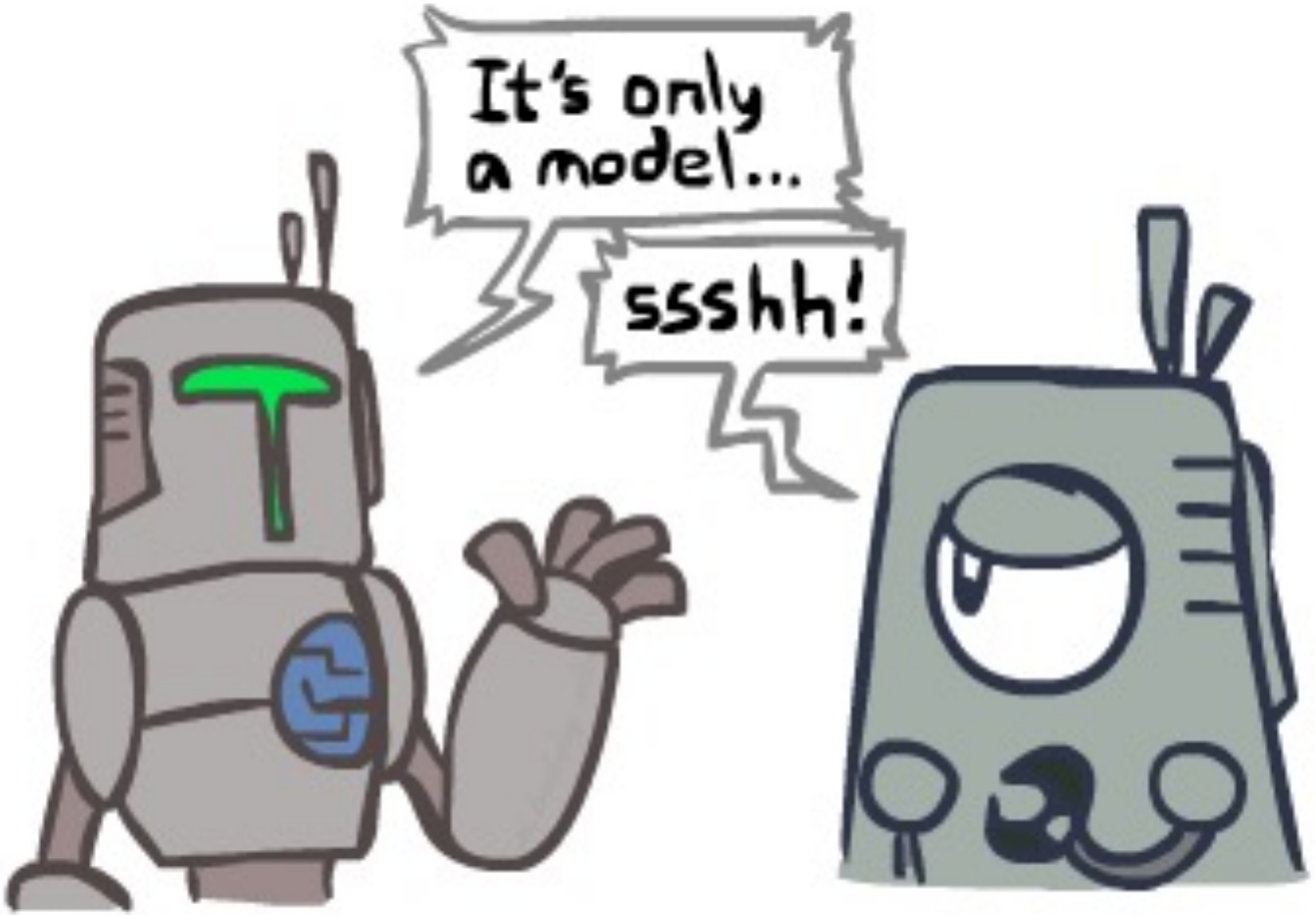
The One Queue

- All these search algorithms are the same except for fringe strategies
  - Conceptually, all fringes are priority queues (i.e. collections of nodes with attached priorities)
  - Practically, for DFS and BFS, you can avoid the log(n) overhead from an actual priority queue, by using stacks and queues
  - Can even code one implementation that takes a variable queuing object



Search and Models

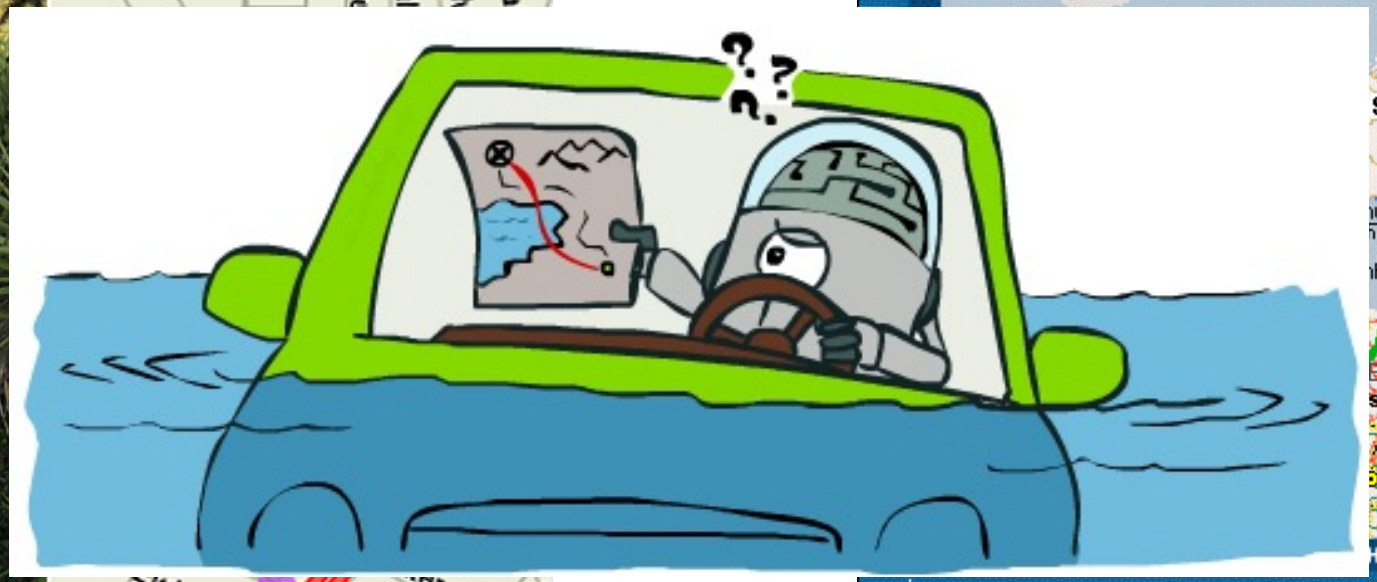
- Search operates over models of the world
  - The agent doesn't actually try all the plans out in the real world!
  - Planning is all "in simulation"
  - Your search is only as good as your models…



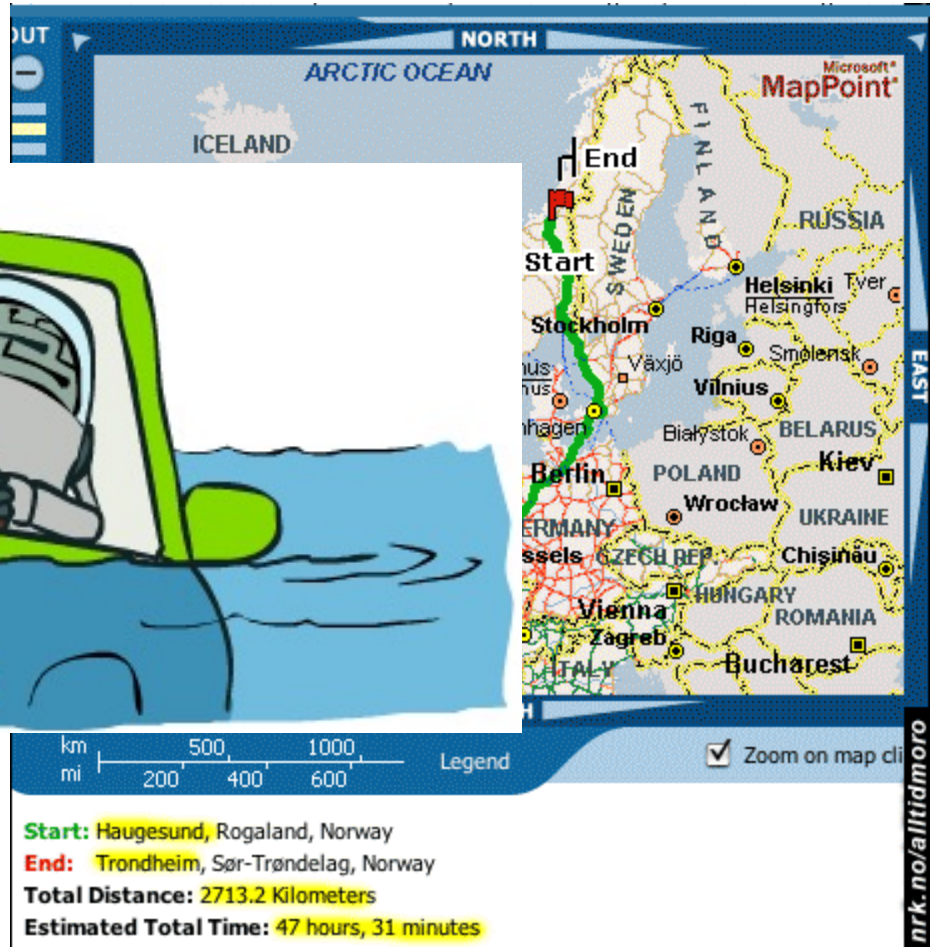
Search Gone Wrong?



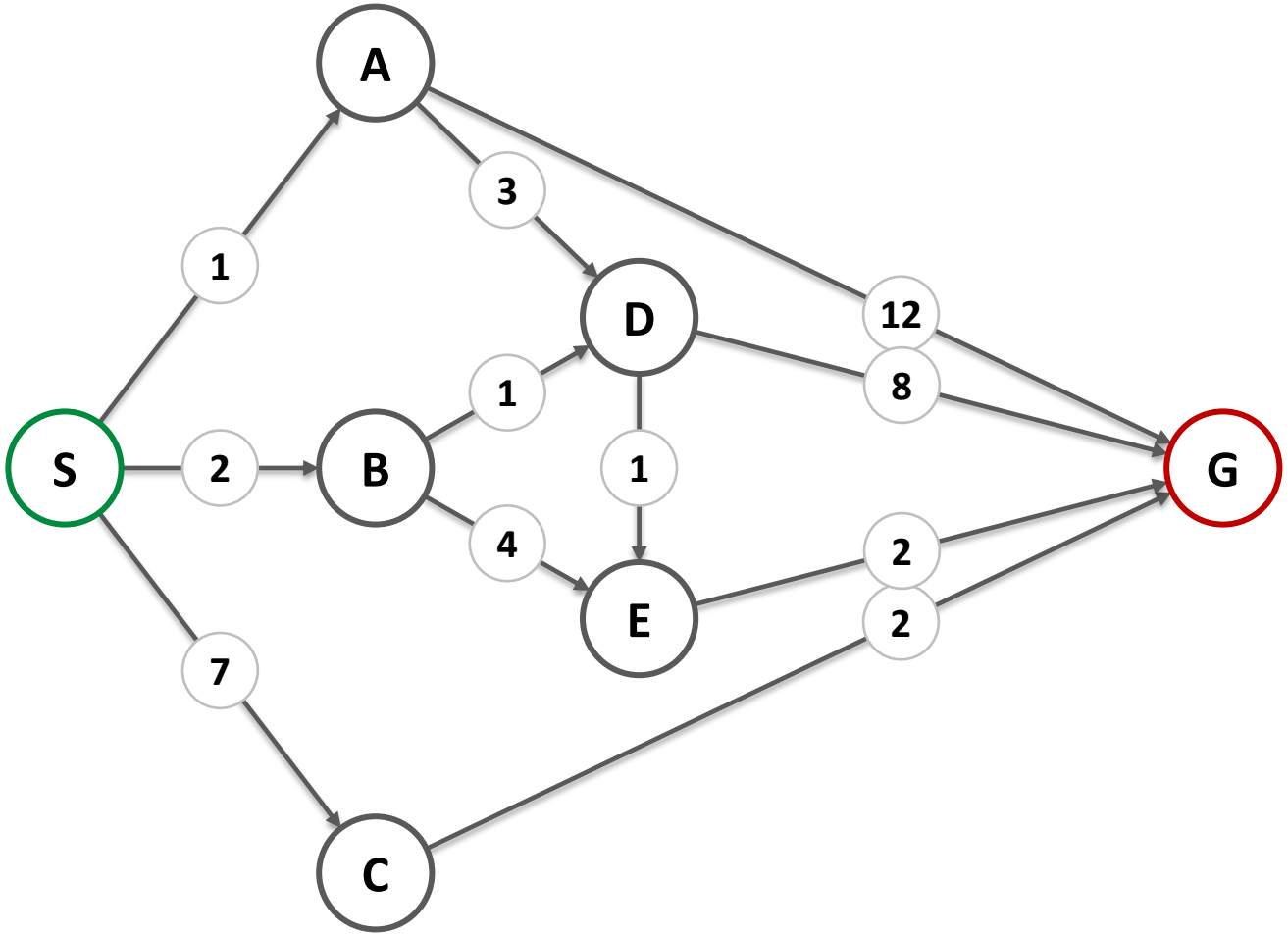








Your Turn: Practice Problem



This graph is **directed**: follow the arrows.

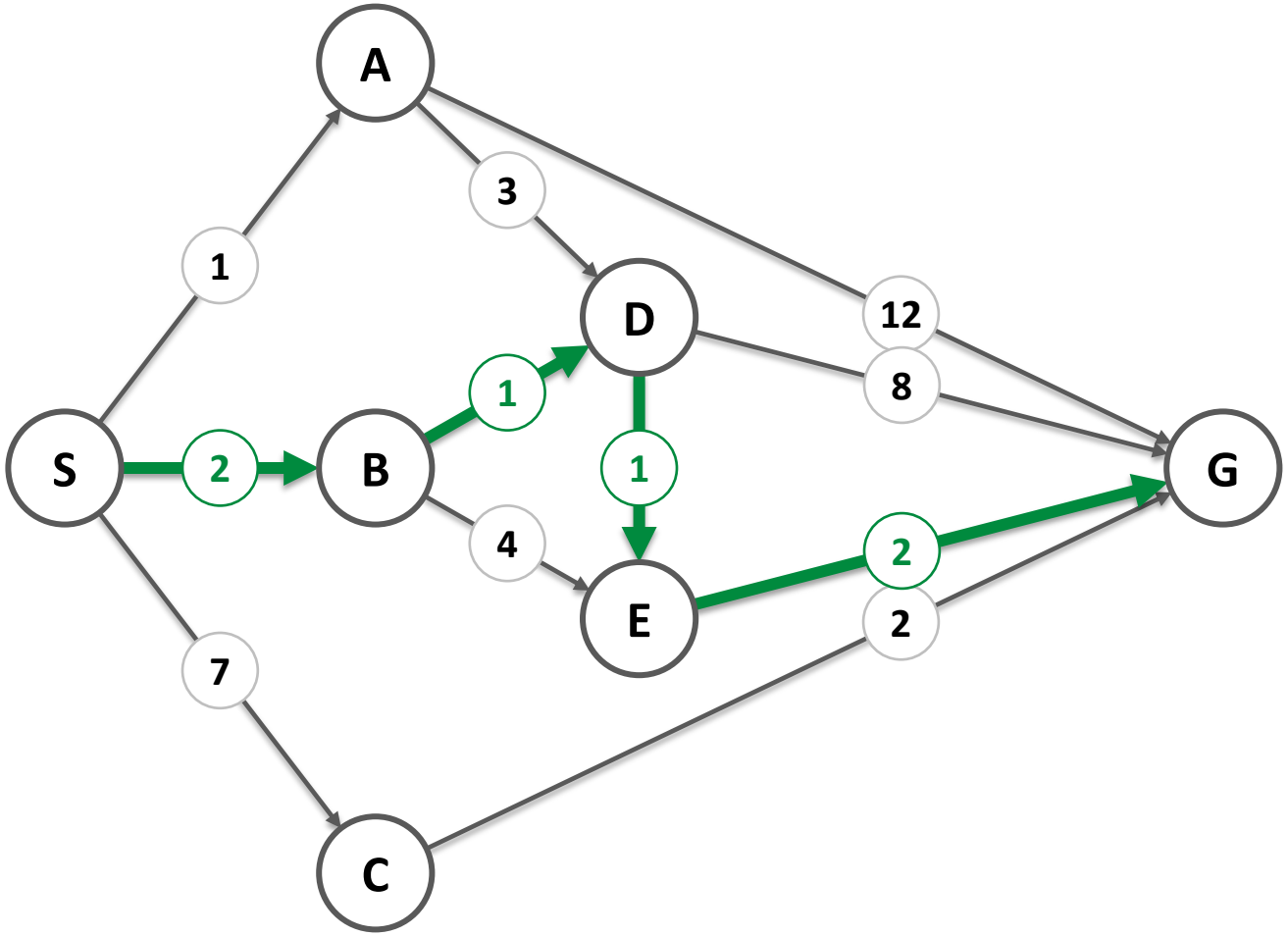
For **DFS, BFS, and UCS** (graph search, ties alphabetical, goal test on pop):

1. In what order are states expanded?

2. Which path is returned, and what does it cost?

Work in pairs for 10 minutes, then we compare answers.

Practice Problem: Solution



DFS

Expanded: S, A, D, E, G

Path: S → A → D → E → G cost 7

BFS

Expanded: S, A, B, C, D, G

Path: S → A → G cost 13

UCS

Expanded: S, A, B, D, E, G

Path: S → B → D → E → G cost 6

Only UCS finds the cheapest path (green).

Homework 0: Search

Released today in the course repo: hw/hw0. **Due Saturday, 21 November 2026, 11:59 PM** Guyana time.

Covers Weeks 2 and 3: problem formulation, uninformed search, and informed search (A* and heuristics).

A. Pac-Man Search 25 pts

Implement DFS, BFS, UCS, A*, and heuristics (autograded Q1–Q8). Q1–Q3 use today's material.

B. Written Problems 25 pts

Search theory, proofs about heuristics, and a hand trace on the Guyana road map.

C. Route Planning in Guyana 30 pts

Model a real Guyanese road network as a search problem, then run experiments (autograded Q9–Q10).

D. Search vs. LLMs 15 pts

Give the same problems to an LLM and compare its answers with A*.

Plus 5 pts for submission quality: Git history, PR description, and GenAI log. Start with Q1–Q3 now. Setting up Python and Git takes time.

Getting Started and Submitting HW0

- **Fork** ChrisIsKing/MAI5100 and clone your fork. You need Python 3.10+ and Git
- Copy the starter code into your own folder:
  - hw/hw0/submissions/firstname_lastname/src and written.md
- Check your setup: python autograder.py --no-graphics (0 points at first is expected)
- **Commit often** with clear messages. Your Git history is graded
- Open a **pull request** titled “HW0: Firstname Lastname” and fill in the PR template
  - Autograder output, checklist, and reflection questions
- Generative AI may be used as a collaborator, but **log every use** in written.md

Before Next Class (14 November)

- Read Russell & Norvig **Chapter 3.1–3.4**; skim 3.5 for a preview
- Set up HW0 and finish **Part A, Q1–Q3**: DFS, BFS, and UCS in Pac-Man
- Redo today's practice problem on your own
- **Next lecture:** informed search. Can we use knowledge about where the goal is to search faster?
  - Heuristics, greedy search, and A*

Acknowledgements

- Slides adapted from **UC Berkeley CS 188: Introduction to Artificial Intelligence** (Dan Klein, Pieter Abbeel, and the CS 188 staff)
- Worked example graph from CS 188 Fall 2026, Discussion 1
- Homework 0 Part A is adapted from the CS 188 Pac-Man search project